

Financial Analysis Prompt Library

Comprehensive Translation, Config Coverage, and Dependency Map

financial-services-plugins repository

Generated on 2026-05-12 10:03

Contents

1	Repository Structure Overview	2
1.1	Inventory Summary	2
1.2	Folder Tree	2
2	Prompt Dependency Structure	4
2.1	Command-to-Skill Dependencies	4
2.2	Skill-to-Resource Dependencies	4
3	Translated Prompt Corpus	6
3.1	Directory: .claude-plugin	6
3.2	Plugin Manifest: .claude-plugin	6
3.3	Directory: .mcp.json	6
3.4	MCP Configuration:	6
3.5	Directory: commands	9
3.6	Command: 3-statement-model.md	9
3.7	Command: competitive-analysis.md	9
3.8	Command: comps.md	9
3.9	Command: dcf.md	12
3.10	Command: debug-model.md	15
3.11	Command: lbo.md	15
3.12	Command: ppt-template.md	15
3.13	Directory: hooks	16
3.14	Config File: hooks/hooks.json	16
3.15	Directory: skills	16

3.16 Skill Resource: skills/3-statement-model/references/formatting.md	16
3.17 Skill Resource: skills/3-statement-model/references/formulas.md	19
3.18 Skill Resource: skills/3-statement-model/references/sec-filings.md	27
3.19 Skill: 3-statement-model	30
3.20 Skill: audit-xls	41
3.21 Skill: clean-data-xls	45
3.22 Skill Resource: skills/competitive-analysis/references/frameworks.md	46
3.23 Skill Resource: skills/competitive-analysis/references/schemas.md	47
3.24 Skill: competitive-analysis	47
3.25 Skill: comps-analysis	55
3.26 Skill Resource: skills/dcf-model/requirements.txt	71
3.27 Script File: skills/dcf-model/scripts/validate_dcf.py	71
3.28 Skill: dcf-model	79
3.29 Skill Resource: skills/dcf-model/TROUBLESHOOTING.md	111
3.30 Skill: deck-refresh	112
3.31 Skill Resource: skills/ib-check-deck/references/ib-terminology.md	115
3.32 Skill Resource: skills/ib-check-deck/references/report-format.md	116
3.33 Script File: skills/ib-check-deck/scripts/extract_numbers.py	118
3.34 Skill: ib-check-deck	126
3.35 Skill: lbo-model	128
3.36 Skill: ppt-template-creator	137
3.37 Skill: pptx-author	142
3.38 Skill Resource: skills/skill-creator/references/output-patterns.md	143
3.39 Skill Resource: skills/skill-creator/references/workflows.md	145
3.40 Script File: skills/skill-creator/scripts/init_skill.py	145
3.41 Script File: skills/skill-creator/scripts/package_skill.py	153
3.42 Script File: skills/skill-creator/scripts/quick_validate.py	156
3.43 Skill: skill-creator	159
3.44 Skill: xlsx-author	168

1 Repository Structure Overview

1.1 Inventory Summary

Metric	Count
Total included files	39
Markdown or markdown-like files	30
JSON config files	3
YAML manifest files	0
Scripts	5
Other text files	1
Commands	7
Skills	13
Plugin manifests	1
Managed-agent manifests	0

1.2 Folder Tree

Root directory: plugins/vertical-plugins/financial-analysis

- .claude-plugin
- .mcp.json
- commands
- hooks
- skills
- .claude-plugin/plugin.json
- commands/3-statement-model.md
- commands/competitive-analysis.md
- commands/comps.md
- commands/DCF.md
- commands/debug-model.md
- commands/lbo.md
- commands/ppt-template.md
- hooks/hooks.json
- skills/3-statement-model
- skills/audit-xls
- skills/clean-data-xls
- skills/competitive-analysis

- skills/comps-analysis
- skills/dcf-model
- skills/deck-refresh
- skills/ib-check-deck
- skills/lbo-model
- skills/ppt-template-creator
- skills/pptx-author
- skills/skill-creator
- skills/xlsx-author
- skills/3-statement-model/references
- skills/3-statement-model/SKILL.md
- skills/audit-xls/SKILL.md
- skills/clean-data-xls/SKILL.md
- skills/competitive-analysis/references
- skills/competitive-analysis/SKILL.md
- skills/comps-analysis/SKILL.md
- skills/dcf-model/requirements.txt
- skills/dcf-model/scripts
- skills/dcf-model/SKILL.md
- skills/dcf-model/TROUBLESHOOTING.md
- skills/deck-refresh/SKILL.md
- skills/ib-check-deck/references
- skills/ib-check-deck/scripts
- skills/ib-check-deck/SKILL.md
- skills/lbo-model/SKILL.md
- skills/ppt-template-creator/SKILL.md
- skills/pptx-author/SKILL.md
- skills/skill-creator/references
- skills/skill-creator/scripts
- skills/skill-creator/SKILL.md
- skills/xlsx-author/SKILL.md
- skills/3-statement-model/references/formatting.md
- skills/3-statement-model/references/formulas.md
- skills/3-statement-model/references/sec-filings.md

- skills/competitive-analysis/references/frameworks.md
- skills/competitive-analysis/references/schemas.md
- skills/DCF-model/scripts/validate_dcf.py
- skills/ib-check-deck/references/ib-terminology.md
- skills/ib-check-deck/references/report-format.md
- skills/ib-check-deck/scripts/extract_numbers.py
- skills/skill-creator/references/output-patterns.md
- skills/skill-creator/references/workflows.md
- skills/skill-creator/scripts/init_skill.py
- skills/skill-creator/scripts/package_skill.py
- skills/skill-creator/scripts/quick_validate.py

2 Prompt Dependency Structure

2.1 Command-to-Skill Dependencies

Command Prompt	Loaded Skill Prompt
commands/ 3-statement-model.md	skills/ 3-statement-model/ SKILL.md
commands/ competitive-analysis.md	skills/ competitive-analysis/ SKILL.md
commands/ comps.md	skills/ comps-analysis/ SKILL.md
commands/ dcf.md	skills/ comps-analysis/ SKILL.md
commands/ dcf.md	skills/ dcf-model/ SKILL.md
commands/ debug-model.md	skills/ audit-xls/ SKILL.md
commands/ lbo.md	skills/ lbo-model/ SKILL.md
commands/ ppt-template.md	skills/ full/ SKILL.md
commands/ ppt-template.md	skills/ ppt-template-creator/ SKILL.md

2.2 Skill-to-Resource Dependencies

Skill Prompt	Referenced Prompt File
skills/ 3-statement-model/ SKILL.md	skills/ 3-statement-model/ references/ formulas.md
skills/ 3-statement-model/ SKILL.md	skills/ 3-statement-model/ references/ sec-filings.md
skills/ competitive-analysis/ SKILL.md	skills/ competitive-analysis/ references/ frameworks.md
skills/ competitive-analysis/ SKILL.md	skills/ competitive-analysis/ references/ schemas.md
skills/ dcf-model/ SKILL.md	skills/ dcf-model/ TROUBLESHOOTING.md
skills/ ib-check-deck/ SKILL.md	skills/ ib-check-deck/ references/ ib-terminology.md
skills/ ib-check-deck/ SKILL.md	skills/ ib-check-deck/ references/ report-format.md
skills/ skill-creator/ SKILL.md	skills/ skill-creator/ DOCX-JS.md

skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md
skills/ skill-creator/ SKILL.md

skills/ skill-creator/ EXAMPLES.md
skills/ skill-creator/ FORMS.md
skills/ skill-creator/ OOXML.md
skills/ skill-creator/ REDLINING.md
skills/ skill-creator/ REFERENCE.md
skills/ skill-creator/ references/ api_docs.md
skills/ skill-creator/ references/ finance.md
skills/ skill-creator/ references/ mnda.md
skills/ skill-creator/ references/ output-patterns.md
skills/ skill-creator/ references/ policies.md
skills/ skill-creator/ references/ schema.md
skills/ skill-creator/ references/ workflows.md

3 Translated Prompt Corpus

This section translates the prompt, manifest, configuration, and script files under plugins/vertical-plugins/financial-analysis into a print-oriented format that preserves structure while improving readability.

3.1 Directory: .claude-plugin

3.2 Plugin Manifest: .claude-plugin

JSON Summary

Key	Type	Summary
name	str	financial-analysis
version	str	0.1.0
description	str	Core financial modeling and analysis tools: DCF, comps, LBO, 3-statement models, competitive analysis, and deck QC
author	dict	object with 1 key(s)

Structured JSON Content

- {
- “name”: “financial-analysis”,
- “version”: “0.1.0”,
- “description”: “Core financial modeling and analysis tools: DCF, comps, LBO, 3-statement models, competitive analysis, and deck QC”,
- “author”: {
- “name”: “Anthropic FSI”
- }
- }

3.3 Directory: .mcp.json

3.4 MCP Configuration: .

JSON Summary

Key	Type	Summary
mcpServers	dict	object with 11 key(s)

MCP Server Inventory

Connector	Type	URL
daloopa	http	https://mcp.daloopa.com/server/mcp
morningstar	http	https://mcp.morningstar.com/mcp
sp-global	http	https://kfinance.kensho.com/integrations/mcp
factset	http	https://mcp.factset.com/mcp
moodys	http	https://api.moodys.com/genai-ready-data/ml/mcp
mtnewswire	http	https://vast-mcp.blueskyapi.com/mtnewswires
aiera	http	https://mcp-pub.aiera.com
lseg	http	https://api.analytics.lseg.com/lfa/mcp
pitchbook	http	https://premium.mcp.pitchbook.com/mcp
chronograph	http	https://ai.chronograph.pe/mcp
egnyte	http	https://mcp-server.egnyte.com/mcp

Structured JSON Content

- {
- "mcpServers": {
- "daloopa": {
- "type": "http",
- "url": "https://mcp.daloopa.com/server/mcp"
- },
- "morningstar": {
- "type": "http",
- "url": "https://mcp.morningstar.com/mcp"
- },
- "sp-global": {
- "type": "http",
- "url": "https://kfinance.kensho.com/integrations/mcp"
- },
- "factset": {
- "type": "http",
- "url": "https://mcp.factset.com/mcp"
- },

- “moody’s”: {
 - “type”: “http”,
 - “url”: “https://api.moody’s.com/genai-ready-data/m1/mcp”
 - },
 - “mtnewswire”: {
 - “type”: “http”,
 - “url”: “https://vast-mcp.blueskyapi.com/mtnewswires”
 - },
 - “aiera”: {
 - “type”: “http”,
 - “url”: “https://mcp-pub.aiera.com”
 - },
 - “lseg”: {
 - “type”: “http”,
 - “url”: “https://api.analytics.lseg.com/lfa/mcp”
 - },
 - “pitchbook”: {
 - “type”: “http”,
 - “url”: “https://premium.mcp.pitchbook.com/mcp”
 - },
 - “chronograph”: {
 - “type”: “http”,
 - “url”: “https://ai.chronograph.pe/mcp”
 - },
 - “egnyte”: {
 - “type”: “http”,
 - “url”: “https://mcp-server.egnyte.com/mcp”
 - }
 - }
 - }
-

3.5 Directory: commands

3.6 Command: 3-statement-model.md

File Metadata

Field	Value
description	Fill out a 3-statement financial model template
argument-hint	[path to template file]

Load the *3-statement-model* skill and populate a 3-statement financial model (Income Statement, Balance Sheet, Cash Flow Statement).

If a file path is provided, use it as the template. Otherwise ask the user for their model template.

3.7 Command: competitive-analysis.md

File Metadata

Field	Value
description	Create a competitive landscape analysis
argument-hint	[company or industry]

Load the *competitive-analysis* skill and build a competitive landscape analysis for the specified company or industry.

If a company/industry is provided as an argument, use it. Otherwise ask the user what they want to analyze.

3.8 Command: comps.md

File Metadata

Field	Value
description	Build a comparable company analysis with trading multiples
argument-hint	[company name or ticker]

Comparable Company Analysis Command

Build an institutional-grade comparable company analysis with operating metrics, valuation multiples, and statistical benchmarking.

Workflow

Step 1: Gather Company Information

If a company name or ticker is provided, use it. Otherwise ask:

- “What company would you like to analyze?”

Step 2: Load Comps Analysis Skill

Use *skill*: “*comps-analysis*” to build the analysis:

1. Clarify the analysis purpose:

- “What’s the key question?” (valuation, efficiency, growth comparison)
- “Who is the audience?” (IC, board, quick reference)
- “Do you have a preferred format or template?”

2. Identify peer group (4-6 comparable companies):

- Similar business model
- Similar scale/market cap range
- Same industry/sector
- Geographic comparability

3. Gather data (prioritize MCP sources if available):

- Operating metrics: Revenue, Growth, Gross Margin, EBITDA, EBITDA Margin
- Valuation: Market Cap, Enterprise Value, EV/Revenue, EV/EBITDA, P/E
- Additional metrics based on industry (Rule of 40 for SaaS, etc.)

4. Build the analysis:

- Operating Statistics section with company data + statistics (Max, 75th, Median, 25th, Min)
- Valuation Multiples section with same statistical summary
- Notes & Methodology documentation

Step 3: Create Excel Output

Generate Excel file with:

- Header block (analysis title, companies, date, units)
- Operating Statistics & Financial Metrics section
- Valuation Multiples section
- Statistical summary for each metric
- Notes section documenting sources and methodology

Step 4: Deliver Output

Provide:

1. **Excel file** (.xlsx) - the comps analysis
2. **Summary** highlighting:
 - Peer group selection rationale
 - Key insights (who trades at premium/discount)
 - Median multiples for reference

Output Format Reference

Structured Template

- [SECTOR] - COMPARABLE COMPANY ANALYSIS
- [Company 1] • [Company 2] • [Company 3] • [Company 4]
- As of [Date] | All figures in USD Millions
- OPERATING STATISTICS & FINANCIAL METRICS
- Company Revenue Growth Gross EBITDA EBITDA
- (LTM) (YoY) Margin (LTM) Margin
- [Data rows for each company]
- Maximum =MAX =MAX =MAX =MAX =MAX
- 75th % =QUART =QUART =QUART =QUART =QUART
- Median =MEDIAN =MEDIAN =MEDIAN =MEDIAN =MEDIAN
- 25th % =QUART =QUART =QUART =QUART =QUART
- Minimum =MIN =MIN =MIN =MIN =MIN
- VALUATION MULTIPLES
- Company Mkt Cap EV EV/Rev EV/EBITDA P/E
- [Data rows + statistics]

Industry-Specific Metrics

Industry	Additional Metrics
Software/ SaaS	ARR, Net Dollar Retention, Rule of 40
Retail	Same-store sales, Inventory Turns
Financials	ROE, ROA, Efficiency Ratio
Manufacturing	Asset Turnover, CapEx/ Revenue
Healthcare	R&D/ Revenue, Pipeline Value

Quality Checklist

Before delivery:

- ☐ 4-6 truly comparable companies
 - ☐ Consistent time periods (all LTM or all FY)
 - ☐ All formulas reference cells (no hardcoded values)
 - ☐ Cell comments on all hardcoded inputs with sources
 - ☐ Statistics include Max, 75th, Median, 25th, Min
 - ☐ Notes section documents sources and methodology
 - ☐ Blue = inputs, Black = formulas
 - ☐ Sanity checks pass (margins logical, multiples reasonable)
-

3.9 Command: dcf.md

File Metadata

Field	Value
description	Build a DCF valuation model with comps-informed terminal multiples
argument-hint	[company name or ticker]

DCF Valuation Command

Build an institutional-quality DCF model that uses comparable company analysis to inform valuation ranges.

Workflow

Step 1: Gather Company Information

If a company name or ticker is provided, use it. Otherwise ask:

- “What company would you like to value?”

Step 2: Run Comparable Company Analysis

First, load the comps-analysis skill to build trading comps:

Use *skill*: “*comps-analysis*” to:

1. Identify 4-6 comparable public companies
2. Pull operating metrics (Revenue, EBITDA, margins, growth)

3. Pull valuation multiples (EV/Revenue, EV/EBITDA, P/E)
4. Calculate statistical summary (median, 25th/75th percentiles)

Key outputs to capture from comps:

- Median EV/EBITDA multiple → informs terminal value exit multiple
- Median EV/Revenue multiple → sanity check on DCF output
- Peer growth rates → benchmark for revenue projections
- Peer margins → benchmark for margin assumptions

Step 3: Build DCF Model

Load the dcf-model skill to construct the valuation:

Use *skill*: “*dcf-model*” to:

1. Gather historical financials and market data
2. Build revenue projections (Bear/Base/Bull cases)
3. Model operating expenses and FCF
4. Calculate WACC using CAPM
5. Discount cash flows and calculate terminal value
6. Bridge to equity value and implied share price

Use comps to inform DCF assumptions:

Comps Output	DCF Input
Peer median EV/ EBITDA	Terminal exit multiple range
Peer 25th-75th EV/ EBITDA	Sensitivity analysis range
Peer median growth rate	Benchmark for revenue assumptions
Peer median EBITDA margin	Target margin in terminal year
Peer median P/ E	Cross-check implied P/ E from DCF

Step 4: Cross-Check Valuation

After DCF is complete, validate:

1. **Implied EV/EBITDA** from DCF vs peer median
 - If DCF implies 25x but peers trade at 12x, investigate why
2. **Implied P/E** from DCF vs peer median
3. **Terminal value as % of EV** (should be 50-70%)
4. **Implied growth** embedded in valuation vs peer growth rates

Step 5: Deliver Output

Provide:

1. **Comps analysis spreadsheet** (.xlsx) with peer trading multiples
2. **DCF model** (.xlsx) with:
 - Bear/Base/Bull scenarios
 - Sensitivity tables (WACC vs Terminal Growth, etc.)
 - Valuation summary with implied upside/downside
3. **Summary** explaining:
 - Key valuation drivers
 - How comps informed the analysis
 - Risks and sensitivities to watch

Example Output Summary

Structured Template

- VALUATION SUMMARY: [Company] ([Ticker])
 - Comparable Companies Analysis:
 - - Peer Group: [List of 4-6 comps]
 - - Median EV/EBITDA: 12.5x (range: 10.2x - 15.8x)
 - - Median EV/Revenue: 3.2x (range: 2.1x - 4.5x)
 - DCF Valuation (Base Case):
 - - Implied Share Price: \$XX.XX
 - - Current Price: \$YY.YY
 - - Implied Upside: +XX%
 - Valuation Cross-Check:
 - - DCF Implied EV/EBITDA: 13.2x (vs peer median 12.5x)
 - - DCF Implied P/E: 22.4x (vs peer median 20.1x)
 - - Terminal Value: 62% of EV (within normal range)
 - Key Assumptions:
 - - Revenue CAGR: X% (vs peer median X%)
 - - Terminal EBITDA Margin: X% (vs peer median X%)
 - - WACC: X.X%
 - - Terminal Growth: X.X%
-

3.10 Command: debug-model.md

File Metadata

Field	Value
description	Debug and audit a financial model for errors
argument-hint	[path to .xlsx model file]

Load the *audit-xls* skill with scope **model** and audit the specified financial model for broken formulas, balance sheet imbalances, hardcoded overrides, circular references, and logic errors — including the full model-integrity checks (BS balance, cash tie-out, roll-forwards, model-type-specific bugs).

If a file path is provided, use it. Otherwise ask the user for the model to review.

3.11 Command: lbo.md

File Metadata

Field	Value
description	Build an LBO model for a PE acquisition
argument-hint	[company name or deal details]

Load the *lbo-model* skill and build a leveraged buyout model for the specified company or deal.

If a company name is provided as an argument, use it. Otherwise ask the user for the target company and deal parameters.

3.12 Command: ppt-template.md

File Metadata

Field	Value
description	Create a reusable PPT template skill from a PowerPoint template file
argument-hint	[path to .pptx or .potx file]
allowed-tools	["Read", "Write", "Bash", "Glob"]

PPT Template Creator Command

Create a self-contained PPT template skill from a user-provided PowerPoint template.

Instructions

1. **Ask for the template file** if not provided:
 - “Please provide the path to your PowerPoint template file (.pptx or .potx)”
 - The template should contain the slide layouts and branding you want to use
 2. **Load the ppt-template-creator skill:**
 - Use the *skill*: “*ppt-template-creator*” tool to load the full skill instructions
 - Follow the workflow in the skill to analyze the template and generate a new skill
 3. **Gather additional info:**
 - Company/template name (for naming the skill)
 - Primary use cases (pitch decks, board materials, client presentations, etc.)
 4. **Execute the skill workflow:**
 - Analyze template structure (layouts, placeholders, dimensions)
 - Generate skill directory with assets/ and SKILL.md
 - Create example presentation to validate
 - Package the skill
 5. **Deliver the packaged skill** to the user
-

3.13 Directory: hooks

3.14 Config File: hooks/hooks.json

JSON Summary

Item	Type	Summary
------	------	---------

Structured JSON Content

- []
-

3.15 Directory: skills

3.16 Skill Resource: skills/3-statement-model/references/formatting.md

Formatting Standards Reference

Element	Format
Hard-coded inputs	Blue font

Formulas	Black font
Links to other sheets	Green font
Check cells	Red if error, green if balanced
Negative values	Parentheses, not minus signs
Currency	No decimals for large figures, 2 decimals for per-share
Percentages	1 decimal place
Headers	Bold, bottom border
Units row	Include units row below headers (\$ millions, %, etc.)

Visual Separation Guidelines

- Thin vertical border between historical and projected columns
- Thick bottom border after section totals (e.g., Total Assets)
- Single bottom border for subtotals
- Double bottom border for grand totals

Total and Subtotal Row Formatting

All total and subtotal rows must use **bold font formatting** for their numerical values to clearly distinguish aggregated figures from individual line items.

Income Statement (P&L) Tab

Row	Formatting
Gross Revenue	Bold
Total Cost of Revenue	Bold
Gross Profit	Bold
Total SG&A	Bold
EBITDA	Bold
EBIT	Bold
EBT	Bold
Net Profit After Tax	Bold

Balance Sheet Tab

Row	Formatting
Total Current Assets	Bold
Total Non-Current Assets	Bold
Total Other Assets	Bold
Total Assets	Bold
Total Current Liabilities	Bold
Total Non-Current Liabilities	Bold

Total Equity	Bold
Total Liabilities and Equity	Bold

Cash Flow Statement Tab

Row	Formatting
Cash Generated from Operations Before Working Capital Changes	Bold
Total Working Capital Changes	Bold
Net Cash Generated from Operations	Bold
Net Cash Flow from Investing Activities	Bold
Net Cash Flow from Financing Activities	Bold
Closing Cash Balance	Bold

Note: This list is non-exhaustive. Apply bold formatting to any row that represents a total, subtotal, or summary calculation across the model.

Balance Sheet Check Row Formatting

The Balance Sheet check row (below Total Liabilities and Equity) uses conditional number formatting that displays non-zero values in red. When the balance sheet balances correctly (check = 0), the values display in black or standard formatting.

Check Value	Font Color
= 0 (balanced)	Black (standard)
≠ 0 (error)	Red

Implementation: Apply custom number format *[Red][<>0]0.00;[Red]<>0;0.00* or use Excel conditional formatting with the rule “Cell Value ≠ 0” → Red font.

Margin Row Formatting

Element	Format
Margin % rows	Indent, italics, 1 decimal place
Positive trend	No special formatting (or subtle green)
Negative trend	Flag for review (subtle yellow)
Below peer average	Consider highlighting for discussion

Credit Metric Formatting

Element	Format
---------	--------

Leverage multiples	1 decimal with “x” suffix (e.g., 2.5x)
Percentages	1 decimal with “%” suffix
Net Debt negative	Parentheses, indicates net cash position
Section header	Bold, “CREDIT METRICS”
Separator line	Thin border above credit metrics section

Credit Metric Threshold Colors

Metric	Green	Yellow	Red
Total Debt / EBITDA	< 2.5x	2.5x-4.0x	> 4.0x
Net Debt / EBITDA	< 2.0x	2.0x-3.5x	> 3.5x
Interest Coverage	> 4.0x	2.5x-4.0x	< 2.5x
Debt / Total Cap	< 40%	40%-60%	> 60%
Current Ratio	> 1.5x	1.0x-1.5x	< 1.0x
Quick Ratio	> 1.0x	0.75x-1.0x	< 0.75x

Conditional Formatting for Checks Tab

- Cell contains pass indicator → Green fill
- Cell contains fail indicator → Red fill
- Cell contains warning → Yellow fill
- Difference cells = 0 → Light green fill
- Difference cells ≠ 0 → Light red fill

Margin Reasonability Flags

- Gross Margin < 0% → ERROR: Review COGS
- Gross Margin > 80% → WARNING: Verify revenue/COGS
- EBITDA Margin < 0% → FLAG: Operating losses
- EBITDA Margin > 50% → WARNING: Unusually high
- Net Margin < 0% → FLAG: Net losses (may be acceptable in growth phase)
- Net Margin > Gross Margin → ERROR: Formula issue

3.17 Skill Resource: [skills/3-statement-model/references/formulas.md](#)

Formula Reference

IMPORTANT: Use the formulas outlined in this reference document unless otherwise specified by the user.

Core Linkages

Structured Template

- Balance Sheet: $\text{Assets} = \text{Liabilities} + \text{Equity}$
- Net Income: $\text{IS Net Income} \rightarrow \text{CF Operations (starting point)}$
- Cash Flow: $\Delta\text{Cash} = \text{CFO} + \text{CFI} + \text{CFF}$
- Cash Tie-Out: $\text{Ending Cash (CF)} = \text{Cash (BS Asset)}$
- Cash Monthly/Annual: $\text{Closing Cash (Monthly)} = \text{Closing Cash (Annual)}$
- Retained Earnings: $\text{Prior RE} + \text{Net Income} - \text{Dividends} = \text{Ending RE}$
- Equity Raise: $\Delta\text{Common Stock/APIC (BS)} = \text{Equity Issuance (CFF)}$
- Year 0 Equity: $\text{Equity Raised (Year 0)} = \text{Beginning Equity (Year 1)}$

Gross Profit Calculation

IMPORTANT: Gross Profit must be calculated from Net Revenue, not Gross Revenue.

Structured Template

- $\text{Net Revenue} - \text{Cost of Revenue} = \text{Gross Profit}$

Term	Definition
Gross Revenue	Total revenue before any deductions
Net Revenue	$\text{Gross Revenue} - \text{Returns} - \text{Allowances} - \text{Discounts}$
Cost of Revenue	Direct costs attributable to production of goods/ services sold
Gross Profit	$\text{Net Revenue} - \text{Cost of Revenue}$

Note: Always use Net Revenue (also called “Net Sales” or simply “Revenue” on most financial statements) as the starting point for profitability calculations. Gross Revenue overstates the true top-line performance.

Margin Formulas

Structured Template

- $\text{Gross Margin \%} = \text{Gross Profit} / \text{Net Revenue}$
- $\text{EBITDA} = \text{EBIT} + \text{D\&A (or = Gross Profit - OpEx)}$
- $\text{EBITDA Margin \%} = \text{EBITDA} / \text{Net Revenue}$
- $\text{EBIT Margin \%} = \text{EBIT} / \text{Net Revenue}$
- $\text{Net Income Margin \%} = \text{Net Income} / \text{Net Revenue}$

Credit Metric Formulas

Structured Template

- $\text{Total Debt} = \text{Current Portion of Debt} + \text{Long-Term Debt}$
- $\text{Net Debt} = \text{Total Debt} - \text{Cash}$
- $\text{Total Debt} / \text{EBITDA} = \text{Total Debt} / \text{EBITDA (from IS)}$
- $\text{Net Debt} / \text{EBITDA} = \text{Net Debt} / \text{EBITDA (from IS)}$
- $\text{Interest Coverage} = \text{EBITDA} / \text{Interest Expense (from IS)}$
- $\text{Net Int Exp \% Debt} = \text{Net Interest Expense} / \text{Long-Term Debt}$
- $\text{Debt} / \text{Total Cap} = \text{Total Debt} / (\text{Total Debt} + \text{Total Equity})$
- $\text{Debt} / \text{Equity} = \text{Total Debt} / \text{Total Equity}$
- $\text{Current Ratio} = \text{Total Current Assets} / \text{Total Current Liabilities}$
- $\text{Quick Ratio} = (\text{Total Current Assets} - \text{Inventory}) / \text{Total Current Liabilities}$

Forecast Formulas (% of Net Revenue Method)

Structured Template

- $\text{Cost of Revenue (Forecast)} = \text{Net Revenue} \times \text{Cost of Revenue \% Assumption}$
- $\text{S\&M (Forecast)} = \text{Net Revenue} \times \text{S\&M \% Assumption}$
- $\text{G\&A (Forecast)} = \text{Net Revenue} \times \text{G\&A \% Assumption}$
- $\text{R\&D (Forecast)} = \text{Net Revenue} \times \text{R\&D \% Assumption}$
- $\text{SBC (Forecast)} = \text{Net Revenue} \times \text{SBC \% Assumption}$

Working Capital Formulas

Structured Template

- Accounts Receivable
- Prior AR
- + Revenue (from IS)
- - Cash Collections (plug)
- = Ending AR
- $\text{DSO} = (\text{AR} / \text{Revenue}) \times 365$
- Inventory
- Prior Inventory
- + Purchases (plug)
- - COGS (from IS)

- = Ending Inventory
- $DIO = (Inventory / COGS) \times 365$
- Accounts Payable
- Prior AP
- + Purchases (from Inventory calc)
- - Cash Payments (plug)
- = Ending AP
- $DPO = (AP / COGS) \times 365$
- Net Working Capital = AR + Inventory - AP
- $\Delta WC = \text{Current NWC} - \text{Prior NWC}$

D&A Schedule Formulas

Structured Template

- Beginning PP&E (Gross)
- + CapEx
- = Ending PP&E (Gross)
- Beginning Accumulated Depreciation
- + Depreciation Expense
- = Ending Accumulated Depreciation
- $PP\&E \text{ (Net)} = \text{Gross PP\&E} - \text{Accumulated Depreciation}$

Debt Schedule Formulas

Structured Template

- Beginning Debt Balance
- + New Borrowings
- - Repayments
- = Ending Debt Balance
- $\text{Interest Expense} = \text{Avg Debt Balance} \times \text{Interest Rate}$
- (Use beginning balance to avoid circularity, or iterate if circular refs enabled)

Retained Earnings Formula

Structured Template

- Beginning Retained Earnings

- + Net Income (from IS)
- + Stock-Based Compensation (SBC) (from IS)
- - Dividends
- = Ending Retained Earnings

NOL (Net Operating Loss) Schedule Formulas

Structured Template

- NOL CARRYFORWARD SCHEDULE
- Beginning NOL Balance (Year 1 / Formation = 0)
- + NOL Generated (if $EBT < 0$, then $ABS(EBT)$, else 0)
- - NOL Utilized (limited by taxable income and utilization cap)
- = Ending NOL Balance
- STARTING BALANCE RULE
- For a new business or first modeled period:
- Beginning NOL Balance = 0
- NOL can only increase through realized losses ($EBT < 0$)
- NOL cannot be created from thin air or assumed
- NOL UTILIZATION CALCULATION
- Pre-Tax Income (EBT)
- If $EBT > 0$:
- NOL Available = Beginning NOL Balance
- Utilization Limit = $EBT \times 80\%$ (post-2017 federal limit)
- NOL Utilized = $\text{MIN}(\text{NOL Available}, \text{Utilization Limit})$
- Taxable Income = $EBT - \text{NOL Utilized}$
- If $EBT \leq 0$:
- NOL Utilized = 0
- Taxable Income = 0
- NOL Generated = $ABS(EBT)$
- TAX CALCULATION WITH NOL
- Taxes Payable = $\text{MAX}(0, \text{Taxable Income} \times \text{Tax Rate})$
- (Taxes cannot be negative; losses create NOL asset instead)
- DEFERRED TAX ASSET (DTA) FOR NOL

- $\text{DTA} - \text{NOL Carryforward} = \text{Ending NOL Balance} \times \text{Tax Rate}$
- $\Delta \text{DTA} = \text{Current DTA} - \text{Prior DTA}$
- (Increase in DTA = non-cash benefit on CF)
- (Decrease in DTA = non-cash expense on CF)

Balance Sheet Structure

Structured Template

- ASSETS
- Cash (from CF ending cash)
- Accounts Receivable (from WC)
- Inventory (from WC)
- Total Current Assets
- PP&E, Net (from DA)
- Deferred Tax Asset - NOL (from NOL schedule)
- Total Non-Current Assets
- Total Assets
- LIABILITIES
- Accounts Payable (from WC)
- Current Portion of Debt (from Debt)
- Total Current Liabilities
- Long-Term Debt (from Debt)
- Total Liabilities
- EQUITY
- Common Stock
- Retained Earnings (from RE schedule)
- Total Equity
- CHECK: $\text{Assets} - \text{Liabilities} - \text{Equity} = 0$

Cash Flow Statement Structure

Structured Template

- CASH FROM OPERATIONS (CFO)
- Net Income (LINK: IS)
- + D&A (LINK: DA schedule)

- + Stock-Based Compensation (SBC) (LINK: IS or Assumptions)
- - Δ DTA (Deferred Tax Asset) (LINK: NOL schedule; increase in DTA = use of cash)
- - Δ AR (LINK: WC)
- - Δ Inventory (LINK: WC)
- + Δ AP (LINK: WC)
- = CFO
- CASH FROM INVESTING (CFI)
- - CapEx (LINK: DA schedule)
- = CFI
- CASH FROM FINANCING (CFF)
- + Debt Issuance (LINK: Debt)
- - Debt Repayment (LINK: Debt)
- + Equity Issuance (LINK: BS Common Stock/APIC)
- - Dividends (LINK: RE schedule)
- = CFF
- Net Change in Cash = CFO + CFI + CFF
- Beginning Cash
- + Net Change in Cash
- = Ending Cash (LINK TO: BS Cash)

Income Statement Structure

Structured Template

- Net Revenue
- Growth %
- (-) Cost of Revenue
- % of Net Revenue
- Gross Profit (= Net Revenue - Cost of Revenue)
- Gross Margin %
- (-) S&M
- % of Net Revenue
- (-) G&A
- % of Net Revenue

- (-) R&D
- % of Net Revenue
- (-) D&A
- (-) SBC
- % of Net Revenue
- EBIT
- EBIT Margin %
- EBITDA
- EBITDA Margin %
- (-) Interest Expense
- EBT (Pre-Tax Income)
- (-) NOL Utilization (from NOL schedule, reduces taxable income)
- Taxable Income
- (-) Taxes (Taxable Income $\times$ Tax Rate)
- Net Income
- Net Income Margin %

Check Formulas

Structured Template

- BS Balance Check: = Assets - Liabilities - Equity (must = 0)
 - Cash Tie-Out: = BS Cash - CF Ending Cash (must = 0)
 - RE Roll-Forward: = Prior RE + NI + SBC - Div - BS RE (must = 0)
 - DTA Tie-Out: = NOL Schedule DTA - BS DTA (must = 0)
 - Equity Raise Tie-Out: = Δ Common Stock/APIC (BS) - Equity Issuance (CFF) (must = 0)
 - Year 0 Equity Tie-Out: = Equity Raised (Year 0) - Beginning Equity (Year 1) (must = 0)
 - Cash Monthly vs Annual: = Closing Cash (Monthly) - Closing Cash (Annual) (must = 0)
 - NOL Utilization Cap: = NOL Utilized $\leq$ EBT $\times$ 80% (must be TRUE for post-2017)
 - NOL Non-Negative: = Ending NOL Balance $\geq$ 0 (must be TRUE)
 - NOL Starting Balance: = Beginning NOL (Year 1) = 0 (must be TRUE for new business)
 - NOL Accumulation: = NOL increases only when EBT < 0 (losses generate NOL)
-

3.18 Skill Resource: skills/3-statement-model/references/sec-filings.md

SEC Filings Data Extraction Reference

When to Use: Only reference this file when a model template specifically requires pulling data from SEC filings (10-K, 10-Q). For templates that provide data directly or use other data sources, this reference is not needed.

Extracting Data from SEC Filings (10-K / 10-Q)

When populating a model template with public company data, extract financials directly from SEC filings.

Step 1: Locate the Filing

1. Use SEC EDGAR: [https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany&CIK=\[TICKER\]&type=10-K](https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany&CIK=[TICKER]&type=10-K)
2. For quarterly data, use *type=10-Q*

Step 2: Identify Filing Currency

Before extracting data, identify the reporting currency:

- Check the cover page or header for reporting currency
- Look at statement headers (e.g., “in thousands of U.S. dollars”)
- Review Note 1 (Summary of Significant Accounting Policies)

Common Currency Indicators

Indicator	Currency
\$, USD	US Dollar
€, EUR	Euro
£, GBP	British Pound
¥, JPY	Japanese Yen
¥, CNY, RMB	Chinese Yuan
CHF	Swiss Franc
CAD, C\$	Canadian Dollar

Set model currency to match filing; document in Assumptions tab.

Step 3: Navigate to Financial Statements

Within the 10-K or 10-Q, locate:

- **Item 8** (10-K) or **Item 1** (10-Q): Financial Statements
- Key sections to extract:
 - Consolidated Statements of Operations (Income Statement)

- Consolidated Balance Sheets
- Consolidated Statements of Cash Flows
- Notes to Financial Statements (for schedule details)

Step 4: Data Extraction Mapping

Income Statement (from Consolidated Statements of Operations)

Filing Line Item	Model Line Item
Net revenues / Net sales	Revenue
Cost of goods sold	COGS
Selling, general and administrative	SG&A
Depreciation and amortization	D&A
Interest expense, net	Interest Expense
Income tax expense	Taxes
Net income	Net Income

Balance Sheet (from Consolidated Balance Sheets)

Filing Line Item	Model Line Item
Cash and cash equivalents	Cash
Accounts receivable, net	AR
Inventories	Inventory
Property, plant and equipment, net	PP&E (Net)
Total assets	Total Assets
Accounts payable	AP
Short-term debt / Current portion of LT debt	Current Debt
Long-term debt	LT Debt
Retained earnings	Retained Earnings
Total stockholders' equity	Total Equity

Cash Flow Statement (from Consolidated Statements of Cash Flows)

Filing Line Item	Model Line Item
Net income	Net Income
Depreciation and amortization	D&A
Changes in accounts receivable	Δ AR
Changes in inventories	Δ Inventory
Changes in accounts payable	Δ AP
Capital expenditures	CapEx
Proceeds from issuance of common stock	Equity Issuance
Proceeds from / Repayments of debt	Debt activity

Step 5: Extract Supporting Detail from Notes

For schedules, pull from Notes to Financial Statements:

- **Note: Debt** → Maturity schedule, interest rates, covenants
- **Note: Property, Plant & Equipment** → Gross PP&E, accumulated depreciation, useful lives
- **Note: Revenue** → Segment breakdowns, geographic splits
- **Note: Leases** → Operating vs. finance lease obligations

Step 6: Historical Data Requirements

Extract 3 years of historical data minimum:

- 10-K provides 3 years of IS/CF, 2 years of BS
- For 3rd year BS, pull from prior year's 10-K
- Use 10-Qs to fill in quarterly granularity if needed

Data Extraction Checklist

- Identify reporting currency and scale (thousands, millions)
- 3 years historical Income Statement
- 3 years historical Cash Flow Statement
- 3 years historical Balance Sheet
- Verify IS Net Income = CF starting Net Income (each year)
- Verify BS Cash = CF Ending Cash (each year)
- Extract debt maturity schedule from notes
- Extract D&A detail or useful life assumptions
- Note any non-recurring / one-time items to normalize

Handling Common Filing Variations

Variation	How to Handle
D&A embedded in COGS/ SG&A	Pull D&A from Cash Flow Statement
“Other” line items are material	Check notes for breakdown
Restatements	Use restated figures, note in assumptions
Fiscal year ≠ calendar year	Label with fiscal year end (e.g., FYE Jan 2025)
Non-USD reporting currency	Adapt model currency to match filing

3.19 Skill: 3-statement-model

File Metadata

Field	Value
name	3-statement-model
description	Complete, populate and fill out 3-statement financial model templates (Income Statement, Balance Sheet, Cash Flow Statement) . Use when asked to fill out model templates, complete existing model frameworks, populate financial models with data, complete a partially filled IS/ BS/ CF framework, or link integrated financial statements within an existing template structure. Triggers include requests to fill in, complete, or populate a 3-statement model template

3-Statement Financial Model Template Completion

Complete and populate integrated financial model templates with proper linkages between Income Statement, Balance Sheet, and Cash Flow Statement.

⚠ CRITICAL PRINCIPLES — Read Before Populating Any Template

Environment — Office JS vs Python:

- **If running inside Excel (Office Add-in / Office JS):** Use Office JS directly. Write formulas via *range.formulas* = `[[["=D14*(1+Assumptions!$B$5)"]]]` — never *range.values* for derived cells. No separate recalc; Excel computes natively. Use *context.workbook.worksheets.getItem(...)* to navigate tabs.
- **If generating a standalone .xlsx file:** Use Python/openpyxl. Write `ws["D15"] = "=D14*(1+Assumptions!$B$5)"`, then run *recalc.py* before delivery.
- **Office JS merged cell pitfall:** Do NOT call *.merge()* then set *.values* on the merged range — throws *InvalidArgument* because the range still reports its pre-merge dimensions. Instead write value to top-left cell alone, then merge + format the full range: `ws.getRange("A1").values = [[["INCOME STATEMENT"]]]; const h = ws.getRange("A1:G1"); h.merge(); h.format.fill.color = "#1F4E79";`
- All principles below apply identically in either environment.

Formulas over hardcodes (non-negotiable):

- Every projection cell, roll-forward, linkage, and subtotal MUST be an Excel formula — never a pre-computed value
- When using Python/openpyxl: write formula strings (`ws["D15"] = "=D14*(1+Assumptions!$B$5)"`), NOT computed results (`ws["D15"] = 12500`)
- The ONLY cells that should contain hardcoded numbers are: (1) historical actuals, (2) assumption drivers in the Assumptions tab
- If you find yourself computing a value in Python and writing the result to a cell — STOP. Write the formula instead.

- Why: the model must flex when scenarios toggle or assumptions change. Hardcodes break every downstream integrity check silently.

Verify step-by-step with the user:

1. **After mapping the template** → show the user which tabs/sections you've identified and confirm before touching any cells
2. **After populating historicals** → show the user the historical block and confirm values/periods match source data
3. **After building IS projections** → run the subtotal checks, show the user the projected IS, confirm before moving to BS
4. **After building BS** → show the user the balance check (Assets = L+E) for every period, confirm before moving to CF
5. **After building CF** → show the user the cash tie-out (CF ending cash = BS cash), confirm before finalizing
6. **Do NOT populate the entire model end-to-end and present it complete** — break at each statement, show the work, catch errors early

Formatting — Professional Blue/Grey Palette (Default unless template/user specifies otherwise)

Keep colors minimal. Use only blues and greys for cell fills. Do NOT introduce greens, yellows, oranges, or multiple accent colors — a clean model uses restraint.

Element	Fill	Font
Section headers (IS / BS / CF titles)	Dark blue #1F4E79	White bold
Column headers (FY2024A, FY2025E, Light blue #D9E1F2 etc.)		Black bold
Input cells (historicals, assumption drivers)	Light grey #F2F2F2 or white	Blue #0000FF
Formula cells	White	Black
Cross-tab links	White	Green #008000
Check rows / key totals	Medium blue #BDD7EE	Black bold

That's 3 blues + 1 grey + white. If the template has its own color scheme, follow the template instead.

Font color signals *what* a cell is (input/formula/link). Fill color signals *where* you are (header/data/check).

Model Structure

Identifying Template Tab Organization

Templates vary in their tab naming conventions and organization. Before populating, review all tabs to understand the template's structure. Below are common tab names and their typical contents:

Common Tab Names	Contents to Look For
------------------	----------------------

IS, P&L, Income Statement	Income Statement
BS, Balance Sheet	Balance Sheet
CF, CFS, Cash Flow	Cash Flow Statement
WC, Working Capital	Working Capital Schedule
DA, D&A, Depreciation, PP&E	Depreciation & Amortization Schedule
Debt, Debt Schedule	Debt Schedule
NOL, Tax, DTA	Net Operating Loss Schedule
Assumptions, Inputs, Drivers	Driver assumptions and inputs
Checks, Audit, Validation	Error-checking dashboard

Template Review Checklist

- Identify which tabs exist in the template (not all templates include every schedule)
- Note any template-specific tabs not listed above
- Understand tab dependencies (e.g., which schedules feed into the main statements)
- Locate input cells vs. formula cells on each tab

Understanding Template Structure

Before populating a template, familiarize yourself with its existing layout to ensure data is entered in the correct locations and formulas remain intact.

Identifying Row Structure

- Locate the model title at top of each tab
- Identify section headers and their visual separation
- Find the units row indicating \$ millions, %, x, etc.
- Note column headers distinguishing Actuals vs. Estimates periods
- Confirm period labels (e.g., FY2024A, FY2025E)
- Identify input cells vs. formula cells (typically distinguished by font color)

Identifying Column Structure

- Confirm line item labels in leftmost column
- Verify historical years precede projection years
- Note the visual border separating historical from projected periods
- Check for consistent column order across all tabs

Working with Named Ranges Templates often use named ranges for key inputs and outputs. Before entering data:

- Review existing named ranges in the template (Formulas → Name Manager in Excel)

- Common named ranges include: Revenue growth rates, cost percentages, key outputs (Net Income, EBITDA, Total Debt, Cash), scenario selector cell
- Ensure inputs are entered in cells that feed into these named ranges

Projection Period

- Templates typically project 5 years forward from last historical year
- Verify historical (A) vs. projected (E) columns are clearly separated
- Confirm columns use fiscal year notation (e.g., FY2024A, FY2025E)

Margin Analysis

Note: The following margin analysis should only be performed if prompted by the user or if the template explicitly requires it. If no prompt is given, skip this section.

Calculate and display profitability margins on the Income Statement (IS) tab to track operational efficiency and enable peer comparison.

Core Margins to Include

Margin	Formula	What It Measures
Gross Margin	Gross Profit / Revenue	Pricing power, production efficiency
EBITDA Margin	EBITDA / Revenue	Core operating profitability
EBIT Margin	EBIT / Revenue	Operating profitability after D&A
Net Income Margin	Net Income / Revenue	Bottom-line profitability

Income Statement Layout with Margins

Display margin percentages directly below each profit line item:

- Gross Margin % below Gross Profit
- EBIT Margin % below EBIT
- EBITDA Margin % below EBITDA
- Net Income Margin % below Net Income

Credit Metrics

Note: The following Credit analysis should only be performed if prompted by the user or if the template explicitly requires it. If no prompt is given, skip this section.

Calculate and display credit/leverage metrics on the Balance Sheet (BS) tab to assess financial health, debt capacity, and covenant compliance.

Core Credit Metrics to Include

Metric	Formula	What It Measures
Total Debt / EBITDA	Total Debt / LTM EBITDA	Leverage multiple
Net Debt / EBITDA	(Total Debt - Cash) / LTM EBITDA	Leverage net of cash
Interest Coverage	EBITDA / Interest Expense	Ability to service debt
Debt / Total Cap	Total Debt / (Total Debt + Equity)	Capital structure
Debt / Equity	Total Debt / Total Equity	Financial leverage
Current Ratio	Current Assets / Current Liabilities	Short-term liquidity
Quick Ratio	(Current Assets - Inventory) / Current Liabilities	Immediate liquidity

Credit Metric Hierarchy Checks

Validate that Upside shows strongest credit profile:

- Leverage: Upside < Base < Downside (lower is better)
- Coverage: Upside > Base > Downside (higher is better)
- Liquidity: Upside > Base > Downside (higher is better)

Covenant Compliance Tracking

If debt covenants are known, add explicit compliance checks comparing actual metrics to covenant thresholds.

Scenario Analysis (Base / Upside / Downside)

Use a scenario toggle (dropdown) in the Assumptions tab with CHOOSE or INDEX/MATCH formulas.

Scenario	Description
Base Case	Management guidance or consensus estimates
Upside Case	Above-guidance growth, margin expansion
Downside Case	Below-trend growth, margin compression

Key Drivers to Sensitize: Revenue growth, Gross margin, SG&A %, DSO/DIO/DPO, CapEx %, Interest rate, Tax rate.

Scenario Audit Checks: Toggle switches all statements, BS balances in all scenarios, Cash ties out, Hierarchy holds (Upside > Base > Downside for NI, EBITDA, FCF, margins).

SEC Filings Data Extraction

If the template specifically requires pulling data from SEC filings (10-K, 10-Q), see [references/sec-filings.md](#) for detailed extraction guidance. This reference is only needed when populating templates with public company data from regulatory filings.

Completing Model Templates

This section provides general guidance for completing any 3-statement financial model template while preserving existing formulas and ensuring data integrity.

Step 1: Analyze the Template Structure

Before entering any data, thoroughly review the template to understand its architecture:

Identify Input vs. Formula Cells

- Look for visual cues (font color, cell shading) that distinguish input cells from formula cells
- Common conventions: Blue font = inputs, Black font = formulas, Green font = links to other sheets
- Use Excel's Trace Precedents/Dependents (Formulas → Trace Precedents) to understand cell relationships
- Check for named ranges that may control key inputs (Formulas → Name Manager)

Map the Template's Flow

- Identify which tabs feed into others (e.g., Assumptions → IS → BS → CF)
- Note any supporting schedules and their linkages to main statements
- Document the template's specific line items and structure before populating

Step 2: Filling in Data Without Breaking Formulas

Golden Rules for Data Entry

Rule	Description
Only edit input cells	Never overwrite cells containing formulas unless intentionally replacing the formula
Preserve cell references	When copying data, use Paste Values (Ctrl+Shift+V) to avoid overwriting formulas with source formatting
Match the template's units	Verify if template uses thousands, millions, or actual values before entering data
Respect sign conventions	Follow the template's existing sign convention (e.g., expenses as positive or negative)
Check for circular references	If the template uses iterative calculations, ensure Enable Iterative Calculation is turned on

Safe Data Entry Process

1. Identify the exact cells designated for input (usually highlighted or labeled)
2. Enter historical data first, then verify formulas are calculating correctly for those periods
3. Enter assumption drivers that feed forecast calculations
4. Review calculated outputs to confirm formulas are working as intended
5. If a formula cell must be modified, document the original formula before making changes

Handling Pre-Built Formulas

- If formulas reference cells you haven't populated yet, expect temporary errors (#REF!, #DIV/0!) until all inputs are complete
- When formulas produce unexpected results, trace precedents to identify missing or incorrect inputs
- Never delete rows/columns without checking for formula dependencies across all tabs

Step 3: Validating Formulas

Formula Integrity Checks

Before relying on template outputs, validate that formulas are functioning correctly:

Check Type	Method
Trace precedents	Select a formula cell → Formulas → Trace Precedents to verify it references correct inputs
Trace dependents	Verify key inputs flow to expected output cells
Evaluate formula	Use Formulas → Evaluate Formula to step through complex calculations
Check for hardcodes	Projection formulas should reference assumptions, not contain hardcoded values
Test with known values	Input simple test values to verify formulas produce expected results
Cross-tab consistency	Ensure the same formula logic applies across all projection periods

Common Formula Issues to Watch For

- Mixed absolute/relative references causing incorrect results when copied across periods
- Broken links to external files or deleted ranges (#REF! errors)
- Division by zero in early periods before revenue ramps (#DIV/0! errors)
- Circular reference warnings (may be intentional for interest calculations)
- Inconsistent formulas across projection columns (use Ctrl+\ to find differences)

Validating Cross-Tab Linkages

- Confirm values that appear on multiple tabs are linked (not duplicated)
- Verify schedule totals tie to corresponding line items on main statements
- Check that period labels align across all tabs

Step 4: Quality Checks by Sheet

Perform these validation checks on each sheet after populating the template:

Income Statement (IS) Quality Checks

- Revenue figures match source data for historical periods
- All expense line items sum to reported totals
- Subtotals (Gross Profit, EBIT, EBT, Net Income) calculate correctly
- Tax calculation logic is appropriate (handles losses correctly)
- Forecast drivers reference assumptions tab (no hardcodes)
- Period-over-period changes are directionally reasonable

Balance Sheet (BS) Quality Checks

- Assets = Liabilities + Equity for every period (primary check)
- Cash balance matches Cash Flow Statement ending cash
- Working capital accounts tie to supporting schedules (if applicable)
- Retained Earnings rolls forward correctly: Prior RE + Net Income - Dividends +/- Adjustments = Ending RE
- Debt balances tie to debt schedule (if applicable)
- All balance sheet items have appropriate signs (assets positive, most liabilities positive)

Cash Flow Statement (CF) Quality Checks

- Net Income at top of CFO matches Income Statement Net Income
- Non-cash add-backs (D&A, SBC, etc.) tie to their source schedules/statements
- Working capital changes have correct signs (increase in asset = use of cash = negative)
- CapEx ties to PP&E schedule or fixed asset roll-forward
- Financing activities tie to changes in debt and equity accounts on BS
- Ending Cash matches Balance Sheet Cash
- Beginning Cash equals prior period Ending Cash

Supporting Schedule Quality Checks

- Opening balances equal prior period closing balances
- Roll-forward logic is complete (Beginning + Additions - Deductions = Ending)
- Schedule totals tie to main statement line items
- Assumptions used in calculations match Assumptions tab

Step 5: Cross-Statement Integrity Checks

After validating individual sheets, confirm the three statements are properly integrated:

Check	Formula	Expected Result
Balance Sheet Balance	Assets - Liabilities - Equity	= 0

Cash Tie-Out	CF Ending Cash - BS Cash	= 0
Net Income Link	IS Net Income - CF Starting Net Income	= 0
Retained Earnings	Prior RE + NI - Dividends - BS Ending RE	= 0 (adjust for SBC/ other items as needed)

Step 6: Final Review

Before considering the model complete:

- Toggle through all scenarios (if applicable) to verify checks pass in each case
- Review all #REF!, #DIV/0!, #VALUE!, and #NAME? errors and resolve or document
- Confirm all input cells have been populated (search for placeholder values)
- Verify units are consistent across all tabs
- Save a clean version before making any additional modifications

Model Validation and Audit

This section consolidates all validation checks and audit procedures for completed templates.

Core Linkages (Must Always Hold)

See [references/formulas.md](#) for all formula details.

Check	Formula	Expected Result
Balance Sheet Balance	Assets - Liabilities - Equity	= 0
Cash Tie-Out	CF Ending Cash - BS Cash	= 0
Cash Monthly vs Annual	Closing Cash (Monthly) - Closing Cash (Annual)	= 0
Net Income Link	IS Net Income - CF Starting Net Income	= 0
Retained Earnings	Prior RE + NI + SBC - Dividends - BS Ending RE	= 0
Equity Financing	Δ Common Stock/ APIC (BS) - Equity Issuance (CFF)	= 0
Year 0 Equity	Equity Raised (Year 0) - Beginning Equity Capital (Year 1)	= 0

Sign Convention Reference

Statement	Item	Sign Convention
CFO	D&A, SBC	Positive (add-back)
CFO	Δ AR (increase)	Negative (use of cash)
CFO	Δ AP (increase)	Positive (source of cash)

CFI	CapEx	Negative
CFF	Debt issuance	Positive
CFF	Debt repayments	Negative
CFF	Dividends	Negative

Circular Reference Handling

Interest expense creates circularity: Interest → Net Income → Cash → Debt Balance → Interest

Enable iterative calculation in Excel: File → Options → Formulas → Enable iterative calculation. Set maximum iterations to 100, maximum change to 0.001. Add a circuit breaker toggle in Assumptions tab.

Check Categories

Section 1: Currency Consistency

- Currency identified and documented in Assumptions
- All tabs use consistent currency symbol and scale
- Units row matches model currency

Section 2: Balance Sheet Integrity

- Assets = Liabilities + Equity (for each period)
- Formula: Assets - Liabilities - Equity (must = 0)

Section 3: Cash Flow Integrity

- Cash ties to BS (CF Ending Cash = BS Cash)
- Cash Monthly vs Annual: Closing Cash (Monthly) = Closing Cash (Annual)
- NI ties to IS (CF Net Income = IS Net Income)
- D&A ties to schedule
- SBC ties to IS
- Δ AR, Δ Inventory, Δ AP tie to WC schedule
- CapEx ties to DA schedule

Section 4: Retained Earnings

- RE roll-forward check: Prior RE + NI + SBC - Dividends = Ending RE
- Show component breakdown for debugging

Section 5: Working Capital

- AR, Inventory, AP tie to BS
- DSO, DIO, DPO reasonability checks (flag if outside normal ranges)

Section 6: Debt Schedule

- Total Debt ties to BS (Current + LT Debt)
- Interest calculation ties to IS

Section 6b: Equity Financing

- Equity issuance proceeds tie to BS Common Stock/APIC increase
- Cash increase from equity = Equity account increase (must balance)
- Equity Raise Tie-Out: $\Delta \text{Common Stock/APIC (BS)} = \text{Equity Issuance (CFF)}$ (must = 0)
- Year 0 Equity Tie-Out: $\text{Equity Raised (Year 0)} = \text{Beginning Equity Capital (Year 1)}$

Section 6c: NOL Schedule

- Beginning NOL (Year 1 / Formation) = 0 (new business starts with zero NOL)
- NOL increases only when $\text{EBT} < 0$ (losses must be realized to generate NOL)
- DTA ties to BS (NOL Schedule DTA = BS Deferred Tax Asset)
- NOL utilization $\leq 80\%$ of EBT (post-2017 federal limitation)
- NOL balance is non-negative (cannot utilize more than available)
- NOL generated only when $\text{EBT} < 0$
- Tax expense = 0 when taxable income ≤ 0

Section 7: Scenario Hierarchy

- Absolute metrics: Upside > Base > Downside (NI, EBITDA, FCF)
- Margins: Upside > Base > Downside (GM%, EBITDA%, NI%)
- Credit metrics: Upside < Base < Downside for leverage (inverted)

Section 8: Formula Integrity

- COGS, S&M, G&A, R&D, SBC driven by % of Revenue (no hardcodes)
- Consistent formulas across projection years
- No #REF!, #DIV/0!, #VALUE! errors

Section 9: Credit Metric Thresholds

- Flag metrics as Green/Yellow/Red based on covenant thresholds
- Summary of any red flags

Master Check Formula

Aggregate all section statuses into a single master check:

- If all sections pass → “✓ ALL CHECKS PASS”
- If any section fails → “✗ ERRORS DETECTED - REVIEW BELOW”

Quick Debug Workflow

When Master Status shows errors:

1. Scroll to find red-highlighted sections
 2. Identify which check category has failures
 3. Navigate to source tab to investigate
 4. Fix the underlying issue
 5. Return to Checks tab to verify resolution
-

3.20 Skill: audit-xls

File Metadata

Field	Value
name	audit-xls
description	Audit a spreadsheet for formula accuracy, errors, and common mistakes. Scopes to a selected range, a single sheet, or the entire model (including financial-model integrity checks like BS balance, cash tie-out, and logic sanity). Triggers on “audit this sheet”, “check my formulas”, “find formula errors”, “QA this spreadsheet”, “sanity check this”, “debug model”, “model check”, “model won’t balance”, “something’s off in my model”, “model review”.

Audit Spreadsheet

Audit formulas and data for accuracy and mistakes. Scope determines depth — from quick formula checks on a selection up to full financial-model integrity audits.

Step 1: Determine scope

If the user already gave a scope, use it. Otherwise **ask them**:

“What scope do you want me to audit? - **selection** — just the currently selected range - **sheet** — the current active sheet only - **model** — the whole workbook, including financial-model integrity checks (BS balance, cash tie-out, roll-forwards, logic sanity)”

The **model** scope is the deepest — use it for DCF, LBO, 3-statement, merger, comps, or any integrated financial model before sending to a client or IC.

Step 2: Formula-level checks (ALL scopes)

Run these regardless of scope:

Check	What to look for
Formula errors	<i>#REF!</i> , <i>#VALUE!</i> , <i>#N/A</i> , <i>#DIV/0!</i> , <i>#NAME?</i>
Hardcodes inside formulas	<i>=A1*1.05</i> — the <i>1.05</i> should be a cell reference
Inconsistent formulas	A formula that breaks the pattern of its neighbors in a row/column
Off-by-one ranges	<i>SUM/ AVERAGE</i> that misses the first or last row
Pasted-over formulas	Cell that looks like a formula but is actually a hardcoded value
Circular references	Intentional or accidental
Broken cross-sheet links	References to cells that moved or were deleted
Unit/ scale mismatches	Thousands mixed with millions, % stored as whole numbers
Hidden rows/ tabs	Could contain overrides or stale calculations

Step 3: Model-integrity checks (MODEL scope only)

If scope is **model**, identify the model type (DCF / LBO / 3-statement / merger / comps / custom) and run the appropriate integrity checks below.

3a. Structural review

Check	What to look for
Input/ formula separation	Are inputs clearly separated from calculations?
Color convention	Blue=input, black=formula, green=link — or whatever the model uses, applied consistently?
Tab flow	Logical order (Assumptions → IS → BS → CF → Valuation)?
Date headers	Consistent across all tabs?
Units	Consistent (thousands vs millions vs actuals)?

3b. Balance Sheet

Check	Test
BS balances	Total Assets = Total Liabilities + Equity (every period)
RE rollforward	Prior RE + Net Income – Dividends = Current RE
Goodwill/ intangibles	Flow from acquisition assumptions (if M&A)

If BS doesn't balance, **quantify the gap per period and trace where it breaks** — nothing else matters until this is fixed.

3c. Cash Flow Statement

Check	Test
Cash tie-out	CF Ending Cash = BS Cash (every period)
CF sums	$CFO + CFI + CFF = \Delta \text{Cash}$
D&A match	D&A on CF = D&A on IS
CapEx match	CapEx on CF matches PP&E rollforward on BS
WC changes	Signs match BS movements (ΔAR , ΔAP , $\Delta \text{Inventory}$)

3d. Income Statement

Check	Test
Revenue build	Ties to segment/ product detail
Tax	Tax expense = Pre-tax income $\times$ tax rate (allow for deferred tax adj)
Share count	Ties to dilution schedule (options, converts, buybacks)

3e. Circular references

- Interest $\rightarrow$ debt balance $\rightarrow$ cash $\rightarrow$ interest is a common intentional circ in LBO/3-stmt models
- If intentional: verify iteration toggle exists and works
- If unintentional: trace the loop and flag how to break it

3f. Logic & reasonableness

Check	Flag if
Growth rates	$>100\%$ revenue growth without explanation
Margins	Outside industry norms
Terminal value dominance	TV $> \sim 75\%$ of DCF EV (yellow flag)
Hockey-stick	Projections ramp unrealistically in out-years
Compounding	EBITDA compounds to absurd \$ by Year 10
Edge cases	Model breaks at 0% or negative growth, negative EBITDA, leverage goes negative

3g. Model-type-specific bugs

DCF:

- Discount rate applied to wrong period (mid-year vs end-of-year)
- Terminal value not discounted back
- WACC uses book values instead of market values

- FCF includes interest expense (should be unlevered)
- Tax shield double-counted

LBO:

- Debt paydown doesn't match cash sweep mechanics
- PIK interest not accruing to principal
- Management rollover not reflected in returns
- Exit multiple applied to wrong EBITDA (LTM vs NTM)
- Fees/expenses not deducted from Day 1 equity

Merger:

- Accretion/dilution uses wrong share count (pre- vs post-deal)
- Synergies not phased in
- Purchase price allocation doesn't balance
- Foregone interest on cash not included
- Transaction fees not in sources & uses

3-statement:

- Working capital changes have wrong sign
- Depreciation doesn't match PP&E schedule
- Debt maturity schedule doesn't match principal payments
- Dividends exceed net income without explanation

Step 4: Report

Output a findings table:

#	Sheet	Cell/ Range	Severity	Category	Issue	Suggested Fix
---	-------	-------------	----------	----------	-------	---------------

Severity:

- **Critical** — wrong output (BS doesn't balance, formula broken, cash doesn't tie)
- **Warning** — risky (hardcodes, inconsistent formulas, edge-case failures)
- **Info** — style/best-practice (color coding, layout, naming)

For **model** scope, prepend a summary line:

“Model type: [DCF/LBO/3-stmt/...] — Overall: [Clean / Minor Issues / Major Issues] — [N] critical, [N] warnings, [N] info”

Don't change anything without asking — report first, fix on request.

Notes

- **BS balance first** — if it doesn't balance, everything downstream is suspect
 - **Hardcoded overrides are the #1 source of silent bugs** — search aggressively
 - **Sign convention errors** (positive vs negative for cash outflows) are extremely common
 - If the model uses VBA macros, note any macro-driven calculations that can't be audited from formulas alone
-

3.21 Skill: clean-data-xls

File Metadata

Field	Value
name	clean-data-xls
description	Clean up messy spreadsheet data — trim whitespace, fix inconsistent casing, convert numbers-stored-as-text, standardize dates, remove duplicates, and flag mixed-type columns. Use when data is messy, inconsistent, or needs prep before analysis. Triggers on “clean this data”, “clean up this sheet”, “normalize this data”, “fix formatting”, “dedupe”, “standardize this column”, “this data is messy”.

Clean Data

Clean messy data in the active sheet or a specified range.

Environment

- **If running inside Excel (Office Add-in / Office JS):** Use Office JS directly (*Excel.run(async (context) => {...})*). Read via *range.values*, write helper-column formulas via *range.formulas = [[“=TRIM(A2)”]]*. The in-place vs helper-column decision still applies.
- **If operating on a standalone .xlsx file:** Use Python/openpyxl.

Workflow

Step 1: Scope

- If a range is given (e.g. *A1:F200*), use it
- Otherwise use the full used range of the active sheet
- Profile each column: detect its dominant type (text / number / date) and identify outliers

Step 2: Detect issues

Issue	What to look for
Whitespace	leading/ trailing spaces, double spaces
Casing	inconsistent casing in categorical columns (<i>usa</i> / <i>USA</i> / <i>Usa</i>)
Number-as-text	numeric values stored as text; stray \$, ,, % in number cells
Dates	mixed formats in the same column (<i>3/ 8/ 26</i> , <i>2026-03-08</i> , <i>March 8 2026</i>)
Duplicates	exact-duplicate rows and near-duplicates (case/ whitespace differences)
Blanks	empty cells in otherwise-populated columns
Mixed types	a column that's 98% numbers but has 3 text entries
Encoding	mojibake (<i>Ã©</i> , <i>â€™</i>), non-printing characters
Errors	<i>#REF!</i> , <i>#N/ A</i> , <i>#VALUE!</i> , <i>#DIV/ 0!</i>

Step 3: Propose fixes

Show a summary table before changing anything:

Column	Issue	Count	Proposed Fix
--------	-------	-------	--------------

Step 4: Apply

- **Prefer formulas over hardcoded cleaned values** — where the cleaned output can be expressed as a formula (e.g. `=TRIM(A2)`, `=VALUE(SUBSTITUTE(B2,"$",""))`, `=UPPER(C2)`, `=DATEVALUE(D2)`), write the formula in an adjacent helper column rather than computing the result in Python and overwriting the original. This keeps the transformation transparent and auditable.
- Only overwrite in place with computed values when the user explicitly asks for it, or when no sensible formula equivalent exists (e.g. encoding/mojibake repair)
- For destructive operations (removing duplicates, filling blanks, overwriting originals), confirm with the user first
- After each category of fix (whitespace → casing → number conversion → dates → dedup), show the user a sample of what changed and get confirmation before moving to the next category
- Report a before/after summary of what changed

3.22 Skill Resource: [skills/competitive-analysis/references/frameworks.md](#)

Frameworks Reference

2x2 Matrix: Common Axis Pairs by Industry

Technology/SaaS: Product breadth × Customer segment, Integration depth × Geographic reach

Consumer/Retail: Price point × Product range, Online × Offline presence

Financial Services: Product complexity × Customer sophistication, Scale × Specialization

Healthcare: Care setting × Payer mix, Technology enablement × Service breadth

Industrial: Customization × Scale, Geographic scope × Vertical focus

3.23 Skill Resource: skills/competitive-analysis/references/schemas.md

Schemas Reference

Additional table formats not shown in main SKILL.md.

M&A Transaction Table

Acquirer	Target	Date	Deal Value	Multiple	Rationale
Company A	Company B	MMM YYYY	\$X.XB	X.Xx EV/ Rev	[Strategic logic]

State multiple methodology: “X.Xx EV/Revenue” or “X.Xx EV/EBITDA”

Scenario Analysis Table

Scenario	Probability	Valuation	Key Assumptions
Bull	XX%	\$XXB	[Specific, quantified]
Base	XX%	\$XXB	[Specific, quantified]
Bear	XX%	\$XXB	[Specific, quantified]

Slide Structure

Structured Template

- [Insight headline, not topic]
 - [Main Content]
 - Source: [Citation] ([Date])
-

3.24 Skill: competitive-analysis

File Metadata

Field	Value
name	competitive-analysis

description

Framework for building competitive landscape decks — market positioning, competitor deep-dives, comparative analysis, strategic synthesis. Use when the user asks for a competitive landscape, competitor analysis, peer comparison, market positioning assessment, strategic review, or investment memo deck. Also triggers on “who are the competitors to X”, “benchmark X against peers”, “build a market map”, or any request to systematically evaluate competitive dynamics across an industry.

Competitive Landscape Mapping

Build a complete competitive analysis deck. This is a two-phase process: gather requirements and get outline approval first, then build.

Environment check

This skill works in both the PowerPoint add-in and chat. Identify which you’re in before starting — the mechanics differ, the workflow doesn’t:

- **Add-in** — the deck is open live; build slides directly into it.
- **Chat** — generate a *.pptx* file (or build into one the user uploaded).

Everything below applies in both.

Phase 1 — Scope the analysis

Competitive analysis means different things to different people. Before any research or slide-building, use *ask_user_question* to pin down what they actually want. Don’t guess — a 20-slide peer benchmarking deck and a 5-slide market map are both “competitive analysis” and take completely different shapes.

Gather in one round if you can (the tool takes up to 4 questions):

- **Scope** — Single target company with competitors around it? Or multi-company side-by-side with no protagonist?
- **Competitor set** — Which companies are in scope? If the user names them, use exactly those. If they say “the usual suspects,” propose a set and confirm.
- **Audience and depth** — Quick read for someone already in the space, or a full primer? This drives whether you need market sizing, industry economics, and history — or can skip to the comparison.
- **Investment context** — Do they need bull/base/bear scenarios and signposts? That’s Step 9 below; skip it if this is a strategic review rather than an investment thesis.

If they’ve uploaded an Excel/CSV with competitor data, confirm which columns map to which metrics before you start pulling numbers. Source-file fidelity matters: use values exactly as given, don’t recalculate or re-round.

Phase 2 — Outline, approve, then build

Do not create slides until the outline is approved. Propose slide titles and one-line content notes, present them to the user, get a yes. A competitive deck is 10-20 slides of interlocking content — rebuilding because slide 4 was wrong is expensive. The outline is the cheap iteration point.

When proposing the outline, *ask_user_question* works well for the structural decisions: which positioning visualization (2×2 matrix / radar / tier diagram — Step 5 below), how to group competitors (by business model / segment / posture — Step 4). These are taste calls the user likely has an opinion on.

Standards — apply throughout

Prompt fidelity

When the user specifies something, that's a requirement, not a suggestion:

- **Slide titles and section names** — exact wording. If they say “Overview and Competitive Scope,” don't swap in “FY2024 Competitive Landscape.”
- **Chart vs. table** — not interchangeable. “Embedded chart” means a real chart object with data labels on the bars/slices, not a formatted table.
- **Complete data series** — if they list 7 competitors, include all 7. If they show 2015-2025, include every year.
- **Exact values and ratios** — “surpasses DoorDash 4:1, Lyft 8:1” means those ratios, not “7.6x Lyft.”

Source quality, when sources conflict

1. 10-Ks / annual reports (audited)
2. Earnings calls / investor presentations (management commentary)
3. Sell-side research (analyst estimates, useful for private company sizing)
4. Industry reports (McKinsey, Gartner — market sizing, trends)
5. News (recent developments only; verify against primary sources)

Data comparability

- All competitor metrics from the same fiscal year; flag exceptions explicitly (“FY24” vs “H1 2024”)
- Same metric definitions across competitors
- Convert to USD for international; note the exchange rate and date
- Missing data shows as “-” or “N/A” with an “[E]” flag for estimates — never blank
- Every number has a citation: “[Company] [Document] ([Date])”

Design

- **Slide titles are insights, not labels.** “Scale leaders pulling away from niche players” — not “Competitive Analysis.”

- **Signposts are quantified.** “Margin below 40%” — not “margins decline.”
- **Ratings show the actual.** “●●● \$160B” — not just “●●●.”
- **Charts are real chart objects** — not text tables dressed up to look like charts.

Typography — set explicitly, don’t rely on defaults:

- Slide titles: 28-32pt bold
- Section headers: 18-20pt bold
- Body text: 14-16pt (never below 14pt)
- Table text: 14pt
- Sources/footnotes: 14pt, gray
- Same element type = same size throughout the deck

Charts:

- Legend inside the chart boundary, not floating over the plot area
- Right-side legend for pies (≤ 6 slices), bottom legend for line/bar (≤ 4 series)
- More than 6 series → split into multiple charts or use a table
- Pie charts show percentages on slices, not just in the legend

Tables:

- Light gray header row, bold
- Right-align numbers, left-align text
- Enough cell padding that text doesn’t touch borders

Color: 2-3 colors max. Muted — navy, gray, one accent. Same color meanings throughout.

What’s strict vs. flexible

Always	Case-by-case
Exact titles/ sections when user specifies	Creative titles when they don’t
Chart when user says chart; table when they say table	Visualization type when unspecified
Every competitor/ data point they list	Number of competitors when unspecified
Exact values when specified	Rounding when precision unspecified
Titles fit without overflow	Number of competitor categories
No overlapping elements	Which dimensions to compare

Analysis workflow

Step 0 — Industry-defining metrics

Before anything else: what 3-5 metrics does this industry actually run on? Use these consistently across every competitor.

Industry	Key metrics
SaaS	ARR, NRR, CAC payback, LTV/ CAC, Rule of 40
Payments	GPV, take rate, attach rate, transaction margin
Marketplaces	GMV, take rate, buyer/ seller ratio, repeat rate
Retail	Same-store sales, inventory turns, sales per sq ft
Logistics	Volume, cost per unit, on-time delivery %, capacity utilization

Industry not listed — pick the metrics investors and operators benchmark on.

Step 1 — Market context

Size, growth, drivers, headwinds. With sources.

Correct: “Embedded payments is \$80-100B in 2024, growing 20-25% CAGR (McKinsey 2024)” Wrong: “The market is large and growing rapidly”

Step 2 — Industry economics

Map how value flows. Approach depends on industry structure:

- **Vertically structured** — value chain layers, typical margin at each
- **Platform/network** — ecosystem participants, value flows between them
- **Fragmented** — consolidation dynamics, margin differences by scale

Step 3 — Target company profile

Structured Template

- | Metric | Value |
- | — | — |
- | Revenue | \$4.96B |
- | Growth | +26% YoY |
- | Gross Margin | 45% |
- | Profitability | \$373M Adj. EBITDA |
- | Customers | 134K |
- | Retention | 92% |
- | Market Share | ~15% |

Multi-segment companies add a breakdown:

Structured Template

- | Segment | Revenue | Rev YoY | Rev % | EBITDA | EBITDA YoY | Margin |
- |—|—|—|—|—|—|—|
- | Seg A | \$25.1B | +26% | 57% | \$6.5B | +31% | 26% |
- | Seg B | \$13.8B | +31% | 31% | \$2.5B | +64% | 18% |
- | Seg C | \$5.1B | -2% | 12% | -\$74M | -16% | -1% |
- | Total | \$44.0B | +18% | 100% | \$6.5B* | - | 15% |

*Note corporate costs if applicable

Step 4 — Competitor mapping

Group by whichever lens fits (this is a good *ask_user_question* decision if the user hasn't specified):

- By business model — platform / vertical / horizontal
- By segment — enterprise / SMB / consumer
- By posture — direct / adjacent / emerging
- By origin — incumbent / disruptor / new entrant

Step 5 — Positioning visualization

Type	When
2×2 matrix	Two dominant competitive factors
Radar/ spider	Multi-factor comparison
Tier diagram	Natural clustering into strategic groups
Value chain map	Vertical industries
Ecosystem map	Platform markets

See *references/frameworks.md* for 2×2 axis pairs by industry.

Step 6 — Competitor deep-dives

Two tables per competitor.

Metrics:

Structured Template

- | Metric | Value |
- |—|—|
- | Revenue | \$X.XB |

- | Growth | +XX% YoY |
- | Gross Margin | XX% |
- | Market Cap | \$X.XB |
- | Profitability | \$XXXM EBITDA |
- | Customers | XXK |
- | Retention | XX% |
- | Market Share | ~XX% |

Qualitative:

Structured Template

- | Category | Assessment |
- | — | — |
- | Business | What they do (1 sentence) |
- | Strengths | 2-3 bullets |
- | Weaknesses | 2-3 bullets |
- | Strategy | Current priorities |

Step 7 — Comparative analysis

Structured Template

- | Dimension | Company A | Company B | Company C |
- | — | — | — | — |
- | Scale | ●●● \$160B | ●●○ \$45B | ●○○ \$8B |
- | Growth | ●●○ +26% | ●●● +35% | ●●○ +22% |
- | Margins | ●●○ 7.5% | ●○○ 3.2% | ●●● 15% |

Step 8 — Strategic context

M&A transactions (multiples, rationale), partnership trends, capital raising patterns, regulatory developments. See *references/schemas.md* for the M&A transaction table format.

Step 9 — Synthesis

Moat assessment — rate each competitor Strong / Moderate / Weak on:

Moat	What to assess
Network effects	User/ supplier flywheel strength; cross-side vs same-side
Switching costs	Technical integration depth, contractual lock-in, behavioral habits

Scale economies
Intangible assets

Unit cost advantages at volume; minimum efficient scale
Brand, proprietary data, regulatory licenses, patents

Required synthesis elements:

- Durable advantages (hard to replicate) — map to moat categories
- Structural vulnerabilities (hard to fix)
- Current state vs. trajectory

For investment contexts (skip if the Phase 1 scoping said no):

Structured Template

- | Scenario | Probability | Key driver |
 - | —|—|—|
 - | Bull | 30% | Market share gains, margin expansion |
 - | Base | 50% | Current trajectory continues |
 - | Bear | 20% | Competitive pressure, margin compression |
-

Quality checklist

Before finishing:

Prompt fidelity

- Slide titles match what the user specified, verbatim
- Charts where they said chart; tables where they said table
- Every competitor/year/data point they listed is present
- Exact values and formats as specified

Data consistency

- Source-file values extracted directly, not recalculated
- Same metric shows the same value on every slide it appears
- Same decimal precision as the source

Layout

- Titles fit without overflow
- No overlapping elements
- All text within containers, no clipping

Content

- Every number has a citation
- All metrics from the same fiscal period (or flagged)
- Slide titles state insights, not topics
- Charts are real chart objects

Run standard visual verification checks on every slide — this catches overlaps, overflow, and low-contrast text that don't show up when you're reading back the XML.

3.25 Skill: comps-analysis

File Metadata

Field	Value
name	comps-analysis
description	
**Perfect for	**
**Not ideal for	**

Comparable Company Analysis

⚠ CRITICAL: Data Source Priority (READ FIRST)

ALWAYS follow this data source hierarchy:

1. **FIRST: Check for MCP data sources** - If S&P Kensho MCP, FactSet MCP, or Daloopa MCP are available, use them exclusively for financial and trading information
2. **DO NOT use web search** if the above MCP data sources are available
3. **ONLY if MCPs are unavailable:** Then use Bloomberg Terminal, SEC EDGAR filings, or other institutional sources
4. **NEVER use web search as a primary data source** - it lacks the accuracy, audit trails, and reliability required for institutional-grade analysis

Why this matters: MCP sources provide verified, institutional-grade data with proper citations. Web search results can be outdated, inaccurate, or unreliable for financial analysis.

Overview

This skill teaches Claude to build institutional-grade comparable company analyses that combine operating metrics, valuation multiples, and statistical benchmarking. The output is a structured Excel/spreadsheet that enables informed investment decisions through peer comparison.

Reference Material & Contextualization:

An example comparable company analysis is provided in *examples/comps_example.xlsx*. When using this or other example files in this skill directory, use them intelligently:

DO use examples for:

- Understanding structural hierarchy (how sections flow)
- Grasping the level of rigor expected (statistical depth, documentation standards)
- Learning principles (clear headers, transparent formulas, audit trails)

DO NOT use examples for:

- Exact reproduction of format or metrics
- Copying layout without considering context
- Applying the same visual style regardless of audience

ALWAYS ask yourself first:

1. **“Do you have a preferred format or should I adapt the template style?”**
2. **“Who is the audience?”** (Investment committee, board presentation, quick reference, detailed memo)
3. **“What’s the key question?”** (Valuation, growth analysis, competitive positioning, efficiency)
4. **“What’s the context?”** (M&A evaluation, investment decision, sector benchmarking, performance review)

Adapt based on specifics:

- **Industry context:** Big tech mega-caps need different metrics than emerging SaaS startups
- **Sector-specific needs:** Add relevant metrics early (e.g., cloud ARR, enterprise customers, developer ecosystem for tech)
- **Company familiarity:** Well-known companies may need less background, more focus on delta analysis
- **Decision type:** M&A requires different emphasis than ongoing portfolio monitoring

Core principle: Use template principles (clear structure, statistical rigor, transparent formulas) but vary execution based on context. The goal is institutional-quality analysis, not institutional-looking templates.

User-provided examples and explicit preferences always take precedence over defaults.

Core Philosophy

“Build the right structure first, then let the data tell the story.”

Start with headers that force strategic thinking about what matters, input clean data, build transparent formulas, and let statistics emerge automatically. A good comp should be immediately readable by someone who didn’t build it.

⚠ CRITICAL: Formulas Over Hardcodes + Step-by-Step Verification

Environment — Office JS vs Python:

- **If running inside Excel (Office Add-in / Office JS):** Use Office JS directly (*Excel.run(async (context) => {...})*). Write formulas via *range.formulas = [["=E7/C7"]]*, not *range.values*. No separate recalc step — Excel handles it natively. Use *range.format.** for colors/fonts.
- **If generating a standalone .xlsx file:** Use Python/openpyxl. Write *cell.value = "=E7/C7"* (formula string).
- Same principles either way — just translate the API calls.
- **Office JS merged cell pitfall:** Do NOT call *.merge()* then set *.values* on the merged range (throws *InvalidArgument* — range still reports its pre-merge dimensions). Instead write the value to the top-left cell alone, then merge + format the full range: *"js ws.getRange("A1").values = [["TECHNOLOGY — COMPARABLE COMPANY ANALYSIS"]]; const hdr = ws.getRange("A1:H1"); hdr.merge(); hdr.format.fill.color = "#1F4E79"; hdr.format.font.color = "#FFFFFF"; hdr.format.font.bold = true; "*

Formulas, not hardcodes:

- Every derived value (margin, multiple, statistic) MUST be an Excel formula referencing input cells — never a pre-computed number pasted in
- When using Python/openpyxl to build the sheet: write *cell.value = "=E7/C7"* (formula string), NOT *cell.value = 0.687* (computed result)
- The only hardcoded values should be raw input data (revenue, EBITDA, share price, etc.) — and every one of those gets a cell comment with its source
- Why: the model must update automatically when an input changes. A hardcoded margin is a silent bug waiting to happen.

Verify step-by-step with the user:

- After setting up the structure → show the user the header layout before filling data
 - After entering raw inputs → show the user the input block and confirm sources/periods before building formulas
 - After building operating metrics formulas → show the calculated margins and sanity-check with the user before moving to valuation
 - After building valuation multiples → show the multiples and confirm they look reasonable before adding statistics
 - Do NOT build the entire sheet end-to-end and then present it — catch errors early by confirming each section
-

Section 1: Document Structure & Setup

Header Block (Rows 1-3)

Structured Template

- Row 1: [ANALYSIS TITLE] - COMPARABLE COMPANY ANALYSIS
- Row 2: [List of Companies with Tickers] • [Company 1 (TICK1)] • [Company 2 (TICK2)] • [Company 3 (TICK3)]
- Row 3: As of [Period] | All figures in [USD Millions/Billions] except per-share amounts and ratios

Why this matters: Establishes context immediately. Anyone opening this file knows what they're looking at, when it was created, and how to interpret the numbers.

Visual Convention Standards (OPTIONAL - User preferences and uploaded templates always override)

IMPORTANT: These are suggested defaults only. Always prioritize:

1. User's explicit formatting preferences
2. Formatting from any uploaded template files
3. Company/team style guides
4. These defaults (only if no other guidance provided)

Suggested Font & Typography:

- **Font family:** Times New Roman (professional, readable, industry standard)
- **Font size:** 11pt for data cells, 12pt for headers
- **Bold text:** Section headers, company names, statistic labels

Default Color & Shading — Professional Blue/Grey Palette (minimal is better):

- **Keep it restrained** — only blues and greys. Do NOT introduce greens, oranges, reds, or multiple accent colors. A clean comps sheet uses 3-4 colors total.
- **Section headers** (e.g., "OPERATING STATISTICS & FINANCIAL METRICS"):
 - Dark blue background (*#1F4E79* or *#17365D* navy)
 - White bold text
 - Full row shading across all columns
- **Column headers** (e.g., "Company", "Revenue", "Margin"):
 - Light blue background (*#D9E1F2* or similar pale blue)
 - Black bold text
 - Centered alignment
- **Data rows:**
 - White background for company data
 - Black text for formulas; blue text for hardcoded inputs
- **Statistics rows** (Maximum, 75th Percentile, etc.):
 - Light grey background (*#F2F2F2*)
 - Black text, left-aligned labels
- **That's the whole palette:** dark blue + light blue + light grey + white. Nothing else unless the user's template says otherwise.

Suggested Formatting Conventions:

- **Decimal precision:**
 - Percentages: 1 decimal (12.3%)
 - Multiples: 1 decimal (13.5x)
 - Dollar amounts: No decimals, thousands separator (69,632)
 - Margins shown as percentages: 1 decimal (68.7%)
- **Borders:** No borders (clean, minimal appearance)
- **Alignment:** All metrics center-aligned for clean, uniform appearance
- **Cell dimensions:** All column widths should be uniform/even, all row heights should be consistent (creates clean, professional grid)

Note: If the user provides a template file or specifies different formatting, use that instead.

Section 2: Operating Statistics & Financial Metrics

Core Columns (Start with these)

1. **Company** - Names with consistent formatting
2. **Revenue** - Size metric (can be LTM, quarterly, or annual depending on context)
3. **Revenue Growth** - Year-over-year percentage change
4. **Gross Profit** - Revenue minus cost of goods sold
5. **Gross Margin** - GP/Revenue (fundamental profitability)
6. **EBITDA** - Earnings before interest, tax, depreciation, amortization
7. **EBITDA Margin** - EBITDA/Revenue (operating efficiency)

Optional Additions (Choose based on industry/purpose)

- **Quarterly vs LTM** - Include both if seasonality matters
- **Free Cash Flow** - For capital-intensive or SaaS businesses
- **FCF Margin** - FCF/Revenue (cash generation efficiency)
- **Net Income** - For mature, profitable companies
- **Operating Income** - For businesses with varying D&A
- **CapEx metrics** - For asset-heavy industries
- **Rule of 40** - Specifically for SaaS (Growth % + Margin %)
- **FCF Conversion** - For quality of earnings analysis (advanced)

Formula Examples (Using Row 7 as example)

Structured Template

- // Core ratios - these are always calculated
- Gross Margin (F7): $=E7/C7$
- EBITDA Margin (H7): $=G7/C7$
- // Optional ratios - include if relevant
- FCF Margin: $=[\text{FCF}]/[\text{Revenue}]$
- Net Margin: $=[\text{Net Income}]/[\text{Revenue}]$
- Rule of 40: $=[\text{Growth \%}]+[\text{FCF Margin \%}]$

Golden Rule: Every ratio should be [Something] / [Revenue] or [Something] / [Something from this sheet]. Keep it simple.

Statistics Block (After company data)

CRITICAL: Add statistics formulas for all comparable metrics (ratios, margins, growth rates, multiples).

Structured Template

- [Leave one blank row for visual separation]
- - Maximum: $=\text{MAX}(B7:B9)$
- - 75th Percentile: $=\text{QUARTILE}(B7:B9,3)$
- - Median: $=\text{MEDIAN}(B7:B9)$
- - 25th Percentile: $=\text{QUARTILE}(B7:B9,1)$
- - Minimum: $=\text{MIN}(B7:B9)$

Columns that NEED statistics (comparable metrics):

- Revenue Growth %, Gross Margin %, EBITDA Margin %, EPS
- EV/Revenue, EV/EBITDA, P/E, Dividend Yield %, Beta

Columns that DON'T need statistics (size metrics):

- Revenue, EBITDA, Net Income (absolute size varies by company scale)
- Market Cap, Enterprise Value (not comparable across different-sized companies)

Note: Add one blank row between company data and statistics rows for visual separation. Do NOT add a “SECTOR STATISTICS” or “VALUATION STATISTICS” header row.

Why quartiles matter: They show distribution, not just average. A 75th percentile multiple tells you what “premium” companies trade at.

Section 3: Valuation Multiples & Investment Metrics

Core Valuation Columns (Start with these)

1. **Company** - Same order as operating section
2. **Market Cap** - Current market valuation
3. **Enterprise Value** - $\text{Market Cap} \pm \text{Net Debt/Cash}$
4. **EV/Revenue** - How much market pays per dollar of sales
5. **EV/EBITDA** - How much market pays per dollar of earnings
6. **P/E Ratio** - Price relative to net earnings

Optional Valuation Metrics (Choose based on context)

- **FCF Yield** - $\text{FCF}/\text{Market Cap}$ (for cash-focused analysis)
- **PEG Ratio** - $\text{P/E}/\text{Growth Rate}$ (for growth companies)
- **Price/Book** - Market value vs. book value (for asset-heavy businesses)
- **ROE/ROA** - Return metrics (for profitability comparison)
- **Revenue/EBITDA CAGR** - Historical growth rates (for trend analysis)
- **Asset Turnover** - $\text{Revenue}/\text{Assets}$ (for operational efficiency)
- **Debt/Equity** - Leverage (for capital structure analysis)

Key Principle: Include 3-5 core multiples that matter for your industry. Don't include every possible metric just because you can.

Formula Examples

Structured Template

- // Core multiples - always include these
- EV/Revenue: $=[\text{Enterprise Value}]/[\text{LTM Revenue}]$
- EV/EBITDA: $=[\text{Enterprise Value}]/[\text{LTM EBITDA}]$
- P/E Ratio: $=[\text{Market Cap}]/[\text{Net Income}]$
- // Optional multiples - include if data available
- FCF Yield: $=[\text{LTM FCF}]/[\text{Market Cap}]$
- PEG Ratio: $=[\text{P/E}]/[\text{Growth Rate \%}]$

Cross-Reference Rule

CRITICAL: Valuation multiples **MUST** reference the operating metrics section. Never input the same raw data twice. If revenue is in C7, then EV/Revenue formula should reference C7.

Statistics Block

Same structure as operating section: Max, 75th, Median, 25th, Min for every metric. Add one blank row for visual separation between company data and statistics. Do NOT add a “VALUATION STATISTICS” header row.

Section 4: Notes & Methodology Documentation

Required Components

Data Sources & Quality:

- Where did the data come from? (S&P Kensho MCP, FactSet MCP, Daloopa MCP, Bloomberg, SEC filings)
- What period does it cover? (Q4 2024, audited figures)
- How was it verified? (Cross-checked against 10-K/10-Q)
- Note: Prioritize MCP data sources (S&P Kensho, FactSet, Daloopa) if available for better accuracy and traceability

Key Definitions:

- EBITDA calculation method (Gross Profit + D&A, or Operating Income + D&A)
- Free Cash Flow formula (Operating CF - CapEx)
- Special metrics explained (Rule of 40, FCF Conversion)
- Time period definitions (LTM, CAGR calculation periods)

Valuation Methodology:

- How was Enterprise Value calculated? (Market Cap + Net Debt)
- What growth rates were used? (Historical CAGR, forward estimates)
- Any adjustments made? (One-time items excluded, normalized margins)

Analysis Framework:

- What's the investment thesis? (Cloud/SaaS efficiency)
 - What metrics matter most? (Cash generation, capital efficiency)
 - How should readers interpret the statistics? (Quartiles provide context)
-

Section 5: Choosing the Right Metrics (Decision Framework)

Start with “What question am I answering?”

“Which company is undervalued?” → Focus on: EV/Revenue, EV/EBITDA, P/E, Market Cap → Skip: Operational details, growth metrics

“Which company is most efficient?” → Focus on: Gross Margin, EBITDA Margin, FCF Margin, Asset Turnover
→ Skip: Size metrics, absolute dollar amounts

“Which company is growing fastest?” → Focus on: Revenue Growth %, EBITDA CAGR, User/Customer Growth
→ Skip: Margin metrics, leverage ratios

“Which is the best cash generator?” → Focus on: FCF, FCF Margin, FCF Conversion, CapEx intensity → Skip: EBITDA, P/E ratios

Industry-Specific Metric Selection

Software/SaaS: Must have: Revenue Growth, Gross Margin, Rule of 40 Optional: ARR, Net Dollar Retention, CAC Payback Skip: Asset Turnover, Inventory metrics

Manufacturing/Industrials: Must have: EBITDA Margin, Asset Turnover, CapEx/Revenue Optional: ROA, Inventory Turns, Backlog Skip: Rule of 40, SaaS metrics

Financial Services: Must have: ROE, ROA, Efficiency Ratio, P/E Optional: Net Interest Margin, Loan Loss Reserves Skip: Gross Margin, EBITDA (not meaningful for banks)

Retail/E-commerce: Must have: Revenue Growth, Gross Margin, Inventory Turnover Optional: Same-Store Sales, Customer Acquisition Cost Skip: Heavy R&D or CapEx metrics

The “5-10 Rule”

5 operating metrics - Revenue, Growth, 2-3 margins/efficiency metrics **5 valuation metrics** - Market Cap, EV, 3 multiples = **10 total columns** - Enough to tell the story, not so many you lose the thread

If you have more than 15 metrics, you’re probably including noise. Edit ruthlessly.

Section 6: Best Practices & Quality Checks

Before You Start

1. **Define the peer group** - Companies must be truly comparable (similar business model, scale, geography)
2. **Choose the right period** - LTM smooths seasonality; quarterly shows trends
3. **Standardize units upfront** - Millions vs. billions decision affects everything
4. **Map data sources** - Know where each number comes from

As You Build

1. **Input all raw data first** - Complete the blue text before writing formulas
2. **Add cell comments to ALL hard-coded inputs** - Right-click cell → Insert Comment → Document source OR assumption **For sourced data, cite exactly where it came from:**
 - Example: “Bloomberg Terminal - MSFT Equity DES, accessed 2024-10-02”
 - Example: “Q4 2024 10-K filing, page 42, line item ’Total Revenue”
 - Example: “FactSet consensus estimate as of 2024-10-02”

- **Include hyperlinks when possible:** Right-click cell → Link → paste URL to SEC filing, data source, or report **For assumptions, explain the reasoning:**
 - Example: “Assumed 15% EBITDA margin based on peer median, company does not disclose”
 - Example: “Estimated Enterprise Value as Market Cap + \$50M net debt (from Q3 balance sheet, Q4 not yet available)”
 - Example: “Forward P/E based on street consensus EPS of \$3.45 (average of 12 analyst estimates)” **Why this matters:** Enables audit trails, data verification, assumption transparency, and future updates
3. **Build formulas row by row** - Test each calculation before moving on
 4. **Use absolute references for headers** - \$C\$6 locks the header row
 5. **Format consistently** - Percentages as percentages, not decimals
 6. **Add conditional formatting** - Highlight outliers automatically

Sanity Checks

- **Margin test:** Gross margin > EBITDA margin > Net margin (always true by definition)
- **Multiple reasonableness:**
 - EV/Revenue: typically 0.5-20x (varies widely by industry)
 - EV/EBITDA: typically 8-25x (fairly consistent across industries)
 - P/E: typically 10-50x (depends on growth rate)
- **Growth-multiple correlation:** Higher growth usually means higher multiples
- **Size-efficiency trade-off:** Larger companies often have better margins (scale benefits)

Common Mistakes to Avoid

✗ Mixing market cap and enterprise value in formulas ✗ Using different time periods for numerator and denominator (LTM vs quarterly) ✗ Hardcoding numbers into formulas instead of cell references ✗ **Hard-coded inputs without cell comments citing the source OR explaining the assumption** ✗ Missing hyperlinks to SEC filings or data sources when available ✗ Including too many metrics without clear purpose ✗ Including non-comparable companies (different business models) ✗ Using outdated data without disclosure ✗ Calculating averages of percentages incorrectly (should be median)

Section 6: Advanced Features

Dynamic Headers

For columns showing calculations, use clear unit labels:

Structured Template

- Revenue Growth (YoY) % | EBITDA Margin | FCF Margin | Rule of 40

Quartile Analysis Benefits

Instead of just mean/median, quartiles show:

- **75th percentile** = “Premium” companies trade here
- **Median** = Typical market valuation
- **25th percentile** = “Discount” territory

This helps answer: “Is our target company trading rich or cheap vs. peers?”

Industry-Specific Modifications

Software/SaaS:

- Add: ARR, Net Dollar Retention, CAC Payback Period
- Emphasize: Rule of 40, FCF margins, gross margins >70%

Healthcare:

- Add: R&D/Revenue, Pipeline value, Regulatory status
- Emphasize: EBITDA margins, growth rates, reimbursement risk

Industrials:

- Add: Backlog, Order book trends, Geographic mix
- Emphasize: ROIC, asset turnover, cyclical adjustments

Consumer:

- Add: Same-store sales, Customer acquisition cost, Brand value
 - Emphasize: Revenue growth, gross margins, inventory turns
-

Section 7: Workflow & Practical Tips

Step-by-Step Process

1. Set up structure (30 minutes)

- Create all headers
- Format cells (blue for inputs, black for formulas)
- Lock in units and date references

2. Gather data (60-90 minutes)

- Pull from primary sources (S&P Kensho MCP, FactSet MCP, Daloopa MCP if available; otherwise Bloomberg, SEC)
- Input all raw numbers in blue

- Document sources in notes section
3. **Build formulas** (30 minutes)
 - Start with simple ratios (margins)
 - Progress to multiples (EV/Revenue)
 - Add cross-checks (do margins make sense?)
 4. **Add statistics** (15 minutes)
 - Copy formula structure for all columns
 - Verify ranges are correct (B7:B9, not B7:B10)
 - Check quartile logic
 5. **Quality control** (30 minutes)
 - Run sanity checks
 - Verify formula references
 - Check for #DIV/0! or #REF! errors
 - Compare against known benchmarks
 6. **Documentation** (15 minutes)
 - Complete notes section
 - Add data sources
 - Define methodologies
 - Date-stamp the analysis

Pro Tips

- **Save templates:** Build once, reuse forever
- **Color-code outliers:** Conditional formatting for values >2 standard deviations
- **Link to source files:** Hyperlink to Bloomberg screenshots or SEC filings
- **Version control:** Save as “Comps_v1_2024-12-15” with clear dating
- **Collaborative reviews:** Have someone else check your formulas

Excel Formatting Checklist (Optional - adapt to user preferences)

- ☐ Font set to user’s preferred style (default: Times New Roman, 11pt data, 12pt headers)
- ☐ Section headers formatted per user’s template (default: dark blue #17365D with white bold text)
- ☐ Column headers formatted per user’s template (default: light blue/gray #D9E2F3 with black bold text)
- ☐ Statistics rows formatted per user’s template (default: light gray #F2F2F2)
- ☐ No borders applied (clean, minimal appearance)
- ☐ **Column widths set to uniform/even width** (creates clean, professional appearance)

- ☐ **Row heights set to consistent height** (typically 20-25pt for data rows)
 - ☐ Numbers formatted with proper decimal precision and thousands separators
 - ☐ **All metrics center-aligned** for clean, uniform appearance
 - ☐ **One blank row for separation between company data and statistics rows**
 - ☐ **No separate “SECTOR STATISTICS” or “VALUATION STATISTICS” header rows**
 - ☐ **Every hard-coded input cell has a comment with either: (1) exact data source, OR (2) assumption explanation**
 - ☐ **Hyperlinks added to cells where applicable** (SEC filings, data provider pages, reports)
-

Section 8: Example Template Layout

Simple Version (Start here):

Structured Template

- TECHNOLOGY - COMPARABLE COMPANY ANALYSIS
- Microsoft • Alphabet • Amazon
- As of Q4 2024 | All figures in USD Millions
- OPERATING METRICS
- Company Revenue Growth Gross EBITDA EBITDA
- (LTM) (YoY) Margin (LTM) Margin
- MSFT 261,400 12.3% 68.7% 205,100 78.4%
- GOOGL 349,800 11.8% 57.9% 239,300 68.4%
- AMZN 638,100 10.5% 47.3% 152,600 23.9%
- [blank row]
- Median =MEDIAN =MEDIAN =MEDIAN =MEDIAN =MEDIAN
- 75th % =QUART =QUART =QUART =QUART =QUART
- 25th % =QUART =QUART =QUART =QUART =QUART
- VALUATION MULTIPLES
- Company Mkt Cap EV EV/Rev EV/EBITDA P/E
- MSFT 3,550,000 3,530,000 13.5x 17.2x 36.0
- GOOGL 2,030,000 1,960,000 5.6x 8.2x 24.5
- AMZN 2,226,000 2,320,000 3.6x 15.2x 58.3
- [blank row]
- Median =MEDIAN =MEDIAN =MEDIAN =MEDIAN =MED

- 75th % =QUART =QUART =QUART =QUART =QRT
- 25th % =QUART =QUART =QUART =QUART =QRT

Add complexity only when needed:

- Include quarterly AND LTM if seasonality matters
 - Add FCF metrics if cash generation is key story
 - Include industry-specific metrics (Rule of 40 for SaaS, etc.)
 - Add more statistics rows if you have >5 companies
-

Section 9: Industry-Specific Additions (Optional)

Only add these if they're critical to your analysis. Most comps work fine with just core metrics.

Software/SaaS: Add if relevant: ARR, Net Dollar Retention, Rule of 40

Financial Services: Add if relevant: ROE, Net Interest Margin, Efficiency Ratio

E-commerce: Add if relevant: GMV, Take Rate, Active Buyers

Healthcare: Add if relevant: R&D/Revenue, Pipeline Value, Patent Timeline

Manufacturing: Add if relevant: Asset Turnover, Inventory Turns, Backlog

Section 10: Red Flags & Warning Signs

Data Quality Issues

► Inconsistent time periods (mixing quarterly and annual) ► Missing data without explanation ► Significant differences between data sources (>10% variance)

Valuation Red Flags

► Negative EBITDA companies being valued on EBITDA multiples (use revenue multiples instead) ► P/E ratios >100x without hypergrowth story ► Margins that don't make sense for the industry

Comparability Issues

► Different fiscal year ends (causes timing problems) ► Mixing pure-play and conglomerates ► Materially different business models labeled as "comps"

When in doubt, exclude the company. Better to have 3 perfect comps than 6 questionable ones.

Section 11: Formulas Reference Guide

Essential Excel Formulas

Structured Template

- // Statistical Functions
- =AVERAGE(range) // Simple mean
- =MEDIAN(range) // Middle value
- =QUARTILE(range, 1) // 25th percentile
- =QUARTILE(range, 3) // 75th percentile
- =MAX(range) // Maximum value
- =MIN(range) // Minimum value
- =STDEV.P(range) // Standard deviation
- // Financial Calculations
- =B7/C7 // Simple ratio (Margin)
- =SUM(B7:B9)/3 // Average of multiple companies
- =IF(B7>0, C7/B7, "N/A") // Conditional calculation
- =IFERROR(C7/D7, 0) // Handle divide by zero
- // Cross-Sheet References
- ='Sheet1'!B7 // Reference another sheet
- =VLOOKUP(A7, Table1, 2) // Lookup from data table
- =INDEX(MATCH()) // Advanced lookup
- // Formatting
- =TEXT(B7, "0.0%") // Format as percentage
- =TEXT(C7, "#,##0") // Thousands separator

Common Ratio Formulas

Structured Template

- Gross Margin = Gross Profit / Revenue
 - EBITDA Margin = EBITDA / Revenue
 - FCF Margin = Free Cash Flow / Revenue
 - FCF Conversion = FCF / Operating Cash Flow
 - ROE = Net Income / Shareholders' Equity
 - ROA = Net Income / Total Assets
 - Asset Turnover = Revenue / Total Assets
 - Debt/Equity = Total Debt / Shareholders' Equity
-

Key Principles Summary

1. **Structure drives insight** - Right headers force right thinking
 2. **Less is more** - 5-10 metrics that matter beat 20 that don't
 3. **Choose metrics for your question** - Valuation analysis $\neq$ efficiency analysis
 4. **Statistics show patterns** - Median/quartiles reveal more than average
 5. **Transparency beats complexity** - Simple formulas everyone understands
 6. **Comparability is king** - Better to exclude than force a bad comp
 7. **Document your choices** - Explain which metrics and why in notes section
-

Output Checklist

Before delivering a comp analysis, verify:

- ☐ All companies are truly comparable
 - ☐ Data is from consistent time periods
 - ☐ Units are clearly labeled (millions/billions)
 - ☐ Formulas reference cells, not hardcoded values
 - ☐ **All hard-coded input cells have comments with either: (1) exact data source with citation, OR (2) clear assumption with explanation**
 - ☐ **Hyperlinks added where relevant** (SEC EDGAR filings, Bloomberg pages, research reports)
 - ☐ Statistics include at least 5 metrics (Max, 75th, Med, 25th, Min)
 - ☐ Notes section documents sources and methodology
 - ☐ Visual formatting follows conventions (blue = input, black = formula)
 - ☐ Sanity checks pass (margins logical, multiples reasonable)
 - ☐ Date stamp is current ("As of [Date]")
 - ☐ Formula auditing shows no errors (#DIV/0!, #REF!, #N/A)
-

Continuous Improvement

After completing a comp analysis, ask:

1. Did the statistics reveal unexpected insights?
2. Were there any data gaps that limited analysis?
3. Did stakeholders ask for metrics you didn't include?
4. How long did it take vs. how long should it take?
5. What would make this more useful next time?

The best comp analyses evolve with each iteration. Save templates, learn from feedback, and refine the structure based on what decision-makers actually use.

3.26 Skill Resource: skills/DCF-model/requirements.txt

Structured File Content

- # DCF Model Builder - Python Dependencies
 - # Excel file handling
 - openpyxl>=3.0.0
 - # HTTP requests
 - requests>=2.28.0
-

3.27 Script File: skills/DCF-model/scripts/validate_DCF.py

Script Summary

Signal	Details
Imports	sys, json, pathlib import Path, typing import Optional
Classes	DCFModelValidator
Functions	validate_DCF_model, main
Entry point	Defines an executable main entry point

Normalized Script Content

- `#!/usr/bin/env python3`
- `"""`
- DCF Model Validation Script
- Validates Excel DCF models for formula errors and common DCF mistakes
- `"""`
- `import sys`
- `import json`
- `from pathlib import Path`
- `from typing import Optional`
- `class DCFModelValidator:`
- `"""Validates DCF models for errors and quality issues"""`
- `def __init__(self, excel_path: str):`

- try:
- import openpyxl
- except ImportError:
- raise ImportError("openpyxl not installed. Run: pip install openpyxl")
- self.excel_path = excel_path
- self.openpyxl = openpyxl
- if not Path(excel_path).exists():
- raise FileNotFoundError(f"File not found: {excel_path}")
- self.workbook_formulas = openpyxl.load_workbook(excel_path, data_only=False)
- self.workbook_values = openpyxl.load_workbook(excel_path, data_only=True)
- self.errors = []
- self.warnings = []
- self.info = []
- def validate_all(self) -> dict:
- `"""`
- Run all validation checks
- Returns:
- Dict with validation results
- `"""`
- from datetime import datetime
- self.check_sheet_structure()
- self.check_formula_errors()
- self.check_dcf_logic()
- results = {
- 'file': self.excel_path,
- 'validation_date': datetime.now().isoformat(),
- 'status': 'PASS' if len(self.errors) == 0 else 'FAIL',
- 'error_count': len(self.errors),
- 'warning_count': len(self.warnings),
- 'errors': self.errors,
- 'warnings': self.warnings,
- 'info': self.info

- }
- return results
- def check_sheet_structure(self):
- """Verify required sheets exist"""
- required_sheets = ['DCF', 'WACC', 'Sensitivity']
- sheet_names = self.workbook_values.sheetnames
- for sheet in required_sheets:
- if sheet not in sheet_names:
- self.warnings.append(f'Recommended sheet missing: {sheet}')
- else:
- self.info.append(f'Found sheet: {sheet}')
- def check_formula_errors(self):
- """Check for Excel formula errors in all sheets"""
- excel_errors = ['#VALUE!', '#DIV/0!', '#REF!', '#NAME?', '#NULL!', '#NUM!', '#N/A']
- error_details = {err: [] for err in excel_errors}
- total_errors = 0
- total_formulas = 0
- for sheet_name in self.workbook_values.sheetnames:
- ws_values = self.workbook_values[sheet_name]
- ws_formulas = self.workbook_formulas[sheet_name]
- for row in ws_values.iter_rows():
- for cell in row:
- formula_cell = ws_formulas[cell.coordinate]
- # Count formulas
- if formula_cell.value and isinstance(formula_cell.value, str) and formula_cell.value.startswith('='):
- total_formulas += 1
- # Check for errors
- if cell.value is not None and isinstance(cell.value, str):
- for err in excel_errors:
- if err in cell.value:
- location = f'{sheet_name}!{cell.coordinate}'
- error_details[err].append(location)

- `total_errors += 1`
- `self.errors.append(f'{err} at {location}')`
- `break`
- `# Add summary info`
- `self.info.append(f'Total formulas: {total_formulas}')`
- `if total_errors == 0:`
- `self.info.append("✓ No formula errors found")`
- `else:`
- `self.errors.append(f'Total formula errors: {total_errors}')`
- `return error_details, total_errors`
- `def check_dcf_logic(self):`
- `"""Validate DCF-specific logic and calculations"""`
- `self._check_terminal_growth_vs_wacc()`
- `self._check_wacc_range()`
- `self._check_terminal_value_proportion()`
- `def _check_terminal_growth_vs_wacc(self):`
- `"""Critical check: Terminal growth must be less than WACC"""`
- `try:`
- `dcf_sheet = self.workbook_values['DCF']`
- `terminal_growth = None`
- `wacc = None`
- `# Search for terminal growth and WACC values`
- `for row in dcf_sheet.iter_rows(max_row=100, max_col=20):`
- `for cell in row:`
- `if cell.value and isinstance(cell.value, str):`
- `cell_str = cell.value.lower()`
- `if 'terminal' in cell_str and 'growth' in cell_str:`
- `# Look for value in adjacent cells`
- `for offset in range(1, 5):`
- `adjacent = dcf_sheet.cell(cell.row, cell.column + offset).value`
- `if isinstance(adjacent, (int, float)) and 0 < adjacent < 1:`
- `terminal_growth = adjacent`

- break
- if 'wacc' in cell_str and wacc is None:
- for offset in range(1, 5):
- adjacent = dcf_sheet.cell(cell.row, cell.column + offset).value
- if isinstance(adjacent, (int, float)) and $0 < \text{adjacent} < 1$:
- wacc = adjacent
- break
- if terminal_growth is not None and wacc is not None:
- if terminal_growth $\geq$ wacc:
- self.errors.append(
 - f"CRITICAL: Terminal growth ({terminal_growth:.2%}) $\geq$ WACC ({wacc:.2%}). "
 - "This creates infinite value and is mathematically invalid."
-)
- else:
- self.info.append(
 - f"✓ Terminal growth ({terminal_growth:.2%}) $<$ WACC ({wacc:.2%})"
-)
- else:
- self.warnings.append("Could not locate terminal growth and WACC values")
- except KeyError:
- self.warnings.append("DCF sheet not found")
- except Exception as e:
- self.warnings.append(f"Could not validate terminal growth vs WACC: {str(e)}")
- def _check_wacc_range(self):
- """Check if WACC is in reasonable range"""
- try:
- wacc_sheet = self.workbook_values.get('WACC') or self.workbook_values['DCF']
- wacc = None
- for row in wacc_sheet.iter_rows(max_row=100, max_col=20):
- for cell in row:
- if cell.value and isinstance(cell.value, str):
- if 'wacc' in cell.value.lower():

- for offset in range(1, 5):
- adjacent = wacc_sheet.cell(cell.row, cell.column + offset).value
- if isinstance(adjacent, (int, float)) and 0 < adjacent < 1:
- wacc = adjacent
- break
- if wacc is not None:
- if wacc < 0.05 or wacc > 0.20:
- self.warnings.append(
- f"WACC ({wacc:.2%}) is outside typical range (5%-20%). Verify calculation."
-)
- else:
- self.info.append(f"✓ WACC ({wacc:.2%}) in reasonable range")
- else:
- self.warnings.append("Could not locate WACC value")
- except Exception as e:
- self.warnings.append(f"Could not validate WACC range: {str(e)}")
- def _check_terminal_value_proportion(self):
- """Check if terminal value is reasonable proportion of enterprise value"""
- try:
- dcf_sheet = self.workbook_values['DCF']
- terminal_value = None
- enterprise_value = None
- for row in dcf_sheet.iter_rows(max_row=200, max_col=20):
- for cell in row:
- if cell.value and isinstance(cell.value, str):
- cell_str = cell.value.lower()
- if 'terminal' in cell_str and 'value' in cell_str and 'pv' in cell_str:
- for offset in range(1, 5):
- adjacent = dcf_sheet.cell(cell.row, cell.column + offset).value
- if isinstance(adjacent, (int, float)) and adjacent > 0:
- terminal_value = adjacent
- break

- if 'enterprise' in cell_str and 'value' in cell_str:
- for offset in range(1, 5):
- adjacent = dcf_sheet.cell(cell.row, cell.column + offset).value
- if isinstance(adjacent, (int, float)) and adjacent > 0:
- enterprise_value = adjacent
- break
- if terminal_value is not None and enterprise_value is not None and enterprise_value > 0:
- proportion = terminal_value / enterprise_value
- if proportion > 0.80:
- self.warnings.append(
- f"Terminal value is {proportion:.1%} of EV (typically should be 50-70%). "
- "Model may be over-reliant on terminal assumptions."
-)
- elif proportion < 0.40:
- self.warnings.append(
- f"Terminal value is {proportion:.1%} of EV (typically should be 50-70%). "
- "Check if terminal assumptions are too conservative."
-)
- else:
- self.info.append(f"✓ Terminal value is {proportion:.1%} of EV")
- else:
- self.warnings.append("Could not locate terminal value and enterprise value")
- except Exception as e:
- self.warnings.append(f"Could not validate terminal value proportion: {str(e)}")
- def validate_dcf_model(excel_path: str) -> dict:
- """
- Validate a DCF model Excel file
- Args:
- excel_path: Path to Excel DCF model
- Returns:
- Dict with validation results
- """

- `validator = DCFModelValidator(excel_path)`
- `return validator.validate_all()`
- `def main():`
- `"""Command-line interface"""`
- `if len(sys.argv) < 2:`
- `print("Usage: python validate_dcf.py [output.json]")`
- `print("\nValidates DCF model for:")`
- `print(" - Formula errors (#REF!, #DIV/0!, etc.)")`
- `print(" - Terminal growth < WACC (critical)")`
- `print(" - WACC in reasonable range (5-20%)")`
- `print(" - Terminal value proportion of EV (40-80%)")`
- `print("\nReturns JSON with errors, warnings, and info")`
- `print("\nExample: python validate_dcf.py model.xlsx")`
- `print("Example: python validate_dcf.py model.xlsx results.json")`
- `sys.exit(1)`
- `excel_file = sys.argv[1]`
- `output_file = sys.argv[2] if len(sys.argv) > 2 else None`
- `try:`
- `results = validate_dcf_model(excel_file)`
- `# Print results`
- `print(json.dumps(results, indent=2))`
- `# Save to file if requested`
- `if output_file:`
- `with open(output_file, 'w') as f:`
- `json.dump(results, f, indent=2)`
- `# Exit with error code if validation failed`
- `sys.exit(0 if results['status'] == 'PASS' else 1)`
- `except Exception as e:`
- `error_result = {`
- `'file': excel_file,`
- `'status': 'ERROR',`
- `'error': str(e)`

- }
 - print(json.dumps(error_result, indent=2))
 - sys.exit(1)
 - if __name__ == "__main__":
 - main()
-

3.28 Skill: dcf-model

File Metadata

Field	Value
name	dcf-model
description	Real DCF (Discounted Cash Flow) model creation for equity valuation. Retrieves financial data from SEC filings and analyst reports, builds comprehensive cash flow projections with proper WACC calculations, performs sensitivity analysis, and outputs professional Excel models with executive summaries. Use when users need to value a company using DCF methodology, request intrinsic value analysis, or ask for detailed financial modeling with growth projections and terminal value calculations.

DCF Model Builder

Overview

This skill creates institutional-quality DCF models for equity valuation following investment banking standards. Each analysis produces a detailed Excel model (with sensitivity analysis included at the bottom of the DCF sheet).

Tools

- Default to using all of the information provided by the user and MCP servers available for data sourcing.

Critical Constraints - Read These First

These constraints apply throughout all DCF model building. Review before starting:

Environment: Office JS vs Python/openpyxl:

- **If running inside Excel (Office Add-in / Office JS environment):** Use Office JS directly — do NOT use Python/openpyxl. Write formulas via `range.formulas = [["=D19*(1+$B$8)"]]`. No separate recalc step needed; Excel calculates natively. Use `range.format.*` for styling. The same formulas-over-hardcodes rule applies: set `.formulas`, never `.values` for derived cells.

- **If generating a standalone .xlsx file (no live Excel session):** Use Python/openpyxl as described below, then run *recalc.py* before delivery.
- The rest of this skill uses openpyxl examples — translate to Office JS API calls when in that environment, but all principles (formula strings, cell comments, section checkpoints, sensitivity table loops) apply identically.

⚠ **Office JS merged cell pitfall:** When building section headers with merged cells, do NOT call *.merge()* then set *.values* on the merged range — Office JS still reports the range’s original dimensions and will throw *InvalidArgument: The number of rows or columns in the input array doesn’t match the size or dimensions of the range*. Instead, write the value to the top-left cell alone, then merge and format the full range:

Web Template Summary

- Contains implementation-level web template code for rendering the report.
- HTML/CSS/JavaScript implementation lines are intentionally omitted in print output for readability.

This applies to every merged section header in the DCF (market data, scenario blocks, cash flow projection, terminal value, valuation summary, sensitivity tables).

Formulas Over Hardcodes (NON-NEGOTIABLE):

- Every projection, margin, discount factor, PV, and sensitivity cell MUST be a live Excel formula — never a value computed in Python and written as a number
- When using openpyxl: `ws["D20"] = "=D19*(1+$B$8)"` is correct; `ws["D20"] = calculated_revenue` is WRONG
- The only hardcoded numbers permitted are: (1) raw historical inputs, (2) assumption drivers (growth rates, WACC inputs, terminal g), (3) current market data (share price, debt balance)
- If you catch yourself computing something in Python and writing the result — STOP. The model must flex when the user changes an assumption.

Verify Step-by-Step With the User (DO NOT build end-to-end):

- After data retrieval → show the user the raw inputs block (revenue, margins, shares, net debt) and confirm before projecting
- After revenue projections → show the projected top line and growth rates, confirm before building margin build
- After FCF build → show the full FCF schedule, confirm logic before computing WACC
- After WACC → show the calculation and inputs, confirm before discounting
- After terminal value + PV → show the equity bridge (EV → equity value → per share), confirm before sensitivity tables
- Catch errors at each stage — a wrong margin assumption discovered after sensitivity tables are built means rebuilding everything downstream

Sensitivity Tables:

- **Use an ODD number of rows and columns** (standard: 5×5, sometimes 7×7) — this guarantees a true center cell

- **Center cell = base case.** Build the axis values so the middle row header and middle column header exactly equal the model's actual assumptions (e.g., if base WACC = 9.0%, the middle row is 9.0%; if terminal g = 3.0%, the middle column is 3.0%). The center cell's output must therefore equal the model's actual implied share price — this is the sanity check that the table is built correctly.
- **Highlight the center cell** with the medium-blue fill (`#BDD7EE`) + bold font so it's immediately visible which cell is the base case.
- Populate ALL cells (typically 3 tables × 25 cells = 75) with full DCF recalculation formulas
- Use openpyxl loops (or Office JS loops) to write formulas programmatically
- NO placeholder text, NO linear approximations, NO manual steps required
- Each cell must recalculate full DCF for that assumption combination

Cell Comments:

- Add cell comments AS each hardcoded value is created
- Format: “Source: [System/Document], [Date], [Reference], [URL if applicable]”
- Every blue input must have a comment before moving to next section
- Do not defer to end or write “TODO: add source”

Model Layout Planning:

- Define ALL section row positions BEFORE writing any formulas
- Write ALL headers and labels first
- Write ALL section dividers and blank rows second
- THEN write formulas using the locked row positions
- Test formulas immediately after creation

Formula Recalculation:

- Run *python recalc.py model.xlsx 30* before delivery
- Fix ALL errors until status is “success”
- Zero formula errors required (`#REF!`, `#DIV/0!`, `#VALUE!`, etc.)

Scenario Blocks:

- Create separate blocks for Bear/Base/Bull cases
- Show assumptions horizontally across projection years within each block
- Use IF formulas: `=IF($B$6=1,[Bear cell],IF($B$6=2,[Base cell],[Bull cell]))`
- Verify formulas reference correct scenario block cells

DCF Process Workflow

Step 1: Data Retrieval and Validation

Fetch data from MCP servers, user provided data, and the web.

Data Sources Priority:

1. **MCP Servers** (if configured) - Structured financial data from providers like Daloopa
2. **User-Provided Data** - Historical financials from their research
3. **Web Search/Fetch** - Current prices, beta, debt and cash when needed

Validation Checklist:

- Verify net debt vs net cash (critical for valuation)
- Confirm diluted shares outstanding (check for recent buybacks/issuances)
- Validate historical margins are consistent with business model
- Cross-check revenue growth rates with industry benchmarks
- Verify tax rate is reasonable (typically 21-28%)

Step 2: Historical Analysis (3-5 years)

Analyze and document:

- **Revenue growth trends:** Calculate CAGR, identify drivers
- **Margin progression:** Track gross margin, EBIT margin, FCF margin
- **Capital intensity:** D&A and CapEx as % of revenue
- **Working capital efficiency:** NWC changes as % of revenue growth
- **Return metrics:** ROIC, ROE trends

Create summary tables showing:

Structured Template

- Historical Metrics (LTM):
- Revenue: \$X million
- Revenue growth: X% CAGR
- Gross margin: X%
- EBIT margin: X%
- D&A % of revenue: X%
- CapEx % of revenue: X%
- FCF margin: X%

Step 3: Build Revenue Projections

Methodology:

1. Start with latest actual revenue (LTM or most recent fiscal year)
2. Apply growth rates for each projection year
3. Show both dollar amounts AND calculated growth %

Growth Rate Framework:

- Year 1-2: Higher growth reflecting near-term visibility
- Year 3-4: Gradual moderation toward industry average
- Year 5+: Approaching terminal growth rate

Formula structure:

- $\text{Revenue}(\text{Year } N) = \text{Revenue}(\text{Year } N-1) \times (1 + \text{Growth Rate})$
- $\text{Growth \%}(\text{Year } N) = \frac{\text{Revenue}(\text{Year } N) - \text{Revenue}(\text{Year } N-1)}{\text{Revenue}(\text{Year } N-1)} \times 100$

Three-scenario approach:

Structured Template

- Bear Case: Conservative growth (e.g., 8-12%)
- Base Case: Most likely scenario (e.g., 12-16%)
- Bull Case: Optimistic growth (e.g., 16-20%)

Step 4: Operating Expense Modeling

Fixed/Variable Cost Analysis:

Operating expenses should model realistic operating leverage:

- **Sales & Marketing:** Typically 15-40% of revenue depending on business model
- **Research & Development:** Typically 10-30% for technology companies
- **General & Administrative:** Typically 8-15% of revenue, shows leverage as company scales

Key principles:

- ALL percentages based on REVENUE, not gross profit
- Model operating leverage: % should decline as revenue scales
- Maintain separate line items for S&M, R&D, G&A
- Calculate EBIT = Gross Profit - Total OpEx

Margin expansion framework:

Structured Template

- Current State → Target State (Year 5)
- Gross Margin: X% → Y% (justify based on scale, efficiency)
- EBIT Margin: X% → Y% (result of revenue growth + opex leverage)

Step 5: Free Cash Flow Calculation

Build FCF in proper sequence:

Structured Template

- EBIT
- (-) Taxes ($\text{EBIT} \times \text{Tax Rate}$)
- = NOPAT (Net Operating Profit After Tax)
- (+) D&A (non-cash expense, % of revenue)
- (-) CapEx (% of revenue, typically 4-8%)
- (-) Δ NWC (change in working capital)
- = Unlevered Free Cash Flow

Working Capital Modeling:

- Calculate as % of revenue change (delta revenue)
- Typical range: -2% to +2% of revenue change
- Negative number = source of cash (working capital release)
- Positive number = use of cash (working capital build)

Maintenance vs Growth CapEx:

- Maintenance CapEx: Sustains current operations (~2-3% revenue)
- Growth CapEx: Supports expansion (additional 2-5% revenue)
- Total CapEx should align with company's growth strategy

Step 6: Cost of Capital (WACC) Research

CAPM Methodology for Cost of Equity:

Structured Template

- Cost of Equity = Risk-Free Rate + Beta $\times$ Equity Risk Premium
- Where:
 - - Risk-Free Rate = Current 10-Year Treasury Yield
 - - Beta = 5-year monthly stock beta vs market index
 - - Equity Risk Premium = 5.0-6.0% (market standard)

Cost of Debt Calculation:

Structured Template

- After-Tax Cost of Debt = Pre-Tax Cost of Debt $\times$ (1 - Tax Rate)
- Determine Pre-Tax Cost of Debt from:
 - - Credit rating (if available)
 - - Current yield on company bonds
 - - Interest expense / Total Debt from financials

Capital Structure Weights:

Structured Template

- Market Value Equity = Current Stock Price $\times$ Shares Outstanding
- Net Debt = Total Debt - Cash & Equivalents
- Enterprise Value = Market Cap + Net Debt
- Equity Weight = Market Cap / Enterprise Value
- Debt Weight = Net Debt / Enterprise Value
- WACC = (Cost of Equity $\times$ Equity Weight) + (After-Tax Cost of Debt $\times$ Debt Weight)

Special Cases:

- **Net Cash Position:** If Cash > Debt, Net Debt is NEGATIVE
 - Debt Weight may be negative
 - WACC calculation adjusts accordingly
- **No Debt:** WACC = Cost of Equity

Typical WACC Ranges:

- Large Cap, Stable: 7-9%
- Growth Companies: 9-12%
- High Growth/Risk: 12-15%

Step 7: Discount Rate Application (5-10 Year Forecast)

Mid-Year Convention:

- Cash flows assumed to occur mid-year
- Discount Period: 0.5, 1.5, 2.5, 3.5, 4.5, etc.
- Discount Factor = $1 / (1 + \text{WACC})^{\text{Period}}$

Present Value Calculation:

Structured Template

- For each projection year:
- $PV \text{ of FCF} = \text{Unlevered FCF} \times \text{Discount Factor}$
- Example (Year 1):
- $FCF = \$1,000$
- $WACC = 10\%$
- $\text{Period} = 0.5$
- $\text{Discount Factor} = 1 / (1.10)^{0.5} = 0.9535$
- $PV = \$1,000 \times 0.9535 = \954

Projection Period Selection:

- **5 years:** Standard for most analyses
- **7-10 years:** High growth companies with longer runway
- **3 years:** Mature, stable businesses

Step 8: Terminal Value Calculation

Perpetuity Growth Method (Preferred):

Structured Template

- $\text{Terminal FCF} = \text{Final Year FCF} \times (1 + \text{Terminal Growth Rate})$
- $\text{Terminal Value} = \text{Terminal FCF} / (WACC - \text{Terminal Growth Rate})$
- Critical Constraint: $\text{Terminal Growth} < WACC$ (otherwise infinite value)

Terminal Growth Rate Selection:

- Conservative: 2.0-2.5% (GDP growth rate)
- Moderate: 2.5-3.5%
- Aggressive: 3.5-5.0% (only for market leaders)

Do not exceed: Risk-free rate or long-term GDP growth

Exit Multiple Method (Alternative):

Structured Template

- $\text{Terminal Value} = \text{Final Year EBITDA} \times \text{Exit Multiple}$
- Where Exit Multiple comes from:
 - - Industry comparable trading multiples
 - - Precedent transaction multiples
 - - Typical range: 8-15x EBITDA

Present Value of Terminal Value:

Structured Template

- $PV \text{ of Terminal Value} = \text{Terminal Value} / (1 + WACC)^{\text{Final Period}}$
- Where Final Period accounts for timing:
- 5-year model with mid-year convention: Period = 4.5

Terminal Value Sanity Check:

- Should represent 50-70% of Enterprise Value
- If >75%, model may be over-reliant on terminal assumptions
- If <40%, check if terminal assumptions are too conservative

Step 9: Enterprise to Equity Value Bridge

Valuation Summary Structure:

Structured Template

- (+) Sum of PV of Projected FCFs = \$X million
- (+) PV of Terminal Value = \$Y million
- = Enterprise Value = \$Z million
- (-) Net Debt [or + Net Cash if negative] = \$A million
- = Equity Value = \$B million
- ÷ Diluted Shares Outstanding = C million shares
- = Implied Price per Share = \$XX.XX
- Current Stock Price = \$YY.YY
- Implied Return = $(\text{Implied Price} / \text{Current Price}) - 1 = XX\%$

Critical Adjustments:

- **Net Debt = Total Debt - Cash & Equivalents**
 - If positive: Subtract from EV (reduces equity value)
 - If negative (Net Cash): Add to EV (increases equity value)
- **Use Diluted Shares:** Includes options, RSUs, convertible securities
- **Other adjustments** (if applicable):
 - Minority interests
 - Pension liabilities
 - Operating lease obligations

Valuation Output Format:

Structured Template

- Valuation Component, Amount (\$M)
- PV Explicit FCFs, X.X
- PV Terminal Value, Y.Y
- Enterprise Value, Z.Z
- (-) Net Debt, A.A
- Equity Value, B.B
- „
- Shares Outstanding (M), C.C
- Implied Price per Share, \$XX.XX
- Current Share Price, \$YY.YY
- Implied Upside/(Downside), +XX%

Step 10: Sensitivity Analysis

Build **three sensitivity tables** at the bottom of the DCF sheet showing how valuation changes with different assumptions:

1. **WACC vs Terminal Growth** - Shows enterprise value sensitivity to discount rate and perpetuity growth
2. **Revenue Growth vs EBIT Margin** - Shows impact of top-line growth and operating leverage
3. **Beta vs Risk-Free Rate** - Shows sensitivity to cost of equity components

Implementation: These are simple 2D grids (NOT Excel's "Data Table" feature) with formulas in each cell. Each cell must contain a full DCF recalculation for that specific assumption combination. See Critical Constraints section for detailed requirements on populating all 75 cells programmatically using openpyxl.

This section contains all the CORRECT patterns to follow when building DCF models.

Scenario Block Selection Pattern - Follow This Approach

Assumptions are organized in separate blocks for each scenario:

CRITICAL STRUCTURE - Three rows per section header:

Structured Template

- BEAR CASE ASSUMPTIONS (section header, merge cells across)
- Assumption, FY1, FY2, FY3, FY4, FY5
- Revenue Growth (%), 12%, 10%, 9%, 8%, 7%
- EBIT Margin (%), 45%, 44%, 43%, 42%, 41%
- BASE CASE ASSUMPTIONS (section header, merge cells across)
- Assumption, FY1, FY2, FY3, FY4, FY5
- Revenue Growth (%), 16%, 14%, 12%, 10%, 9%

- EBIT Margin (%),48%,49%,50%,51%,52%
- BULL CASE ASSUMPTIONS (section header, merge cells across)
- Assumption,FY1,FY2,FY3,FY4,FY5
- Revenue Growth (%),20%,18%,15%,13%,11%
- EBIT Margin (%),50%,51%,52%,53%,54%

Each scenario block MUST have a column header row showing the projection years (FY2025E, FY2026E, etc.) immediately below the section title. Without this, users cannot tell which assumption value corresponds to which year.

How to reference assumptions - Create a consolidation column:

1. Case selector cell (e.g., B6) contains 1=Bear, 2=Base, or 3=Bull
2. Create a consolidation column with INDEX or OFFSET formulas to pull from the correct scenario block
3. Projection formulas reference the consolidation column (clean cell references)
4. Each scenario block contains full set of DCF assumptions across projection years

Recommended consolidation column pattern (using INDEX): `=INDEX(B10:D10, 1, $B$6)`

NOT this - scattered IF statements throughout: `=IF($B$6=1,[Bear block cell],IF($B$6=2,[Base block cell],[Bull block cell]))`

The consolidation column approach centralizes logic and makes the model easier to audit.

Correct Revenue Projection Pattern

Create a consolidation column with INDEX formulas, then reference it in projections:

Step 1 - Consolidation column for FY1 growth: `=INDEX([Bear FY1 growth]:[Bull FY1 growth], 1, $B$6)`

Step 2 - Revenue projection references the consolidation column: *Revenue Year 1:* `=D29*(1+$E$10)`

Where:

- D29 = Prior year revenue
- \$E\$10 = Consolidation column cell for FY1 growth (contains INDEX formula)
- \$B\$6 = Case selector (1=Bear, 2=Base, 3=Bull)

This approach is cleaner than embedding IF statements in every projection formula and makes it much easier to audit which scenario assumptions are being used.

Correct FCF Formula Pattern

Use consolidation columns with INDEX formulas, then reference them in FCF calculations:

Consolidation column approach:

Structured Template

- Item,Formula,Reference

- $D\&A, = E29 * \$E\$21, \$E\21 = consolidation column for D&A %
- $CapEx, = E29 * \$E\$22, \$E\22 = consolidation column for CapEx %
- $\Delta NWC, = (E29 - D29) * \$E\$23, \$E\$23$ = consolidation column for NWC %
- $Unlevered\ FCF, = E57 + E58 - E60 - E62, E57 = NOPAT\ E58 = D\&A\ E60 = CapEx\ E62 = \Delta NWC$

Each consolidation column cell contains an INDEX formula that pulls from the appropriate scenario block based on case selector. This keeps projection formulas clean and auditable.

Before writing formulas, confirm scenario block row locations and set up consolidation columns.

Correct Cell Comment Format

Every hardcoded value needs this format:

“Source: [System/Document], [Date], [Reference], [URL if applicable]”

Examples:

Structured Template

- Item,Source Comment
- Stock price,Source: Market data script 2025-10-12 Close price
- Shares outstanding,Source: 10-K FY2024 Page 45 Note 12
- Historical revenue,Source: 10-K FY2024 Page 32 Consolidated Statements
- Beta,Source: Market data script 2025-10-12 5-year monthly beta
- Consensus estimates,Source: Management guidance Q3 2024 earnings call

Correct Assumption Table Structure

CRITICAL: Each scenario block requires THREE structural elements:

1. **Section header row** (merged cells): e.g., “BEAR CASE ASSUMPTIONS”
2. **Column header row** showing years - THIS IS REQUIRED, DO NOT SKIP
3. **Data rows** with assumption values

Structure:

Structured Template

- BEAR CASE ASSUMPTIONS (section header - merge across columns A:G)
- Assumption,FY1,FY2,FY3,FY4,FY5
- Revenue Growth (%),X%,X%,X%,X%,X%
- EBIT Margin (%),X%,X%,X%,X%,X%
- Terminal Growth,X%,,,,
- WACC,X%,,,,

- BASE CASE ASSUMPTIONS (section header - merge across columns A:G)
- Assumption,FY1,FY2,FY3,FY4,FY5
- Revenue Growth (%),X%,X%,X%,X%,X%
- EBIT Margin (%),X%,X%,X%,X%,X%
- Terminal Growth,X%,,,,
- WACC,X%,,,,
- BULL CASE ASSUMPTIONS (section header - merge across columns A:G)
- Assumption,FY1,FY2,FY3,FY4,FY5
- Revenue Growth (%),X%,X%,X%,X%,X%
- EBIT Margin (%),X%,X%,X%,X%,X%
- Terminal Growth,X%,,,,
- WACC,X%,,,,

WITHOUT the column header row showing projection years (FY2025E, FY2026E, etc.), users cannot tell which assumption value corresponds to which year. This row is MANDATORY.

Then create a consolidation column (typically the next column to the right) that uses INDEX formulas to pull from the selected scenario block based on the case selector. This consolidation column is what your projection formulas reference.

Correct Row Planning Process

1. Write ALL headers and labels FIRST:

Structured Template

- Row,Content
- 1,[Company Name] DCF Model
- 2,Ticker | Date | Year End
- 4,Case Selector
- 7,KEY ASSUMPTIONS
- 26,Assumption headers
- 27-31,Growth assumptions
- ...,...

2. Write ALL section dividers and blank rows

3. THEN write formulas using the locked row positions

4. Test formulas immediately after creation

Think of it like construction:

- Good: Pour foundation, then build walls (stable structure)

- Bad: Build walls, then pour foundation (walls collapse)

Excel version:

- Good: Add headers, then write formulas (formulas stable)
- Bad: Write formulas, then add headers (formulas break)

Correct Sensitivity Table Implementation

IMPORTANT: These are NOT Excel’s “Data Table” feature. These are simple grids where you write regular formulas using openpyxl. Yes, this means ~75 formulas total (3 tables × 25 cells each), but this is straightforward and required.

Programmatic Population with Formulas:

Each sensitivity table must be fully populated with formulas that recalculate the implied share price for each combination of assumptions. **Do not use Excel’s Data Table feature** (it requires manual intervention and cannot be automated via openpyxl).

Implementation approach - CONCRETE EXAMPLE:

Table Structure — 5×5 grid (ODD dimensions, base case centered):

If the model’s base WACC = 9.0% and base terminal growth = 3.0%, build the axes symmetrically around those values:

Structured Template

- WACC vs Terminal Growth, 2.0%, 2.5%, 3.0%, 3.5%, 4.0%
- 8.0%, [fml], [fml], [fml], [fml], [fml]
- 8.5%, [fml], [fml], [fml], [fml], [fml]
- 9.0%, [fml], [fml], [★], [fml], [fml] ← middle row = base WACC
- 9.5%, [fml], [fml], [fml], [fml], [fml]
- 10.0%, [fml], [fml], [fml], [fml], [fml]
- ↑
- middle col = base terminal g

★ = **the center cell**. Its formula output **MUST** equal the model’s actual implied share price (from the valuation summary). Apply the medium-blue fill (`#BDD7EE`) and bold font to this cell so the base case is visually anchored.

Rule for axis values: *axis_values* = [base - 2*step, base - step, base, base + step, base + 2*step] — symmetric around the base, odd count guarantees a center.

Formula Pattern - Cell B88 (WACC=8.0%, Terminal Growth=2.0%):

The formula in B88 should recalculate the implied price using:

- WACC from row header: `$A88` (8.0%)
- Terminal Growth from column header: `B$87` (2.0%)

Recommended approach: Reference the main DCF calculation but substitute these values.

Example formula structure:
$$=([SUM\ of\ PV\ FCFs\ using\ \$A88\ as\ discount\ rate] + [Terminal\ Value\ using\ B\$87\ as\ growth\ rate\ and\ \$A88\ as\ WACC] - [Net\ Debt]) / [Shares]$$

CRITICAL - Write a formula for EVERY cell in the 5x5 grid (25 cells per table, 75 cells total). Use openpyxl to write these formulas programmatically in a loop. Do NOT skip this step or leave placeholder text.

Python implementation pattern:

Structured Template

- # Pseudocode for populating sensitivity table
- for row_idx, wacc_value in enumerate(wacc_range):
- for col_idx, term_growth_value in enumerate(term_growth_range):
- # Build formula that uses wacc_value and term_growth_value
- formula = f"="
- ws.cell(row=start_row+row_idx, column=start_col+col_idx).value = formula

The sensitivity tables must work immediately when the model is opened, with no manual steps required from the user.

This section contains all the WRONG patterns to avoid when building DCF models.

WRONG: Simplified Sensitivity Table Approximations or Placeholder Text

Don't use linear approximations:

Structured Template

- // WRONG - Linear approximation
- B97: =B88*(1+(0.096-0.116)) // Assumes linear relationship
- // WRONG - Division shortcut
- B105: =B88/(1+(E48-0.07)) // Doesn't recalculate full DCF

Don't leave placeholder text:

Structured Template

- // WRONG - Placeholder note
- "Note: Use Excel Data Table feature (Data → What-If Analysis → Data Table) to populate sensitivity tables."
- // WRONG - Empty cells
- [leaving cells blank because "this is complex"]

Don't confuse terminology:

- ❌ "Sensitivity tables need Excel's Data Table feature" (NO - that's a specific Excel tool we can't use)
- ✅ "Sensitivity tables are simple grids with formulas in each cell" (YES - this is what we build)

Why these shortcuts are wrong:

- Linear approximation formulas don't actually recalculate the DCF - they just apply simple math adjustments
- The relationships are not linear, so the results will be inaccurate
- Placeholder text requires manual user intervention
- Model is not immediately usable when delivered
- Not professional or client-ready
- Empty cells = incomplete deliverable

Common rationalization to REJECT: "Writing 75+ formulas feels complex, so I'll leave a note for the user to complete it manually."

Reality: Writing 75 formulas is straightforward when you use a loop in Python with openpyxl. Each formula follows the same pattern - just substitute the row/column values. This is a required part of the deliverable.

Instead: Populate every sensitivity cell with formulas that recalculate the full DCF for that specific combination of assumptions

WRONG: Missing Cell Comments

Don't do this:

- Create all hardcoded inputs without comments
- Think "I'll add them later"
- Write "TODO: add source"
- Leave blue inputs without documentation

Why it's wrong:

- Can't verify where data came from
- Fails xlsx skill requirements
- Not audit-ready
- Wastes time fixing later

Instead: Add cell comment AS EACH hardcoded value is created

WRONG: Formula Row References Off

Symptom: The FCF section references wrong assumption rows: *D&A: =E29*\$E\$34 // Should be \$E\$21, but referencing wrong row CapEx: =E29*\$E\$41 // Should be \$E\$22, but row shifted*

Why this happens:

1. Formulas written first
2. Then headers inserted

3. All row references shifted
4. Now formulas point to wrong cells → #REF! errors

Instead: Lock row layout FIRST, then write formulas

WRONG: Single Row for Each Assumption Across Scenarios

Don't structure assumptions like this:

Structured Template

- Assumption, Bear, Base, Bull
- Revenue Growth FY1, 10%, 13%, 16%
- Revenue Growth FY2, 9%, 12%, 15%

This vertical layout makes it hard to see the progression across years within each scenario.

Why it's wrong:

- Makes it difficult to see assumptions evolving across years within each scenario
- Harder to compare scenario assumptions across full projection period
- Less intuitive for reviewing scenario logic

Instead:

- Create separate blocks for each scenario (Bear, Base, Bull)
- Within each block, show assumptions horizontally across projection years
- This makes each scenario's assumptions easier to review as a cohesive set

WRONG: No Borders

Don't deliver a model without borders:

- No section delineation
- All cells blend together
- Hard to read and unprofessional

Why it's wrong:

- Not client-ready
- Difficult to navigate
- Looks amateur

Instead: Add borders around all major sections

WRONG: Wrong Font Colors or No Font Color Distinction

Don't do this:

- All text is black
- Only use fill colors (no font color changes)
- Mix up which cells are blue vs black

Why it's wrong:

- Can't distinguish inputs from formulas
- Auditing becomes impossible
- Violates xlsx skill requirements

Instead: Blue text for ALL hardcoded inputs, black text for ALL formulas, green for sheet links

WRONG: Operating Expenses Based on Gross Profit

Don't do this: *S&M: =E33*0.15 // E33 = Gross Profit (WRONG)*

Why it's wrong:

- Operating expenses scale with revenue, not gross profit
- Produces unrealistic margin progression
- Not how businesses actually operate

Instead: *S&M: =E29*0.15 // E29 = Revenue (CORRECT)*

TOP 5 ERRORS SUMMARY

1. **Formula row references off** → Define ALL row positions BEFORE writing formulas
2. **Missing cell comments** → Add comments AS cells are created, not at end
3. **Simplified sensitivity tables** → Populate all cells with full DCF recalc formulas, not approximations
4. **Scenario block references wrong** → Ensure IF formulas pull from correct Bear/Base/Bull blocks
5. **No borders** → Add professional section borders for client-ready appearance

In addition, be aware of these errors:

WACC Calculation Errors

- Mixing book and market values in capital structure
- Using equity beta instead of asset/unlevered beta incorrectly
- Wrong tax rate application to cost of debt
- Incorrect risk-free rate (must use current 10Y Treasury)
- Failure to adjust for net debt vs net cash position

Growth Assumption Flaws

- Terminal growth > WACC (creates infinite value)
- Projection growth rates inconsistent with historical performance
- Ignoring industry growth constraints
- Revenue growth not aligned with unit economics
- Margin expansion without operational justification

Terminal Value Mistakes

- Using wrong growth method (perpetuity vs exit multiple)
- Terminal value >80% of enterprise value (suggests over-reliance)
- Inconsistent terminal margins with steady state assumptions
- Wrong discount period for terminal value

Cash Flow Projection Errors

- Operating expenses based on gross profit instead of revenue
- D&A/CapEx percentages misaligned with business model
- Working capital changes not properly calculated
- Tax rate inconsistency between years
- NOPAT calculation errors

These errors are the most common. Re-read this section before starting any DCF build.

Excel File Creation

This skill uses the xlsx skill for all spreadsheet operations. The xlsx skill provides:

- Standardized formula construction rules
- Number formatting conventions
- Automated formula recalculation via *recalc.py* script
- Comprehensive error checking and validation

All Excel files created by this skill must follow xlsx skill requirements, including zero formula errors and proper recalculation.

Quality Rubric

Every DCF model must maximize for:

1. **Realistic revenue and margin assumptions** based on historical performance

2. **Appropriate cost of capital calculation** with proper CAPM methodology
3. **Comprehensive sensitivity analysis** showing valuation ranges
4. **Clear terminal value calculation** with supporting rationale
5. **Professional model structure** enabling scenario analysis
6. **Transparent documentation** of all key assumptions

Input Requirements

Minimum Required Inputs

1. **Company identifier:** Ticker symbol or company name
2. **Growth assumptions:** Revenue growth rates for projection period (or “use consensus”)
3. **Optional parameters:**
 - Projection period (default: 5 years)
 - Scenario cases (Bear/Base/Bull growth and margin assumptions)
 - Terminal growth rate (default: 2.5-3.0%)
 - Specific WACC inputs if not using CAPM

Excel Model Structure

Sheet Architecture

Create **two sheets**:

1. **DCF** - Main valuation model with sensitivity analysis at bottom
2. **WACC** - Cost of capital calculation

CRITICAL: Sensitivity tables go at the **BOTTOM** of the DCF sheet (not on a separate sheet). This keeps all valuation outputs together.

Formula Recalculation (MANDATORY)

After creating or modifying the Excel model, **recalculate all formulas** using the recalc.py script from the xlsx skill:

Structured Template

- `python recalc.py [path_to_excel_file] [timeout_seconds]`

Example:

Structured Template

- `python recalc.py AAPL_DCF_Model_2025-10-12.xlsx 30`

The script will:

- Recalculate all formulas in all sheets using LibreOffice
- Scan ALL cells for Excel errors (#REF!, #DIV/0!, #VALUE!, #NAME?, #NULL!, #NUM!, #N/A)
- Return detailed JSON with error locations and counts

Expected output format:

Structured Template

- {
- “status”: “success”, // or “errors_found”
- “total_errors”: 0, // Total error count
- “total_formulas”: 42, // Number of formulas in file
- “error_summary”: {} // Only present if errors found
- }

If errors are found, the output will include details:

Structured Template

- {
- “status”: “errors_found”,
- “total_errors”: 2,
- “total_formulas”: 42,
- “error_summary”: {
- “#REF!”: {
- “count”: 2,
- “locations”: [“DCF!B25”, “DCF!C25”]
- }
- }
- }

Fix all errors and re-run recal.py until status is “success” before delivering the model.

Formatting Standards

IMPORTANT: Follow the xlsx skill for formula construction rules and number formatting conventions. The DCF skill adds specific visual presentation standards.

Color Scheme - Two Layers:

Layer 1: Font Colors (MANDATORY from xlsx skill)

- **Blue text (RGB: 0,0,255):** ALL hardcoded inputs (stock price, shares, historical data, assumptions)

- **Black text (RGB: 0,0,0):** ALL formulas and calculations
- **Green text (RGB: 0,128,0):** Links to other sheets (WACC sheet references)

Layer 2: Fill Colors — Professional Blue/Grey Palette (Default unless user specifies otherwise)

- **Keep it minimal** — use only blues and greys for fills. Do NOT introduce greens, yellows, oranges, or multiple accent colors. A model with too many colors looks amateurish.
- **Default fill palette:**
 - **Section headers:** Dark blue (RGB: 31,78,121 / #1F4E79) background with white bold text
 - **Sub-headers/column headers:** Light blue (RGB: 217,225,242 / #D9E1F2) background with black bold text
 - **Input cells:** Light grey (RGB: 242,242,242 / #F2F2F2) background with blue font — or just white with blue font if you want maximum minimalism
 - **Calculated cells:** White background with black font
 - **Output/summary rows** (per-share value, EV, etc.): Medium blue (RGB: 189,215,238 / #BDD7EE) background with black bold font
- **That’s it — 3 blues + 1 grey + white.** Resist the urge to add more.
- User-provided templates or explicit color preferences ALWAYS override these defaults.

How the layers work together:

- Input cell: Blue font + light grey fill = “Hardcoded input”
- Formula cell: Black font + white background = “Calculated value”
- Sheet link: Green font + white background = “Reference from another sheet”
- Key output: Black bold font + medium blue fill = “This is the answer”

Font color tells you WHAT it is (input/formula/link). Fill color tells you WHERE you are (header/data/output).

Border Standards (REQUIRED for Professional Appearance)

Thick borders (1.5pt) around major sections:

- KEY INPUTS section
- PROJECTION ASSUMPTIONS section
- 5-YEAR CASH FLOW PROJECTION section
- TERMINAL VALUE section
- VALUATION SUMMARY section
- Each SENSITIVITY ANALYSIS table

Medium borders (1pt) between sub-sections:

- Company Details vs Historical Performance

- Growth Assumptions vs EBIT Margin vs FCF Parameters

Thin borders (0.5pt) around data tables:

- Scenario assumption tables (Bear | Base | Bull | Selected)
- Historical vs projected financials matrix

No borders: Individual cells within tables (keep clean, scannable)

Borders are mandatory - models without professional borders are not client-ready.

Number Formats (follows xlsx skill standards):

- **Years:** Format as text strings (e.g., “2024” not “2,024”)
- **Percentages:** 0.0% (one decimal place)
- **Currency:** \$#,##0 for millions; \$#,##0.00 for per-share - ALWAYS specify units in headers (“Revenue (\$mm)”)
- **Zeros:** Use number formatting to make all zeros “-” (e.g., \$#,##0;(\$#,##0);-)
- **Large numbers:** #,##0 with thousands separator
- **Negative numbers:** (#,##0) in parentheses (NOT minus sign)

Cell Comments (MANDATORY for all hardcoded inputs):

Per the xlsx skill, ALL hardcoded values must have cell comments documenting the source. Format: “Source: [System/Document], [Date], [Reference], [URL if applicable]”

CRITICAL: Add comments AS CELLS ARE CREATED. Do not defer to the end.

DCF Sheet Detailed Structure

Section 1: Header

Structured Template

- Row,Content
- 1,[Company Name] DCF Model
- 2,Ticker: [XXX] | Date: [Date] | Year End: [FYE]
- 3,Blank
- 4,Case Selector Cell (1=Bear 2=Base 3=Bull)
- 5,Case Name Display (formula: =IF([Selector]=1“Bear”IF([Selector]=2“Base”“Bull”)))

Section 2: Market Data (NOT case dependent)

Structured Template

- Item,Value
- Current Stock Price,\$XX.XX
- Shares Outstanding (M),XX.X

- Market Cap (\$M),[Formula]
- Net Debt (\$M),XXX [or Net Cash if negative]

Section 3: DCF Scenario Assumptions

Create separate assumption blocks for each scenario (Bear, Base, Bull) with DCF-specific assumptions (Revenue Growth %, EBIT Margin %, Tax Rate %, D&A % of Revenue, CapEx % of Revenue, NWC Change % of Δ Rev, Terminal Growth Rate, WACC) laid out horizontally across projection years. Each block must include section header, column header row showing the projection years (FY1, FY2, etc.), and data rows. See “ section “Correct Assumption Table Structure” for the exact layout.

Section 4: Historical & Projected Financials

Reference a consolidation column (e.g., “Selected Case”) that pulls from scenario blocks, not scattered IF formulas in every projection row.

Structured Template

- Income Statement (\$M),2020A,2021A,2022A,2023A,2024E,2025E,2026E
- Revenue,XXX,XXX,XXX,XXX,[=E29(1+\$E\$10)],[=F29(1+\$E\$11)],[=G29*(1+\$E\$12)]
- % growth,XX%,XX%,XX%,XX%, [=E29/D29-1], [=F29/E29-1], [=G29/F29-1]
- ,,,,,,
- Gross Profit,XXX,XXX,XXX,XXX,[=E29E33],[=F29F33],[=G29*G33]
- % margin,XX%,XX%,XX%,XX%, [=E33/E29], [=F33/F29], [=G33/G29]
- ,,,,,,
- Operating Expenses:,,,,,,
- S&M,XXX,XXX,XXX,XXX,[=E290.15],[=F290.14],[=G29*0.13]
- R&D,XXX,XXX,XXX,XXX,[=E290.12],[=F290.11],[=G29*0.10]
- G&A,XXX,XXX,XXX,XXX,[=E290.08],[=F290.07],[=G29*0.07]
- Total OpEx,XXX,XXX,XXX,XXX,[=E36+E37+E38],[=F36+F37+F38],[=G36+G37+G38]
- ,,,,,,
- EBIT,XXX,XXX,XXX,XXX,[=E33-E39],[=F33-F39],[=G33-G39]
- % margin,XX%,XX%,XX%,XX%, [=E41/E29], [=F41/F29], [=G41/G29]
- ,,,,,,
- Taxes,(XX),(XX),(XX),(XX), [=E41\$E\$24],[=F41\$E\$24],[=G41*\$E\$24]
- Tax rate,XX%,XX%,XX%,XX%, [=E43/E41],[=F43/F41],[=G43/G41]
- ,,,,,,
- NOPAT,XXX,XXX,XXX,XXX,[=E41-E43],[=F41-F43],[=G41-G43]

Key Formula Pattern:

- Revenue growth: $=E29*(1+$E$10)$ where \$E\$10 is consolidation column for Year 1 growth

- NOT: $=E29*(1+IF(\$B\$6=1,\$B\$10,IF(\$B\$6=2,\$C\$10,\$D\$10)))$

This approach is cleaner, easier to audit, and prevents formula errors by centralizing the scenario logic.

Section 5: Free Cash Flow Build

CRITICAL: Verify row references point to the CORRECT assumption rows. Test formulas immediately after creation.

Structured Template

- Cash Flow (\$M),2020A,2021A,2022A,2023A,2024E,2025E,2026E
- NOPAT,XXX,XXX,XXX,XXX,[=E45],[=F45],[=G45]
- (+) D&A,XXX,XXX,XXX,XXX,[=E29\$E\$21],[=F29\$E\$21],[=G29*\$E\$21]
- % of Rev,XX%,XX%,XX%,XX%, [=E58/E29],[=F58/F29],[=G58/G29]
- (-) CapEx,(XX),(XX),(XX),(XX), [=E29\$E\$22],[=F29\$E\$22],[=G29*\$E\$22]
- % of Rev,XX%,XX%,XX%,XX%, [=E60/E29],[=F60/F29],[=G60/G29]
- (-) Δ NWC,(XX),(XX),(XX),(XX), [= (E29-D29)\$E\$23],[= (F29-E29)\$E\$23],[= (G29-F29)*\$E\$23]
- % of Δ Rev,XX%,XX%,XX%,XX%, [=E62/(E29-D29)],[=F62/(F29-E29)],[=G62/(G29-F29)]
- ,,,,,,
- Unlevered FCF,XXX,XXX,XXX,XXX,[=E57+E58-E60-E62],[=F57+F58-F60-F62],[=G57+G58-G60-G62]

Row reference examples (based on layout planning):

- \$E\$21 = D&A % assumption (consolidation column, row 21)
- \$E\$22 = CapEx % assumption (consolidation column, row 22)
- \$E\$23 = NWC % assumption (consolidation column, row 23)
- E29 = Revenue for year (row 29)
- E45 = NOPAT for year (row 45)

Before writing formulas: Confirm these row numbers match the actual layout. Test one column, then copy across.

Section 6: Discounting & Valuation

Structured Template

- DCF Valuation,2024E,2025E,2026E,2027E,2028E,Terminal
- Unlevered FCF (\$M),XXX,XXX,XXX,XXX,XXX,
- Period,0.5,1.5,2.5,3.5,4.5,
- Discount Factor,0.XX,0.XX,0.XX,0.XX,0.XX,
- PV of FCF (\$M),XXX,XXX,XXX,XXX,XXX,
- ,,,,,,
- Terminal FCF (\$M),,,,,,XXX

- Terminal Value (\$M),,,,,,XXX
- PV Terminal Value (\$M),,,,,,XXX
- ,,,,,,
- Valuation Summary (\$M),,,,,,
- Sum of PV FCFs,XXX,.,.,.
- PV Terminal Value,XXX,.,.,.
- Enterprise Value,XXX,.,.,.
- (-) Net Debt,(XX),.,.,.
- Equity Value,XXX,.,.,.
- ,,,,,,
- Shares Outstanding (M),XX.X,.,.,.
- IMPLIED PRICE PER SHARE,\$XX.XX,.,.,.
- Current Stock Price,\$XX.XX,.,.,.
- Implied Upside/(Downside),XX%,.,.,.

WACC Sheet Structure

Structured Template

- COST OF EQUITY CALCULATION,,
- Risk-Free Rate (10Y Treasury),X.XX%,[Yellow input]
- Beta (5Y monthly),X.XX,[Yellow input]
- Equity Risk Premium,X.XX%,[Yellow input]
- Cost of Equity,X.XX%,[Calculated blue]
- ,,
- COST OF DEBT CALCULATION,,
- Credit Rating,AA-, [Yellow input]
- Pre-Tax Cost of Debt,X.XX%,[Yellow input]
- Tax Rate,XX.X%,[Link to DCF sheet]
- After-Tax Cost of Debt,X.XX%,[Calculated blue]
- ,,
- CAPITAL STRUCTURE,,
- Current Stock Price,\$XX.XX,[Link to DCF]
- Shares Outstanding (M),XX.X,[Link to DCF]

- Market Capitalization (\$M), “X,XXX”, [Calculated]
- „
- Total Debt (\$M), XXX, [Yellow input]
- Cash & Equivalents (\$M), XXX, [Yellow input]
- Net Debt (\$M), XXX, [Calculated]
- „
- Enterprise Value (\$M), “X,XXX”, [Calculated]
- „
- WACC CALCULATION, Weight, Cost, Contribution
- Equity, XX.X%, X.X%, X.XX%
- Debt, XX.X%, X.X%, X.XX%
- „
- WEIGHTED AVERAGE COST OF CAPITAL, X.XX%, [Green output]

Key WACC Formulas:

Structured Template

- Market Cap = Price × Shares
- Net Debt = Total Debt - Cash
- Enterprise Value = Market Cap + Net Debt
- Equity Weight = Market Cap / EV
- Debt Weight = Net Debt / EV
- WACC = (Cost of Equity × Equity Weight) + (After-tax Cost of Debt × Debt Weight)

Sensitivity Analysis (Bottom of DCF Sheet)

TERMINOLOGY REMINDER: “Sensitivity tables” = simple 2D grids with row headers, column headers, and formulas in each data cell. NOT Excel’s “Data Table” feature (Data → What-If Analysis → Data Table). You will use openpyxl to write regular Excel formulas into each cell.

Location: Rows 87+ on DCF sheet (NOT a separate sheet)

Three sensitivity tables, vertically stacked:

1. **WACC vs Terminal Growth** (rows 87-100) - 5x5 grid = 25 cells with formulas
2. **Revenue Growth vs EBIT Margin** (rows 102-115) - 5x5 grid = 25 cells with formulas
3. **Beta vs Risk-Free Rate** (rows 117-130) - 5x5 grid = 25 cells with formulas

Total formulas to write: 75 (this is required, not optional)

CRITICAL: All sensitivity table cells must be populated programmatically with formulas using openpyxl. DO NOT use linear approximation shortcuts. DO NOT leave placeholder text or notes about manual steps. DO NOT rationalize leaving cells empty because “it’s complex” - use a Python loop to generate the formulas.

Table Setup:

1. Create table structure with row/column headers (the assumption values to test)
2. Populate EVERY data cell with a formula that:
 - Uses the row header value (e.g., WACC = 9.0%)
 - Uses the column header value (e.g., Terminal Growth = 3.0%)
 - Recalculates the full DCF with those specific assumptions
 - Returns the implied share price for that scenario
3. All cells must contain working formulas when delivered
4. Format cells with conditional formatting: Green scale for higher values, red scale for lower values
5. Bold the base case cell
6. Leave 1-2 blank rows between tables

No manual intervention required - the sensitivity tables must be fully functional when the user opens the file.

Case Selector Implementation

Three-Case Framework:

Bear Case

- Conservative revenue growth (low end of historical range)
- Margin compression or no expansion
- Higher WACC (risk premium increase)
- Lower terminal growth rate
- Higher CapEx assumptions

Base Case

- Consensus or management guidance revenue growth
- Moderate margin expansion based on operating leverage
- Current market-implied WACC
- GDP-aligned terminal growth (2.5-3.0%)
- Standard CapEx assumptions

Bull Case

- Optimistic revenue growth (high end of projections)
- Significant margin expansion
- Lower WACC (reduced risk premium)
- Higher terminal growth (3.5-5.0%)
- Reduced CapEx intensity

Formula Implementation:

DO NOT use nested IF formulas scattered throughout. Instead, create a consolidation column that uses INDEX or OFFSET formulas to pull from the appropriate scenario block.

Recommended pattern (using INDEX): $=INDEX(B10:D10, I, \$B\$6)$ where $B10:D10$ = Bear/Base/Bull values, I = row offset, $\$B\6 = case selector cell (1, 2, or 3)

Then reference the consolidation column in all projections: *Revenue Year 1:* $=D29*(1+\$E\$10)$ where $\$E\10 is the consolidation column value for Year 1 growth.

This approach centralizes scenario logic, making the model easier to audit and maintain.

Deliverables Structure

File naming: *[Ticker]_DCF_Model_[Date].xlsx*

Two sheets:

1. **DCF** - Complete model with Bear/Base/Bull cases + three sensitivity tables at bottom (WACC vs Terminal Growth, Revenue Growth vs EBIT Margin, Beta vs Risk-Free Rate)
2. **WACC** - Cost of capital calculation

Key features: Case selector (1/2/3), consolidation column with INDEX/OFFSET formulas, color-coded cells, cell comments on all inputs, professional borders

Best Practices

Model Construction

1. **Build incrementally:** Complete each section before moving to next
2. **Test as building:** Enter sample numbers to verify formulas
3. **Use consistent structure:** Similar calculations follow similar patterns
4. **Comment complex formulas:** Add notes for unusual calculations
5. **Build in checks:** Sum checks and balance checks where applicable

Documentation

1. **Document all assumptions:** Explain reasoning behind key inputs
2. **Cite data sources:** Note where each data point came from
3. **Explain methodology:** Describe any non-standard approaches
4. **Flag uncertainties:** Highlight areas with limited visibility

Quality Control

1. **Cross-check calculations:** Verify math in multiple ways
2. **Stress test assumptions:** Run sensitivity to ensure model is robust
3. **Peer review:** Have someone else check formulas
4. **Version control:** Save versions as work progresses

Common Variations

High-Growth Technology Companies

- Longer projection period (7-10 years)
- Higher initial growth rates (20-30%)
- Significant margin expansion over time
- Higher WACC (12-15%)
- Model unit economics (users, ARPU, etc.)

Mature/Stable Companies

- Shorter projection period (3-5 years)
- Modest growth rates (GDP +1-3%)
- Stable margins
- Lower WACC (7-9%)
- Focus on cash generation and capital allocation

Cyclical Companies

- Model through economic cycle
- Normalize margins at mid-cycle
- Consider trough and peak scenarios
- Adjust beta for cyclicalities

Multi-Segment Companies

- Separate DCFs for each business unit
- Different growth rates and margins by segment
- Sum-of-parts valuation
- Consider synergies

Troubleshooting

If you encounter errors or unreasonable results, read [TROUBLESHOOTING.md](#) for detailed debugging guidance.

Workflow Integration

At Start of DCF Build

1. **Gather market data:**
 - Check for available MCP servers for current market data
 - Use web search/fetch for stock prices, beta, and other market metrics
 - Request from user if specific data is needed
2. **Gather historical financials:**
 - Check for available MCP servers (Daloopa, etc.)
 - Request from user if not available via MCP
 - Manual extraction from 10-Ks if necessary
3. **Begin model construction** using the DCF methodology detailed in this skill

During Model Construction

1. **Build Excel model** using openpyxl with formulas (not hardcoded values)
2. **Follow xlsx skill conventions** for formula construction and formatting
3. **Apply fill colors only if requested** by user or if specific brand guidelines are provided

Before Delivering Model (MANDATORY)

1. **Verify structure:**
 - Scenario blocks for Bear/Base/Bull with assumptions across projection years
 - Case selector functional with formulas referencing correct scenario blocks
 - Sensitivity tables at bottom of DCF sheet (not separate sheet)
 - Font colors: Blue inputs, black formulas, green sheet links

- Cell comments on ALL hardcoded inputs
 - Professional borders around major sections
2. **Recalculate formulas:** Run *python recalc.py model.xlsx 30*
 3. **Check output:**
 - If *status* is “*success*” → Continue to step 4
 - If *status* is “*errors_found*” → Check *error_summary* and read [TROUBLESHOOTING.md](#) for debugging guidance
 4. **Fix errors and re-run recalc.py** until status is “success”
 5. **Spot-check formulas:**
 - Test one FCF formula - does it reference the correct assumption rows?
 - Change case selector - does the consolidation column update properly?
 - Verify revenue formulas reference consolidation column (not nested IF formulas)
 6. **Deliver model**

Available Data Sources

- **MCP servers:** If configured (Daloopa for historical financials)
- **Web search/fetch:** For current stock prices, beta, and market data
- **User-provided data:** Historical financials, consensus estimates
- **Manual extraction:** SEC EDGAR filings as fallback

Final Output Checklist

Before delivering DCF model:

Required:

- Run *python recalc.py model.xlsx 30* until status is “success” (zero formula errors)
- Two sheets: DCF (with sensitivity at bottom), WACC
- Font colors: Blue=inputs, Black=formulas, Green=sheet links
- Cell comments on ALL hardcoded inputs
- Sensitivity tables fully populated with formulas
- Professional borders around major sections

Validation:

- OpEx based on revenue (not gross profit)
- Terminal value 50-70% of EV
- Terminal growth < WACC

- Tax rate 21-28%
 - File naming: *[Ticker]_DCF_Model_[Date].xlsx*
-

3.29 Skill Resource: skills/DCF-model/TROUBLESHOOTING.md

DCF Model Troubleshooting Guide

When to read this file: If recalculations show errors OR valuation results seem unreasonable OR case selector not working properly.

Model Returns Error Values

#REF! Errors

- Usually caused by formulas referencing wrong rows after headers were inserted
- Solution: Rebuild with correct row references, or start over following layout planning
- Prevention: Define all row positions BEFORE writing formulas

#DIV/0! Errors

- Division by zero or empty cells
- Solution: Add IF statements to handle zeros: $=IF([Divisor]=0,0,[Numerator]/[Divisor])$

#VALUE! Errors

- Wrong data type in calculation (text instead of number)
- Solution: Verify all inputs are formatted as numbers

Valuation Seems Unreasonable

Implied price far too high

- Check terminal value isn't >80% of EV
- Verify terminal growth < WACC
- Review if growth assumptions are realistic
- Consider if margins are too optimistic

Implied price far too low

- Verify net debt vs net cash is correct
- Check if WACC is too high
- Review if projections are too conservative
- Consider if terminal growth is too low

Case Selector Not Working

Consolidation column not updating when switching scenarios

- Verify case selector cell contains 1, 2, or 3
 - Check INDEX/OFFSET formulas reference correct row range and selector cell
 - Ensure absolute references (\$B\$6) are used for selector
 - Test by manually changing the selector cell and verifying projection values update
-

3.30 Skill: deck-refresh

File Metadata

Field	Value
name	deck-refresh
description	Updates a presentation with new numbers — quarterly refreshes, earnings updates, comp rolls, rebased market data. Use whenever the user asks to “update the deck with Q4 numbers”, “refresh the comps”, “roll this forward”, “swap in the new earnings”, “change all the \$485M to \$512M”, or any request to swap figures across an existing deck without rebuilding it.

Deck Refresh

Update numbers across the deck. The deck is the source of truth for formatting; you’re only changing values.

Environment check

This skill works in both the PowerPoint add-in and chat. Identify which you’re in before starting — the edit mechanism differs, the intent doesn’t:

- **Add-in** — the deck is open live; edit text runs, table cells, and chart data directly.
- **Chat** — the deck is an uploaded file; edit it by regenerating the affected slides with the new values and writing the result back.

Either way: smallest possible change, existing formatting stays intact.

This is a four-phase process and the third phase is an approval gate. Don't edit until the user has seen the plan.

Phase 1 — Get the data

Use *ask_user_question* to find out how the new numbers are arriving:

- **Pasted mapping** — user types or pastes “revenue \$485M → \$512M, EBITDA \$120M → \$135M.” The clearest case.
- **Uploaded Excel** — old/new columns, or a fresh output sheet the user wants pulled from. Read it, confirm which column is which before you trust it.
- **Just the new values** — “Q4 revenue was \$512M, margins were 22%.” You figure out what each one replaces. Workable, but confirm the mapping before you touch anything — a “\$512M” that you map to revenue but the user meant for gross profit is a quiet disaster.

Also ask about **derived numbers**: if revenue moves, does the user want growth rates and share percentages recalculated, or left alone? Most decks have “+15% YoY” baked in somewhere that's now stale. Whether to touch those is a judgment call the user should make, not you.

Phase 2 — Read everything, find everything

Read every slide. For each old value, find every instance — including the ones that don't look the same:

Variant	Example
Scale	<i>\$485M, \$0.485B, \$485,000,000</i>
Precision	<i>\$485M, \$485.0M, ~\$485M</i>
Unit style	<i>\$485M, \$485MM, \$485 million, 485M</i>
Embedded	“revenue grew to \$485M”, “a \$485M business”, axis labels

A deck that says *\$485M* on slide 3, *485* on slide 8's chart axis, and *\$485.0 million* in a footnote on slide 15 has three instances of the same number. Find-replace misses two of them. You shouldn't.

Where numbers hide:

- Text boxes (obvious)
- Table cells
- Chart data labels and axis labels
- Chart source data — the numbers driving the bars, not just the labels on them
- Footnotes, source lines, small print
- Speaker notes, if the user cares about those

Build a list: for each old value, every location it appears, the exact text it appears as, and what it'll become. This list is the plan.

Phase 3 — Present the plan, get approval

This is a destructive operation on a deck someone spent time on. Show the full change list before editing a single thing. Format it so it's scannable:

Structured Template

- \$485M → \$512M (Revenue)
- Slide 3 — Title box: “Revenue grew to \$485M”
- Slide 8 — Chart axis label: “485”
- Slide 15 — Footnote: “\$485.0 million in FY24 revenue”
- \$120M → \$135M (Adj. EBITDA)
- Slide 3 — Table cell
- Slide 11 — Body text: “\$120M of Adj. EBITDA”
- FLAGGED — possibly derived, not in your mapping:
- Slide 3 — “+15% YoY” (growth rate — stale if base year didn't change?)
- Slide 7 — “12% market share” (was this computed from \$485M / market size?)

The flagged section matters. You're not just executing a find-replace — you're catching the second-order effects the user would've missed at 11pm. If the mapping says *\$485M* → *\$512M* and slide 3 also has *+15% YoY* right next to it, that growth rate is probably wrong now. Flag it; don't silently fix it, don't silently leave it.

Use *ask_user_question* for the approval: proceed as shown, proceed but skip the flagged items, or let them revise the mapping first.

Phase 4 — Execute, preserve, report

For each change, make the smallest edit that accomplishes it. How that happens depends on your environment:

- **Add-in** — edit the specific run, cell, or chart series directly in the live deck.
- **Chat** — regenerate the affected slide with the new value in place, preserving every other element exactly as it was, and write it back to the file.

Either way, the standard is the same:

- **Text in a shape** — change the value, leave font/size/color/bold state exactly as they were. If *\$485M* is 14pt navy bold inside a sentence, *\$512M* is 14pt navy bold inside the same sentence.
- **Table cell** — change the cell, leave the table alone.
- **Chart data** — update the underlying series values so the bars/lines actually move. Editing just the label without the data leaves a chart that lies.

Don't reformat anything you didn't need to touch. The deck's existing style is correct by definition; you're a surgeon, not a renovator.

After the last edit, report what actually happened:

Structured Template

- Updated 11 values across 8 slides.
- Changed:
- [the list from Phase 3, now past-tense]
- Still flagged — did NOT change:
- Slide 3 — “+15% YoY” (derived; confirm separately)
- Slide 7 — “12% market share”

Run standard visual verification checks on every edited slide. A number that got longer ($\$485M \rightarrow \$1,205M$) might now overflow its text box or push a table column width. Catch it before the user does.

What you’re not doing

- **Not rebuilding slides** — if a slide’s narrative no longer makes sense with the new numbers (“margins compressed” but margins went up), flag it, don’t rewrite it.
- **Not recalculating unless asked** — derived numbers are the user’s call. Your Phase 1 question covers this.
- **Not touching formatting** — if the deck uses $\$MM$ and the user’s mapping says $\$M$, match the deck, not the mapping. Values change; style stays.

3.31 Skill Resource: [skills/ib-check-deck/references/ib-terminology.md](#)

IB Terminology Reference

Casual to Professional Replacements

Casual/ Informal	IB Standard
“a lot of growth”	“significant growth” or “X% growth”
“pretty good margins”	“attractive margins” or “margins of X%”
“they bought the company”	“the company was acquired”
“big deal”	“transformative transaction”
“cheap valuation”	“attractive valuation” or “valuation discount”
“expensive”	“premium valuation”
“make more money”	“enhance profitability” or “drive margin expansion”
“getting bigger”	“pursuing growth” or “expanding operations”
“cut costs”	“implement cost optimization” or “drive operational efficiencies”
“good fit”	“strategic fit” or “compelling strategic rationale”
“help with”	“support” or “facilitate”
“a bunch of”	“multiple” or “numerous”
“kind of” / “sort of”	[remove or be specific]
“really” / “very”	[remove or quantify]

“tons of”	“substantial” or quantify
“huge”	“significant” or quantify
“pretty much”	[remove or be precise]
“basically”	[remove or clarify]

Language Patterns to Avoid

- **Contractions:** Don’t → Do not, won’t → will not
- **Exclamation points:** Generally inappropriate for IB materials
- **First-person:** “We think...” → “Management believes...” or passive voice
- **Superlatives without evidence:** “best-in-class” requires supporting data
- **Vague quantifiers:** “some”, “many”, “several” → specific numbers

Preferred Phrasing Patterns

Growth narratives:

- “Demonstrated track record of X% revenue CAGR”
- “Consistent margin expansion over [period]”
- “Proven ability to generate organic growth”

Market position:

- “#X player in [specific segment]”
- “Leading provider of [specific offering]”
- “Differentiated positioning through [specific attribute]”

Strategic rationale:

- “Compelling strategic fit driven by...”
 - “Attractive value creation opportunity through...”
 - “Synergy potential of \$Xm from [specific sources]”
-

3.32 Skill Resource: [skills/ib-check-deck/references/report-format.md](#)

Deck Check Report Format

Report Template

Structured Template

- # Deck Check Report: [Presentation Name]

- ## Summary
- - Total issues: X
- - Critical: X (number mismatches, factual errors)
- - Important: X (narrative-data alignment, language)
- - Minor: X (formatting)
- ## Critical Issues
- ### Number Consistency
- 1. **[Issue name]** (Slides X, Y)
- - Slide X: [value]
- - Slide Y: [value]
- - Action: [recommendation]
- ### Data-Narrative Alignment
- 1. **[Issue name]** (Slides X, Y)
- - Claim: “[quoted text]”
- - Data shows: [contradiction]
- - Action: [recommendation]
- ## Important Issues
- ### Language Polish
- 1. **[Issue type]** (Slide X)
- - Current: “[quoted text]”
- - Suggested: “[replacement]”
- ## Minor Issues
- ### Formatting
- 1. **[Issue type]** (Slide X)
- - [Description and fix]
- ## Final Checklist
- - [] Numbers reconciled
- - [] Narrative matches data
- - [] Language meets IB standards
- - [] Charts sourced
- - [] Formatting consistent

Issue Severity Classification

Critical (must fix before client delivery):

- Number mismatches across slides
- Calculation errors
- Factual inaccuracies (names, titles, dates)
- Data contradicting narrative

Important (should fix):

- Casual/informal language
- Vague claims without specificity
- Terminology inconsistency
- Missing chart sources

Minor (polish items):

- Font/color inconsistencies
 - Date format variations
 - Spacing/alignment issues
 - Orphaned text
-

3.33 Script File: skills/ib-check-deck/scripts/extract_numbers.py

Script Summary

Signal	Details
Imports	argparse, json, re, sys, collections import defaultdict, dataclasses import dataclass, asdict, pathlib import Path, typing import Optional
Classes	NumberInstance
Functions	normalize_number, detect_category, extract_numbers, find_inconsistencies, main
Entry point	Defines an executable main entry point
CLI	Exposes command-line argument parsing

Normalized Script Content

- `#!/usr/bin/env python3`
- `"""`
- Extract numerical values from presentation content for consistency checking.

- Usage:
- `python extract_numbers.py presentation-content.md`
- `python extract_numbers.py presentation-content.md --output numbers.json`
- This script parses markdown-formatted presentation content (from markitdown)
- and extracts all numerical values with their context and slide references.
- `"""`
- `import argparse`
- `import json`
- `import re`
- `import sys`
- `from collections import defaultdict`
- `from dataclasses import dataclass, asdict`
- `from pathlib import Path`
- `from typing import Optional`
- `@dataclass`
- `class NumberInstance:`
- `"""A numerical value found in the presentation."""`
- `value: str # Original string representation`
- `normalized: float # Normalized numeric value`
- `unit: str # Detected unit (M, B, K, %, bps, x, etc.)`
- `slide: int # Slide number (0 if unknown)`
- `context: str # Surrounding text for context`
- `line_number: int # Line number in source file`
- `category: str # Detected category (revenue, margin, multiple, etc.)`
- `def normalize_number(value_str: str, unit: str) -> float:`
- `"""Convert a number string with unit to a normalized float value."""`
- `# Remove commas and spaces`
- `clean = re.sub(r'[\s]', '', value_str)`
- `try:`
- `base_value = float(clean)`
- `except ValueError:`
- `return 0.0`

- # Apply unit multipliers
- multipliers = {
- 'T': 1e12,
- 'B': 1e9,
- 'bn': 1e9,
- 'billion': 1e9,
- 'M': 1e6,
- 'mm': 1e6,
- 'mn': 1e6,
- 'million': 1e6,
- 'K': 1e3,
- 'k': 1e3,
- 'thousand': 1e3,
- }
- for unit_key in sorted(multipliers.keys(), key=len, reverse=True):
- if unit_key.lower() in unit.lower():
- return base_value * multipliers[unit_key]
- return base_value
- def detect_category(context: str, unit: str) -> str:
- """Detect the category of a number based on context and unit."""
- context_lower = context.lower()
- # Revenue-related
- if any(term in context_lower for term in ['revenue', 'sales', 'top line', 'topline']):
- return 'revenue'
- # EBITDA-related
- if 'ebitda' in context_lower:
- if any(term in context_lower for term in ['margin', '%', 'percent']):
- return 'ebitda_margin'
- return 'ebitda'
- # Margin-related
- if any(term in context_lower for term in ['margin', 'profit']):
- return 'margin'

- # Growth-related
- if any(term in context_lower for term in ['growth', 'cagr', 'yoy', 'y/y']):
- return 'growth'
- # Valuation multiples
- if any(term in context_lower for term in ['multiple', 'ev/', 'p/e', 'ev/ebitda', 'ev/revenue']):
- return 'multiple'
- # Enterprise value / market cap
- if any(term in context_lower for term in ['enterprise value', 'ev ', 'market cap']):
- return 'valuation'
- # Percentage (generic)
- if unit in ['%', 'bps', 'percent']:
- return 'percentage'
- # Multiple indicator
- if unit == 'x':
- return 'multiple'
- return 'other'
- def extract_numbers(content: str) -> list[NumberInstance]:
- """Extract all numbers from presentation content."""
- numbers = []
- current_slide = 0
- # Pattern for slide markers (from markdown format)
- slide_pattern = re.compile(r'^#\sSlide\s(\d+)|^<!-- Slide (\d+)')
- # Pattern for numbers with various formats
- # Matches: \$500M, 500M, \$500 million, 25%, 25.5%, 2.5x, 150bps, \$1,234.56, etc.
- number_pattern = re.compile(
- r'(?P[€£¥])?' # Optional currency symbol
- r'(?P[\d,]+(?:\.\d+)?)' # The number itself
- r'\s*' #
- r'(?P[%|bps|x])' # Common units
- r'[Tt]rillion|[Bb]illion|[Mm]illion|[Tt]housand' # Full words
- r'[TBMKtbmk]n?[mm|MM])?' # Abbreviations
- r'(?!\d)' # Negative lookahead to avoid partial matches

-)
- lines = content.split('\n')
- for line_num, line in enumerate(lines, 1):
- # Check for slide marker
- slide_match = slide_pattern.match(line)
- if slide_match:
- current_slide = int(slide_match.group(1) or slide_match.group(2))
- continue
- # Find all numbers in the line
- for match in number_pattern.finditer(line):
- value_str = match.group('number')
- currency = match.group('currency') or ''
- unit = match.group('unit') or ''
- # Skip very short numbers without context (likely not financial)
- if len(value_str.replace(',', '').replace('.', '')) < 2 and not unit:
- continue
- # Skip year-like numbers (1900-2099) unless they have units
- try:
- num_val = float(value_str.replace(',', ''))
- if 1900 <= num_val <= 2099 and not unit and not currency:
- continue
- except ValueError:
- pass
- # Build full value string
- full_value = f'{currency}{value_str}{unit}'
- # Get context (surrounding words)
- start = max(0, match.start() - 50)
- end = min(len(line), match.end() + 50)
- context = line[start:end].strip()
- # Normalize unit
- if currency:
- if not unit:

- unit = 'USD' # Assume USD for \$ without unit
- else:
- unit = f"USD_{unit}"
- normalized = normalize_number(value_str, unit)
- category = detect_category(context, unit)
- numbers.append(NumberInstance(
- value=full_value,
- normalized=normalized,
- unit=unit or 'none',
- slide=current_slide,
- context=context,
- line_number=line_num,
- category=category
-))
- return numbers
- def find_inconsistencies(numbers: list[NumberInstance]) -> list[dict]:
- """Find potential inconsistencies in extracted numbers."""
- inconsistencies = []
- # Group numbers by category
- by_category = defaultdict(list)
- for num in numbers:
- if num.category != 'other':
- by_category[num.category].append(num)
- # Check each category for mismatches
- for category, instances in by_category.items():
- if len(instances) < 2:
- continue
- # Group by approximate value (within 5% tolerance)
- value_groups = []
- for inst in instances:
- placed = False
- for group in value_groups:

- `ref_value = group[0].normalized`
- `if ref_value > 0:`
- `diff_pct = abs(inst.normalized - ref_value) / ref_value`
- `if diff_pct < 0.05: # 5% tolerance`
- `group.append(inst)`
- `placed = True`
- `break`
- `if not placed:`
- `value_groups.append([inst])`
- `# If we have multiple groups, there might be inconsistencies`
- `if len(value_groups) > 1:`
- `# Sort groups by size (largest first)`
- `value_groups.sort(key=len, reverse=True)`
- `# The largest group is likely "correct", others are potential issues`
- `main_group = value_groups[0]`
- `for other_group in value_groups[1:]:`
- `inconsistencies.append({`
- `'category': category,`
- `'expected': {`
- `'value': main_group[0].value,`
- `'slides': sorted(set(n.slide for n in main_group)),`
- `'count': len(main_group)`
- `},`
- `'found': {`
- `'value': other_group[0].value,`
- `'slides': sorted(set(n.slide for n in other_group)),`
- `'count': len(other_group)`
- `},`
- `'severity': 'high' if category in ['revenue', 'ebitda', 'valuation'] else 'medium'`
- `})`
- `return inconsistencies`
- `def main():`

- `parser = argparse.ArgumentParser(`
- `description='Extract numbers from presentation content for consistency checking'`
- `)`
- `parser.add_argument('input_file', help='Markdown file with presentation content')`
- `parser.add_argument('-output', '-o', help='Output JSON file (default: stdout)')`
- `parser.add_argument('-check', '-c', action='store_true',`
- `help='Check for inconsistencies and report')`
- `args = parser.parse_args()`
- `# Read input`
- `input_path = Path(args.input_file)`
- `if not input_path.exists():`
- `print(f'Error: File not found: {args.input_file}', file=sys.stderr)`
- `sys.exit(1)`
- `content = input_path.read_text()`
- `# Extract numbers`
- `numbers = extract_numbers(content)`
- `# Prepare output`
- `output = {`
- `'total_numbers': len(numbers),`
- `'by_category': defaultdict(list),`
- `'numbers': [asdict(n) for n in numbers]`
- `}`
- `for num in numbers:`
- `output['by_category'][num.category].append({`
- `'value': num.value,`
- `'slide': num.slide,`
- `'context': num.context[:100]`
- `})`
- `output['by_category'] = dict(output['by_category'])`
- `# Check for inconsistencies if requested`
- `if args.check:`
- `inconsistencies = find_inconsistencies(numbers)`

- `output['inconsistencies'] = inconsistencies`
 - `if inconsistencies:`
 - `print("\n=== POTENTIAL INCONSISTENCIES DETECTED ===\n", file=sys.stderr)`
 - `for inc in inconsistencies:`
 - `print(f'Category: {inc['category'].upper()}', file=sys.stderr)`
 - `print(f' Expected: {inc['expected']['value']} (Slides: {inc['expected']['slides']}, Count: {inc['expected']['count']})', file=sys.stderr)`
 - `print(f' Found: {inc['found']['value']} (Slides: {inc['found']['slides']}, Count: {inc['found']['count']})', file=sys.stderr)`
 - `print(f' Severity: {inc['severity']}', file=sys.stderr)`
 - `print(file=sys.stderr)`
 - `# Output results`
 - `json_output = json.dumps(output, indent=2)`
 - `if args.output:`
 - `Path(args.output).write_text(json_output)`
 - `print(f'Output written to {args.output}', file=sys.stderr)`
 - `else:`
 - `print(json_output)`
 - `if __name__ == '__main__':`
 - `main()`
-

3.34 Skill: ib-check-deck

File Metadata

Field	Value
name	ib-check-deck
description	Investment banking presentation quality checker. Reviews a pitch deck or client-ready presentation for (1) number consistency across slides, (2) data-narrative alignment, (3) language polish against IB standards, (4) visual and formatting QC. Use whenever the user asks to review, check, QC, proof, or do a final pass on a deck, pitch, or client materials — including requests like “check my numbers”, “reconcile figures across slides”, “is this client-ready”, or “what am I missing before I send this out”.

IB Deck Checker

Perform comprehensive QC on the presentation across four dimensions. Read every slide, then report findings.

Environment check

This skill works in both the PowerPoint add-in and chat. Identify which you're in before starting:

- **Add-in** — read from the live open deck.
- **Chat** — read from the uploaded *.pptx* file.

This is read-and-report only — no edits — so the workflow is identical in both.

Workflow

Read the deck

Pull text from every slide, keeping track of which slide each line came from. You'll need slide-level attribution for every finding ("">\$500M appears on slides 3 and 8, but slide 15 shows \$485M"). A deck with 30 slides is too much to hold in working memory reliably — write the extracted text to a file so the number-checking script can process it.

The script expects markdown-ish input with slide markers. Format as:

Structured Template

- `## Slide 1`
- [slide 1 text content]
- `## Slide 2`
- [slide 2 text content]

1. Number consistency

Run the extraction script on what you collected:

Structured Template

- `python scripts/extract_numbers.py /tmp/deck_content.md --check`

It normalizes units (\$500M vs \$500MM vs \$500,000,000 → same number), categorizes values (revenue, EBITDA, multiples, margins), and flags when the same metric category shows conflicting values on different slides. This is the part most likely to catch something a human missed on the fifth read-through.

Beyond what the script flags, verify:

- Calculations are correct (totals sum, percentages add up, growth rates match the endpoints)
- Unit style is consistent — the deck should pick one of \$M or \$MM and stick with it
- Time periods are aligned — FY vs LTM vs quarterly, explicitly labeled

2. Data-narrative alignment

Map claims to the data that's supposed to support them. This is where decks go wrong quietly — someone edits the chart on slide 7 and forgets the narrative on slide 4.

- Trend statements (“declining margins”) → does the chart actually go that direction?
- Market position claims (“#1 player”) → revenue and share data support it?
- Plausibility — “#1 in a \$100B market” with \$200M revenue is 0.2% share; that's not #1

3. Language polish

IB decks have a register. Scan for anything that breaks it: casual phrasing (“pretty good”, “a lot of”), contractions, exclamation points, vague quantifiers without numbers, inconsistent terminology for the same concept.

See *references/ib-terminology.md* for replacement patterns.

4. Visual and formatting QC

Run standard visual verification checks on each slide. You're looking for: missing chart source citations, missing axis labels, typography inconsistencies, number formatting drift (1,000 vs 1K within the same deck), date format drift, footnote and disclaimer gaps.

Visual verification catches overlaps, overflow, and contrast issues that don't show up in text extraction. Don't skip it — a chart with no source citation looks the same as a properly sourced one in the text dump.

Output

Use *references/report-format.md* as the structure. Categorize by severity:

- **Critical** — number mismatches, factual errors, data contradicting narrative. These block client delivery.
- **Important** — language, missing sources, terminology drift. Should fix.
- **Minor** — font sizes, spacing, date formats. Polish.

Lead with criticals. If there aren't any, say so explicitly — “no number inconsistencies found” is a finding, not an absence of one.

3.35 Skill: lbo-model

File Metadata

Field	Value
name	lbo-model

description

This skill should be used when completing LBO (Leveraged Buyout) model templates in Excel for private equity transactions, deal materials, or investment committee presentations. The skill fills in formulas, validates calculations, and ensures professional formatting standards that adapt to any template structure.

TEMPLATE REQUIREMENT

This skill uses templates for LBO models. Always check for an attached template file first.

Before starting any LBO model:

1. **If a template file is attached/provided:** Use that template's structure exactly - copy it and populate with the user's data
2. **If no template is attached:** Ask the user: *"Do you have a specific LBO template you'd like me to use? If not, I can use the standard template which includes Sources & Uses, Operating Model, Debt Schedule, and Returns Analysis."*
3. **If using the standard template:** Copy *examples/LBO_Model.xlsx* as your starting point and populate it with the user's assumptions

IMPORTANT: When a file like *LBO_Model.xlsx* is attached, you **MUST** use it as your template - do not build from scratch. Even if the template seems complex or has more features than needed, copy it and adapt it to the user's requirements. Never decide to "build from scratch" when a template is provided.

CRITICAL INSTRUCTIONS FOR CLAUDE - READ FIRST

Environment: Office JS vs Python

If running inside Excel (Office Add-in / Office JS environment):

- Use Office JS (*Excel.run(async (context) => {...})*) directly — do NOT use Python/openpyxl
- Write formulas via *range.formulas = [["=B5*B6"]]* — Office JS formulas recalculate natively in the live workbook
- The same formulas-over-hardcodes rule applies: set *range.formulas*, never *range.values* for anything that should be a calculation
- Use *range.format.font.color* / *range.format.fill.color* for the blue/black/purple/green convention
- No separate recalc step needed — Excel handles calculation natively
- **Merged cell pitfall:** Do NOT call *.merge()* then set *.values* on the merged range (throws *InvalidArgument* — range still reports original dimensions). Instead: write value to top-left cell alone (*ws.getRange("A7").values = [["SOURCES & USES"]]*), then merge + format the full range (*ws.getRange("A7:F7").merge(); ws.getRange("A7:F7").format.fill.color = "#1F4E79";*)

If generating a standalone .xlsx file (no live Excel session):

- Use Python/openpyxl as described below
- Write formula strings (*ws["D20"] = "=B5*B6"*), then run *recalc.py* before delivery

The rest of this skill is written with openpyxl examples, but the same principles apply to Office JS — just translate the API calls.

Core Principles

- **Every calculation must be an Excel formula** - NEVER compute values in Python and hardcode results into cells. When using openpyxl, write *cell.value = "=B5*B6"* (formula string), NOT *cell.value = 1250* (computed result). The model must be dynamic and update when inputs change.
- **Use the template structure** - Follow the organization in *examples/LBO_Model.xlsx* or the user's provided template. Do not invent your own layout.
- **Use proper cell references** - All formulas should reference the appropriate cells. Never type numbers that should come from other cells.
- **Maintain sign convention consistency** - Follow whatever sign convention the template uses (some use negative for outflows, some use positive). Be consistent throughout.
- **Work section by section, verify with user at each step** - Complete one section fully, show the user what was built, run the section's verification checks, and get confirmation BEFORE moving to the next section. Do NOT build the entire model end-to-end and then present it — later sections depend on earlier ones, so catching a mistake in Sources & Uses after the returns are already built means rework everywhere.

Formula Color Conventions

- **Blue (0000FF)**: Hardcoded inputs - typed numbers that don't reference other cells
- **Black (000000)**: Formulas with calculations - any formula using operators or functions (*=B4*B5*, *=SUM()*, *=MAX(0,B4)*)
- **Purple (800080)**: Links to cells on the **same tab** - direct references with no calculation (*=B9*, *=B45*)
- **Green (008000)**: Links to cells on **different tabs** - cross-sheet references (*=Assumptions!B5*, *=Operating Model!C10*)

Fill Color Palette — Professional Blues & Greys (Default unless user/template specifies otherwise)

- **Keep it minimal** — only use blues and greys for cell fills. Do NOT introduce greens, yellows, reds, or multiple accents. A professional LBO model uses restraint.
- **Default fill palette:**
 - **Section headers** (Sources & Uses, Operating Model, etc.): Dark blue #1F4E79 with white bold text
 - **Column headers** (Year 1, Year 2, etc.): Light blue #D9E1F2 with black bold text
 - **Input cells**: Light grey #F2F2F2 (or just white) — the blue *font* is the signal, fill is secondary
 - **Formula/calculated cells**: White, no fill

- **Key outputs** (IRR, MOIC, Exit Equity): Medium blue #BDD7EE with black bold text
- **That’s the whole palette.** 3 blues + 1 grey + white. If the template uses its own colors, follow the template instead.
- Note: The blue/black/purple/green **font** colors above are for distinguishing inputs vs formulas vs links. Those are separate from the **fill** palette here — both work together.

Number Formatting Standards

- **Currency:** \$#,##0;(\$#,##0);“-” or \$#,##0.0 depending on template
 - **Percentages:** 0.0% (one decimal)
 - **Multiples:** 0.0“x” (one decimal)
 - **MOIC/Detailed Ratios:** 0.00“x” (two decimals for precision)
 - **All numeric cells:** Right-aligned
-

Clarify Requirements First

Before filling any formulas:

- **Examine the template structure** - Identify all sections, understand the timeline (which columns are which periods), note any existing formulas
 - **Ask the user if anything is unclear** - If the template structure, calculation methods, or requirements are ambiguous, ask before proceeding
 - **Confirm key assumptions** - Any key inputs, calculation preferences, or specific requirements
 - **ONLY AFTER understanding the template**, proceed to fill in formulas
-

TEMPLATE ANALYSIS PHASE - DO THIS FIRST

Before filling any formulas, examine the template thoroughly:

1. **Map the structure** - Identify where each section lives and how they relate to each other. Note which sections feed into others.
2. **Understand the timeline** - Which columns represent which periods? Is there a “Closing” or “Pro Forma” column? Where does the projection period start?
3. **Identify input vs formula cells** - Templates often use color coding, borders, or shading to indicate which cells need inputs vs formulas. Respect these conventions.
4. **Read existing labels carefully** - The row labels tell you exactly what calculation is expected. Don’t assume - read what the template is asking for.
5. **Check for existing formulas** - Some templates come partially filled. Don’t overwrite working formulas unless specifically asked.

6. **Note template-specific conventions** - Sign conventions, subtotal structures, how sections are organized, whether there are separate tabs for different components, etc.
-

FILLING FORMULAS - GENERAL APPROACH

For each cell that needs a formula, follow this hierarchy:

Step 1: Check the Template

- Does the cell already have a formula? If yes, verify it's correct and move on.
- Is there a comment or note indicating the expected calculation?
- Does the row/column label make the calculation obvious?
- Do neighboring cells show a pattern you should follow?

Step 2: Check the User's Instructions

- Did the user specify a particular calculation method?
- Are there stated assumptions that affect this formula?
- Any special requirements mentioned?

Step 3: Apply Standard Practice

- If neither template nor user specifies, use standard LBO modeling conventions
 - Document any assumptions you make
 - If genuinely uncertain, ask the user
-

COMMON PROBLEM AREAS

The following calculation patterns frequently cause issues across LBO models. Pay special attention when you encounter these:

Balancing Sections

- When two sections must equal (e.g., Sources = Uses), one item is typically the “plug” (balancing figure)
- Identify which item is the plug and calculate it as the difference

Tax Calculations

- Tax formulas should only reference the relevant income line and tax rate
- Should NOT reference unrelated sections (e.g., debt schedules)
- Consider whether losses create tax shields or are simply ignored

Interest and Circular References

- Interest calculations can create circularity if they reference balances affected by cash flows
- Use **Beginning Balance** (not average or ending) to break circular references
- Pattern: Interest → Cash Flow → Paydown → Ending Balance (if interest uses ending balance, this circles back)

Debt Paydown / Cash Sweeps

- When multiple debt tranches exist, there's usually a priority order
- Cash sweep should respect the priority waterfall
- Balances cannot go negative - use MAX or MIN functions appropriately

Returns Calculations (IRR/MOIC)

- Cash flows must have correct signs: Investment = negative, Proceeds = positive
- If using XIRR, need corresponding dates
- If using IRR, cash flows should be in consecutive periods
- $MOIC = \text{Total Proceeds} / \text{Total Investment}$

Sensitivity Tables

- Use **ODD dimensions** (5×5 or 7×7) — never 4×4 or 6×6. Odd dimensions guarantee a true center cell.
 - **Center cell = base case.** Build the row and column axis values symmetrically around the model's actual assumptions (e.g., if base entry multiple = 10.0x, axis = [8.0x, 9.0x, 10.0x, 11.0x, 12.0x]). The center cell's IRR/MOIC MUST then equal the model's actual IRR/MOIC output — this is the proof the table is wired correctly.
 - **Highlight the center cell** — medium-blue fill (#BDD7EE) + bold font so the base case is visually anchored.
 - Excel's DATA TABLE function may not work with openpyxl — instead write explicit formulas that reference row/column headers
 - Each cell should show a DIFFERENT value — if all same, formulas aren't varying correctly
 - Use mixed references (e.g., \$A5 for row input, B\$4 for column input)
-

VERIFICATION CHECKLIST - RUN AFTER COMPLETION

Run Formula Validation

Structured Template

- python /mnt/skills/public/xlsx/recalc.py model.xlsx

Must return success with zero errors.

Section Balancing

- ☐ Any sections that must balance (Sources/Uses, Assets/Liabilities) balance exactly
- ☐ Plug items are calculated correctly as the balancing figure
- ☐ Amounts that should match across sections are consistent

Income/Operating Projections

- ☐ Revenue/top-line builds correctly from drivers or growth rates
- ☐ All cost and expense items calculated appropriately
- ☐ Subtotals and totals sum correctly
- ☐ Margins and ratios are reasonable
- ☐ Links to assumptions are correct

Balance Sheet (if applicable)

- ☐ Assets = Liabilities + Equity (must balance)
- ☐ All items link to appropriate schedules or roll-forwards
- ☐ Beginning balances = prior period ending balances
- ☐ Check row included and shows zero

Cash Flow (if applicable)

- ☐ Starts with correct income figure
- ☐ Non-cash items added/subtracted appropriately
- ☐ Working capital changes have correct signs
- ☐ Ending Cash = Beginning Cash + Net Cash Flow
- ☐ Cash balances are consistent across statements

Supporting Schedules

- ☐ Roll-forward schedules balance (Beginning + Changes = Ending)
- ☐ Schedules link correctly to main statements
- ☐ Calculated items use appropriate drivers
- ☐ All periods are calculated consistently

Debt/Financing Schedules (if applicable)

- ☐ Beginning balances tie to sources or prior period
- ☐ Interest calculated on appropriate balance (typically beginning)
- ☐ Paydowns respect cash availability and priority
- ☐ Ending balances cannot be negative
- ☐ Totals sum tranches correctly

Returns/Output Analysis

- ☐ Exit/terminal values calculated correctly
- ☐ All relevant adjustments included
- ☐ Cash flow signs are correct (negative for investment, positive for proceeds)
- ☐ IRR/MOIC formulas reference complete ranges
- ☐ Results are reasonable for the scenario

Sensitivity Tables (if applicable)

- ☐ Grid dimensions are ODD (5×5 or 7×7) — there is a true center cell
- ☐ Row and column axis values are symmetric around the base case ($[base-2\Delta, base-\Delta, base, base+\Delta, base+2\Delta]$)
- ☐ Center cell output equals the model's actual IRR/MOIC — confirms the table is wired correctly
- ☐ Center cell is highlighted (medium-blue fill #BDD7EE, bold font)
- ☐ Row and column headers contain appropriate input values
- ☐ Each data cell contains a formula (not hardcoded)
- ☐ Each data cell shows a DIFFERENT value
- ☐ Values move in expected directions (higher exit multiple $\rightarrow$ higher IRR, etc.)

Formatting

- ☐ Hardcoded inputs are blue (0000FF)
- ☐ Calculated formulas are black (000000)
- ☐ Same-tab links are purple (800080)
- ☐ Cross-tab links are green (008000)
- ☐ All numbers are right-aligned
- ☐ Appropriate number formats applied throughout
- ☐ No cells show error values (#REF!, #DIV/0!, #VALUE!, #NAME?)

Logical Sanity Checks

- ☐ Numbers are reasonable order of magnitude
 - ☐ Trends make sense (growth, decline, stabilization as expected)
 - ☐ No obviously wrong values (negative where should be positive, impossible percentages, etc.)
 - ☐ Key outputs are within reasonable ranges for the type of analysis
-

COMMON ERRORS TO AVOID

Error	What Goes Wrong	How to Fix
Hardcoding calculated values	Model doesn't update when inputs change	Always use formulas that reference source cells
Wrong cell references after copying	Formulas point to wrong cells	Verify all links, use appropriate \$ anchoring
Circular reference errors	Model can't calculate	Use beginning balances for interest-type calcs, break the circle
Sections don't balance	Totals that should match don't	Ensure one item is the plug (calculated as difference)
Negative balances where impossible	Paying/ using more than available	Use MAX(0, ...) or MIN functions appropriately
IRR/ return errors	Wrong signs or incomplete ranges	Check cash flow signs and ensure formula covers all periods
Sensitivity table shows same value	Formula not varying with inputs	Check cell references - need mixed references (\$A5, B\$4)
Roll-forwards don't tie	Beginning $\neq$ prior ending	Verify links between periods
Inconsistent sign conventions	Additions become subtractions or vice versa	Follow template's convention consistently throughout

WORKING WITH THE USER — SECTION-BY-SECTION CHECKPOINTS

- **If the template structure is unclear**, ask before proceeding
- **If the user's requirements conflict with the template**, confirm their preference
- **After completing each major section**, STOP and verify with the user before continuing:
 - **After Sources & Uses** → show the balanced table, confirm the plug is correct, get sign-off before building the operating model
 - **After Operating Model / Projections** → show the projected P&L, confirm growth rates and margins look right, get sign-off before the debt schedule
 - **After Debt Schedule** → show beginning/ending balances and interest, confirm the waterfall logic, get sign-off before returns
 - **After Returns (IRR/MOIC)** → show the cash flow series and outputs, confirm signs and ranges, get sign-off before sensitivity tables

- **After Sensitivity Tables** → show that each cell varies, confirm the base case lands where expected
- **If errors are found during verification**, fix them before moving to the next section
- **Show your work** - explain key formulas or assumptions when helpful
- **Never present a completed model without having checked in at each section** — it's faster to catch a wrong cell reference at the source than to trace it backwards from a broken IRR

This skill produces investment banking-quality LBO models by filling templates with correct formulas, proper formatting, and validated calculations. The skill adapts to any template structure while ensuring financial accuracy and professional presentation standards.

3.36 Skill: ppt-template-creator

File Metadata

Field	Value
name	ppt-template-creator
description	Creates self-contained PPT template SKILLS (not presentations) from user-provided PowerPoint templates. Use ONLY when a user wants to create a reusable skill from their template. For creating actual presentations, use the pptx skill instead.

PPT Template Creator

This skill creates SKILLS, not presentations. Use this when a user wants to turn their PowerPoint template into a reusable skill that can generate presentations later. If the user just wants to create a presentation, use the *pptx* skill instead.

The generated skill includes:

- *assets/template.pptx* - the template file
- *SKILL.md* - complete instructions (no reference to this meta skill needed)

For general skill-building best practices, refer to the *skill-creator* skill. This skill focuses on PPT-specific patterns.

Workflow

1. **User provides template** (.pptx or .potx)
2. **Analyze template** - extract layouts, placeholders, dimensions
3. **Initialize skill** - use the *skill-creator* skill to set up the skill structure
4. **Add template** - copy .pptx to *assets/template.pptx*
5. **Write SKILL.md** - follow template below with PPT-specific details

6. **Create example** - generate sample presentation to validate
7. **Package** - use the *skill-creator* skill to package into a .skill file

Step 2: Analyze Template

CRITICAL: Extract precise placeholder positions - this determines content area boundaries.

Structured Template

- from pptx import Presentation
- prs = Presentation(template_path)
- print(f'Dimensions: {prs.slide_width/914400:.2f}\'' x {prs.slide_height/914400:.2f}\''')
- print(f'Layouts: {len(prs.slide_layouts)}')
- for idx, layout in enumerate(prs.slide_layouts):
- print(f'\n[{idx}] {layout.name}:')
- for ph in layout.placeholders:
- try:
- ph_idx = ph.placeholder_format.idx
- ph_type = ph.placeholder_format.type
- # IMPORTANT: Extract exact positions in inches
- left = ph.left / 914400
- top = ph.top / 914400
- width = ph.width / 914400
- height = ph.height / 914400
- print(f'idx={ph_idx}, type={ph_type}')
- print(f'x={left:.2f}\'', y={top:.2f}\'', w={width:.2f}\'', h={height:.2f}\''')
- except:
- pass

Key measurements to document:

- **Title position:** Where does the title placeholder sit?
- **Subtitle/description:** Where is the subtitle line?
- **Footer placeholders:** Where do footers/sources appear?
- **Content area:** The space BETWEEN subtitle and footer is your content area

Finding the True Content Start Position

CRITICAL: The content area does NOT always start immediately after the subtitle placeholder. Many templates have a visual border, line, or reserved space between the subtitle and content area.

Best approach: Look at Layout 2 or similar “content” layouts that have an OBJECT placeholder - this placeholder’s y position indicates where content should actually start.

Structured Template

- # Find the OBJECT placeholder to determine true content start
- for idx, layout in enumerate(prs.slide_layouts):
- for ph in layout.placeholders:
- try:
- if ph.placeholder_format.type == 7: # OBJECT type
- top = ph.top / 914400
- print(f'Layout [{idx}] {layout.name}: OBJECT starts at y={top:.2f}\n')
- # This y value is where your content should start!
- except:
- pass

Example: A template might have:

- Subtitle ending at y=1.38“
- But OBJECT placeholder starting at y=1.90“
- The gap (0.52“) is reserved for a border/line - **do not place content there**

Use the OBJECT placeholder’s y position as your content start, not the subtitle’s end position.

Step 5: Write SKILL.md

The generated skill should have this structure:

Structured Template

- [company]-ppt-template/
- SKILL.md
- assets/
- template.pptx

Generated SKILL.md Template

The generated SKILL.md must be **self-contained** with all instructions embedded. Use this template, filling in the bracketed values from your analysis:

Structured Template

- name: [company]-ppt-template
- description: [Company] PowerPoint template for creating presentations. Use when creating [Company]-branded pitch decks, board materials, or client presentations.
- # [Company] PPT Template
- Template: assets/template.pptx ([WIDTH]" x [HEIGHT]", [N] layouts)
- ## Creating Presentations

from pptx import Presentation

prs = Presentation("path/to/skill/assets/template.pptx")

DELETE all existing slides first

```
while len(prs.slides) > 0: rId = prs.slides._sldIdLst[0].rId prs.part.drop_rel(rId) del prs.slides._sldIdLst[0]
```

Add slides from layouts

```
slide = prs.slides.add_slide(prs.slide_layouts[LAYOUT_IDX])
```

Structured Template

- ## Key Layouts
- | Index | Name | Use For |
- | ————|—————|—————|
- | [0] | [Layout Name] | [Cover/title slide] |
- | [N] | [Layout Name] | [Content with bullets] |
- | [N] | [Layout Name] | [Two-column layout] |
- ## Placeholder Mapping
- **CRITICAL: Include exact positions (x, y coordinates) for each placeholder.**
- ### Layout [N]: [Name]
- | idx | Type | Position | Use |
- | ————|—————|—————|—————|
- | [idx] | TITLE (1) | y=[Y]" | Slide title |
- | [idx] | BODY (2) | y=[Y]" | Subtitle/description |
- | [idx] | BODY (2) | y=[Y]" | Footer |
- | [idx] | BODY (2) | y=[Y]" | Source/notes |
- ### Content Area Boundaries
- **Document the safe content area for custom shapes/tables/charts:**

Content Area (for Layout [N]):

- Left margin: [X]“ (content starts here)
- Top: [Y]“ (below subtitle placeholder)
- Width: [W]“
- Height: [H]“ (ends before footer)

For 4-quadrant layouts:

- Left column: x=[X]“, width=[W]”
- Right column: x=[X]“, width=[W]”
- Top row: y=[Y]“, height=[H]”
- Bottom row: y=[Y]“, height=[H]”

Structured Template

- **Why this matters:** Custom content (textboxes, tables, charts) must stay within these boundaries to avoid overlapping with template placeholders like titles, footers, and source lines.
- ## Filling Content
- **Do NOT add manual bullet characters** - slide master handles formatting.

Fill title

for shape in slide.shapes: if hasattr(shape, 'placeholder_format'): if shape.placeholder_format.type == 1: # TITLE
shape.text = “Slide Title”

Fill content with hierarchy (level 0 = header, level 1 = bullet)

for shape in slide.shapes: if hasattr(shape, 'placeholder_format'): idx = shape.placeholder_format.idx if idx == [CONTENT_IDX]: tf = shape.text_frame for para in tf.paragraphs: para.clear()

content = [(“Section Header”, 0), (“First bullet point”, 1), (“Second bullet point”, 1),]

tf.paragraphs[0].text = content[0][0] tf.paragraphs[0].level = content[0][1] for text, level in content[1:]: p = tf.add_paragraph() p.text = text p.level = level

Structured Template

- ## Example: Cover Slide

slide = prs.slides.add_slide(prs.slide_layouts[[COVER_IDX]]) for shape in slide.shapes: if hasattr(shape, 'placeholder_format'): idx = shape.placeholder_format.idx if idx == [TITLE_IDX]: shape.text = “Company Name” elif idx == [SUBTITLE_IDX]: shape.text = “Presentation Title | Date”

Structured Template

- ## Example: Content Slide

slide = prs.slides.add_slide(prs.slide_layouts[[CONTENT_IDX]]) for shape in slide.shapes: if hasattr(shape, 'placeholder_format'): ph_type = shape.placeholder_format.type idx = shape.placeholder_format.idx if ph_type == 1: shape.text = “Executive Summary” elif idx == [BODY_IDX]: tf = shape.text_frame for para in tf.paragraphs:

```
para.clear() content = [ ("Key Findings", 0), ("Revenue grew 40% YoY to $50M", 1), ("Expanded to 3 new
markets", 1), ("Recommendation", 0), ("Proceed with strategic initiative", 1), ] tf.paragraphs[0].text = content[0][0]
tf.paragraphs[0].level = content[0][1] for text, level in content[1:]: p = tf.add_paragraph() p.text = text p.level =
level
```

Step 6: Create Example Output

Generate a sample presentation to validate the skill works. Save it alongside the skill for reference.

PPT-Specific Rules for Generated Skills

1. **Template in assets/** - always bundle the .pptx file
2. **Self-contained SKILL.md** - all instructions embedded, no external references
3. **No manual bullets** - use *paragraph.level* for hierarchy
4. **Delete slides first** - always clear existing slides before adding new ones
5. **Document placeholders by idx** - placeholder idx values are template-specific

3.37 Skill: pptx-author

File Metadata

Field	Value
name	pptx-author
description	Produce a .pptx file on disk (headless) instead of driving a live PowerPoint document — for managed-agent sessions with no open Office app.

pptx-author

Use this skill when running **headless** (managed-agent / CMA mode) and you need to deliver a PowerPoint deck as a **file artifact** rather than editing a live document via *mcp__office__powerpoint__**.

Output contract

- Write to *./out/.pptx*. Create *./out/* if it does not exist.
- Return the relative path in your final message so the orchestration layer can collect it.

How to build the deck

Write a short Python script and run it with Bash. Use *python-pptx*:

Structured Template

- from pptx import Presentation
- from pptx.util import Inches, Pt
- prs = Presentation("./templates/firm-template.pptx") # if a template is provided
- # or: prs = Presentation()
- slide = prs.slides.add_slide(prs.slide_layouts[5]) # title-only
- slide.shapes.title.text = "Valuation Summary"
- # ... add tables / charts / text boxes ...
- prs.save("./out/pitch-.pptx")

Conventions (mirror the live-Office *pitch-deck* skill)

- **One idea per slide.** Title states the takeaway; body supports it.
- **Every number traces to the model.** If a figure comes from *./out/model.xlsx*, footnote the sheet and cell.
- **Use the firm template** when one is mounted at *./templates/*; otherwise default layouts.
- **Charts:** prefer embedding a PNG rendered from the model over native pptx charts when fidelity matters.
- **No external sends.** This skill writes a file; it never emails or uploads.

When NOT to use

If *mcp__office__powerpoint_** tools are available (Cowork plugin mode), use those instead — they drive the user's live document with review checkpoints. This skill is the file-producing fallback for headless runs.

3.38 Skill Resource: [skills/skill-creator/references/output-patterns.md](#)

Output Patterns

Use these patterns when skills need to produce consistent, high-quality output.

Template Pattern

Provide templates for output format. Match the level of strictness to your needs.

For strict requirements (like API responses or data formats):

Structured Template

- `##` Report structure
- ALWAYS use this exact template structure:
- `#` [Analysis Title]
- `##` Executive summary
- [One-paragraph overview of key findings]

- ## Key findings
- - Finding 1 with supporting data
- - Finding 2 with supporting data
- - Finding 3 with supporting data
- ## Recommendations
- 1. Specific actionable recommendation
- 2. Specific actionable recommendation

For flexible guidance (when adaptation is useful):

Structured Template

- ## Report structure
- Here is a sensible default format, but use your best judgment:
- # [Analysis Title]
- ## Executive summary
- [Overview]
- ## Key findings
- [Adapt sections based on what you discover]
- ## Recommendations
- [Tailor to the specific context]
- Adjust sections as needed for the specific analysis type.

Examples Pattern

For skills where output quality depends on seeing examples, provide input/output pairs:

Structured Template

- ## Commit message format
- Generate commit messages following these examples:
- **Example 1:**
- Input: Added user authentication with JWT tokens
- Output:

feat(auth): implement JWT-based authentication

Add login endpoint and token validation middleware

Structured Template

- **Example 2:**
- Input: Fixed bug where dates displayed incorrectly in reports

- Output:

fix(reports): correct date formatting in timezone conversion

Use UTC timestamps consistently across report generation

Structured Template

- Follow this style: type(scope): brief description, then detailed explanation.

Examples help Claude understand the desired style and level of detail more clearly than descriptions alone.

3.39 Skill Resource: skills/skill-creator/references/workflows.md

Workflow Patterns

Sequential Workflows

For complex tasks, break operations into clear, sequential steps. It is often helpful to give Claude an overview of the process towards the beginning of SKILL.md:

Structured Template

- Filling a PDF form involves these steps:
- 1. Analyze the form (run analyze_form.py)
- 2. Create field mapping (edit fields.json)
- 3. Validate mapping (run validate_fields.py)
- 4. Fill the form (run fill_form.py)
- 5. Verify output (run verify_output.py)

Conditional Workflows

For tasks with branching logic, guide Claude through decision points:

Structured Template

- 1. Determine the modification type:
 - **Creating new content?** → Follow “Creation workflow” below
 - **Editing existing content?** → Follow “Editing workflow” below
 - 2. Creation workflow: [steps]
 - 3. Editing workflow: [steps]
-

3.40 Script File: skills/skill-creator/scripts/init_skill.py

Script Summary

Signal	Details
Imports	sys, pathlib import Path
Functions	main, title_case_skill_name, init_skill, main
Entry point	Defines an executable main entry point

Normalized Script Content

- `#!/usr/bin/env python3`
- `"""`
- Skill Initializer - Creates a new skill from template
- Usage:
- `init_skill.py -path`
- Examples:
- `init_skill.py my-new-skill -path skills/public`
- `init_skill.py my-api-helper -path skills/private`
- `init_skill.py custom-skill -path /custom/location`
- `"""`
- `import sys`
- `from pathlib import Path`
- `SKILL_TEMPLATE = """`
- `name: {skill_name}`
- `description: [TODO: Complete and informative explanation of what the skill does and when to use it. Include WHEN to use this skill - specific scenarios, file types, or tasks that trigger it.]`
- `# {skill_title}`
- `## Overview`
- `[TODO: 1-2 sentences explaining what this skill enables]`
- `## Structuring This Skill`
- `[TODO: Choose the structure that best fits this skill's purpose. Common patterns:`
- **1. Workflow-Based** (best for sequential processes)
- - Works well when there are clear step-by-step procedures
- - Example: DOCX skill with “Workflow Decision Tree” → “Reading” → “Creating” → “Editing”
- - Structure: `## Overview` → `## Workflow Decision Tree` → `## Step 1` → `## Step 2...`
- **2. Task-Based** (best for tool collections)
- - Works well when the skill offers different operations/capabilities

- - Example: PDF skill with “Quick Start” → “Merge PDFs” → “Split PDFs” → “Extract Text”
- - Structure: ## Overview → ## Quick Start → ## Task Category 1 → ## Task Category 2...
- **3. Reference/Guidelines** (best for standards or specifications)
 - - Works well for brand guidelines, coding standards, or requirements
 - - Example: Brand styling with “Brand Guidelines” → “Colors” → “Typography” → “Features”
 - - Structure: ## Overview → ## Guidelines → ## Specifications → ## Usage...
- **4. Capabilities-Based** (best for integrated systems)
 - - Works well when the skill provides multiple interrelated features
 - - Example: Product Management with “Core Capabilities” → numbered capability list
 - - Structure: ## Overview → ## Core Capabilities → ### 1. Feature → ### 2. Feature...
- Patterns can be mixed and matched as needed. Most skills combine patterns (e.g., start with task-based, add workflow for complex operations).
- Delete this entire “Structuring This Skill” section when done - it’s just guidance.]
- ## [TODO: Replace with the first main section based on chosen structure]
- [TODO: Add content here. See examples in existing skills:
 - - Code samples for technical skills
 - - Decision trees for complex workflows
 - - Concrete examples with realistic user requests
 - - References to scripts/templates/references as needed]
- ## Resources
 - This skill includes example resource directories that demonstrate how to organize different types of bundled resources:
 - ### scripts/
 - Executable code (Python/Bash/etc.) that can be run directly to perform specific operations.
- **Examples from other skills:**
 - - PDF skill: fill_fillable_fields.py, extract_form_field_info.py - utilities for PDF manipulation
 - - DOCX skill: document.py, utilities.py - Python modules for document processing
- **Appropriate for:** Python scripts, shell scripts, or any executable code that performs automation, data processing, or specific operations.
- **Note:** Scripts may be executed without loading into context, but can still be read by Claude for patching or environment adjustments.
- ### references/
 - Documentation and reference material intended to be loaded into context to inform Claude’s process and thinking.

- **Examples from other skills:**
 - - Product management: communication.md, context_building.md - detailed workflow guides
 - - BigQuery: API reference documentation and query examples
 - - Finance: Schema documentation, company policies
 - **Appropriate for:** In-depth documentation, API references, database schemas, comprehensive guides, or any detailed information that Claude should reference while working.
- `### assets/`
- Files not intended to be loaded into context, but rather used within the output Claude produces.
- **Examples from other skills:**
 - - Brand styling: PowerPoint template files (.pptx), logo files
 - - Frontend builder: HTML/React boilerplate project directories
 - - Typography: Font files (.ttf, .woff2)
 - **Appropriate for:** Templates, boilerplate code, document templates, images, icons, fonts, or any files meant to be copied or used in the final output.
 - **Any unneeded directories can be deleted.** Not every skill requires all three types of resources.
- `"""`
- `EXAMPLE_SCRIPT = """#!/usr/bin/env python3`
- `"""`
- Example helper script for {skill_name}
- This is a placeholder script that can be executed directly.
- Replace with actual implementation or delete if not needed.
- Example real scripts from other skills:
 - - pdf/scripts/fill_fillable_fields.py - Fills PDF form fields
 - - pdf/scripts/convert_pdf_to_images.py - Converts PDF pages to images
- `"""`
- `def main():`
- `print("This is an example script for {skill_name}")`
- `# TODO: Add actual script logic here`
- `# This could be data processing, file conversion, API calls, etc.`
- `if __name__ == "__main__":`
- `main()`
- `"""`
- `EXAMPLE_REFERENCE = """# Reference Documentation for {skill_title}`

- This is a placeholder for detailed reference documentation.
- Replace with actual reference content or delete if not needed.
- Example real reference docs from other skills:
 - - product-management/references/communication.md - Comprehensive guide for status updates
 - - product-management/references/context_building.md - Deep-dive on gathering context
 - - bigquery/references/ - API references and query examples
- ## When Reference Docs Are Useful
- Reference docs are ideal for:
 - - Comprehensive API documentation
 - - Detailed workflow guides
 - - Complex multi-step processes
 - - Information too lengthy for main SKILL.md
 - - Content that's only needed for specific use cases
- ## Structure Suggestions
- ### API Reference Example
 - - Overview
 - - Authentication
 - - Endpoints with examples
 - - Error codes
 - - Rate limits
- ### Workflow Guide Example
 - - Prerequisites
 - - Step-by-step instructions
 - - Common patterns
 - - Troubleshooting
 - - Best practices
- `“”““`
- `EXAMPLE_ASSET = “”““# Example Asset File`
- This placeholder represents where asset files would be stored.
- Replace with actual asset files (templates, images, fonts, etc.) or delete if not needed.
- Asset files are NOT intended to be loaded into context, but rather used within
- the output Claude produces.

- Example asset files from other skills:
- - Brand guidelines: logo.png, slides_template.pptx
- - Frontend builder: hello-world/ directory with HTML/React boilerplate
- - Typography: custom-font.ttf, font-family.woff2
- - Data: sample_data.csv, test_dataset.json
- ## Common Asset Types
- - Templates: .pptx, .docx, boilerplate directories
- - Images: .png, .jpg, .svg, .gif
- - Fonts: .ttf, .otf, .woff, .woff2
- - Boilerplate code: Project directories, starter files
- - Icons: .ico, .svg
- - Data files: .csv, .json, .xml, .yaml
- Note: This is a text placeholder. Actual assets can be any file type.

- `"""`

- `def title_case_skill_name(skill_name):`
- `"""Convert hyphenated skill name to Title Case for display."""`
- `return ' '.join(word.capitalize() for word in skill_name.split('-'))`
- `def init_skill(skill_name, path):`

- `"""`

- Initialize a new skill directory with template SKILL.md.
- Args:
- skill_name: Name of the skill
- path: Path where the skill directory should be created
- Returns:
- Path to created skill directory, or None if error

- `"""`

- # Determine skill directory path
- skill_dir = Path(path).resolve() / skill_name
- # Check if directory already exists
- if skill_dir.exists():
- print(f"❌ Error: Skill directory already exists: {skill_dir}")
- return None

- # Create skill directory
- try:
- skill_dir.mkdir(parents=True, exist_ok=False)
- print(f"✅ Created skill directory: {skill_dir}")
- except Exception as e:
- print(f"❌ Error creating directory: {e}")
- return None
- # Create SKILL.md from template
- skill_title = title_case_skill_name(skill_name)
- skill_content = SKILL_TEMPLATE.format(
- skill_name=skill_name,
- skill_title=skill_title
-)
- skill_md_path = skill_dir / 'SKILL.md'
- try:
- skill_md_path.write_text(skill_content)
- print(f"✅ Created SKILL.md")
- except Exception as e:
- print(f"❌ Error creating SKILL.md: {e}")
- return None
- # Create resource directories with example files
- try:
- # Create scripts/ directory with example script
- scripts_dir = skill_dir / 'scripts'
- scripts_dir.mkdir(exist_ok=True)
- example_script = scripts_dir / 'example.py'
- example_script.write_text(EXAMPLE_SCRIPT.format(skill_name=skill_name))
- example_script.chmod(0o755)
- print(f"✅ Created scripts/example.py")
- # Create references/ directory with example reference doc
- references_dir = skill_dir / 'references'
- references_dir.mkdir(exist_ok=True)

- `example_reference = references_dir / 'api_reference.md'`
- `example_reference.write_text(EXAMPLE_REFERENCE.format(skill_title=skill_title))`
- `print("✅ Created references/api_reference.md")`
- `# Create assets/ directory with example asset placeholder`
- `assets_dir = skill_dir / 'assets'`
- `assets_dir.mkdir(exist_ok=True)`
- `example_asset = assets_dir / 'example_asset.txt'`
- `example_asset.write_text(EXAMPLE_ASSET)`
- `print("✅ Created assets/example_asset.txt")`
- `except Exception as e:`
- `print(f"❌ Error creating resource directories: {e}")`
- `return None`
- `# Print next steps`
- `print(f"\n✅ Skill '{skill_name}' initialized successfully at {skill_dir}")`
- `print("\nNext steps:")`
- `print("1. Edit SKILL.md to complete the TODO items and update the description")`
- `print("2. Customize or delete the example files in scripts/, references/, and assets/")`
- `print("3. Run the validator when ready to check the skill structure")`
- `return skill_dir`
- `def main():`
- `if len(sys.argv) < 4 or sys.argv[2] != '-path':`
- `print("Usage: init_skill.py -path ")`
- `print("\nSkill name requirements:")`
- `print(" - Hyphen-case identifier (e.g., 'data-analyzer')")`
- `print(" - Lowercase letters, digits, and hyphens only")`
- `print(" - Max 40 characters")`
- `print(" - Must match directory name exactly")`
- `print("\nExamples:")`
- `print(" init_skill.py my-new-skill -path skills/public")`
- `print(" init_skill.py my-api-helper -path skills/private")`
- `print(" init_skill.py custom-skill -path /custom/location")`
- `sys.exit(1)`

- skill_name = sys.argv[1]
 - path = sys.argv[3]
 - print(f"🚀 Initializing skill: {skill_name}")
 - print(f" Location: {path}")
 - print()
 - result = init_skill(skill_name, path)
 - if result:
 - sys.exit(0)
 - else:
 - sys.exit(1)
 - if __name__ == "__main__":
 - main()
-

3.41 Script File: skills/skill-creator/scripts/package_skill.py

Script Summary

Signal	Details
Imports	sys, zipfile, pathlib import Path, quick_validate import validate_skill
Functions	package_skill, main
Entry point	Defines an executable main entry point

Normalized Script Content

- `#!/usr/bin/env python3`
- `"""`
- Skill Packager - Creates a distributable .skill file of a skill folder
- Usage:
- `python utils/package_skill.py [output-directory]`
- Example:
- `python utils/package_skill.py skills/public/my-skill`
- `python utils/package_skill.py skills/public/my-skill ./dist`
- `"""`
- `import sys`

- import zipfile
- from pathlib import Path
- from quick_validate import validate_skill
- def package_skill(skill_path, output_dir=None):
- `"""`
- Package a skill folder into a .skill file.
- Args:
- skill_path: Path to the skill folder
- output_dir: Optional output directory for the .skill file (defaults to current directory)
- Returns:
- Path to the created .skill file, or None if error
- `"""`
- skill_path = Path(skill_path).resolve()
- # Validate skill folder exists
- if not skill_path.exists():
- print(f"❌ Error: Skill folder not found: {skill_path}")
- return None
- if not skill_path.is_dir():
- print(f"❌ Error: Path is not a directory: {skill_path}")
- return None
- # Validate SKILL.md exists
- skill_md = skill_path / "SKILL.md"
- if not skill_md.exists():
- print(f"❌ Error: SKILL.md not found in {skill_path}")
- return None
- # Run validation before packaging
- print("🔍 Validating skill...")
- valid, message = validate_skill(skill_path)
- if not valid:
- print(f"❌ Validation failed: {message}")
- print(" Please fix the validation errors before packaging.")
- return None

- `print(f"✅ {message}\n")`
- `# Determine output location`
- `skill_name = skill_path.name`
- `if output_dir:`
- `output_path = Path(output_dir).resolve()`
- `output_path.mkdir(parents=True, exist_ok=True)`
- `else:`
- `output_path = Path.cwd()`
- `skill_filename = output_path / f'{skill_name}.skill'`
- `# Create the .skill file (zip format)`
- `try:`
- `with zipfile.ZipFile(skill_filename, 'w', zipfile.ZIP_DEFLATED) as zipf:`
- `# Walk through the skill directory`
- `for file_path in skill_path.rglob('*'):`
- `if file_path.is_file():`
- `# Calculate the relative path within the zip`
- `arcname = file_path.relative_to(skill_path.parent)`
- `zipf.write(file_path, arcname)`
- `print(f' Added: {arcname}')`
- `print(f"\n✅ Successfully packaged skill to: {skill_filename}')`
- `return skill_filename`
- `except Exception as e:`
- `print(f"❌ Error creating .skill file: {e}')`
- `return None`
- `def main():`
- `if len(sys.argv) < 2:`
- `print("Usage: python utils/package_skill.py [output-directory]")`
- `print("\nExample:")`
- `print(" python utils/package_skill.py skills/public/my-skill")`
- `print(" python utils/package_skill.py skills/public/my-skill ./dist")`
- `sys.exit(1)`
- `skill_path = sys.argv[1]`

- `output_dir = sys.argv[2] if len(sys.argv) > 2 else None`
 - `print(f'📦 Packaging skill: {skill_path}')`
 - `if output_dir:`
 - `print(f' Output directory: {output_dir}')`
 - `print()`
 - `result = package_skill(skill_path, output_dir)`
 - `if result:`
 - `sys.exit(0)`
 - `else:`
 - `sys.exit(1)`
 - `if __name__ == "__main__":`
 - `main()`
-

3.42 Script File: `skills/skill-creator/scripts/quick_validate.py`

Script Summary

Signal	Details
Imports	<code>sys, os, re, yaml, pathlib import Path</code>
Functions	<code>validate_skill</code>
Entry point	Defines an executable main entry point

Normalized Script Content

- `#!/usr/bin/env python3`
- `"""`
- Quick validation script for skills - minimal version
- `"""`
- `import sys`
- `import os`
- `import re`
- `import yaml`
- `from pathlib import Path`
- `def validate_skill(skill_path):`
- `"""Basic validation of a skill"""`

- `skill_path = Path(skill_path)`
- `# Check SKILL.md exists`
- `skill_md = skill_path / 'SKILL.md'`
- `if not skill_md.exists():`
- `return False, "SKILL.md not found"`
- `# Read and validate frontmatter`
- `content = skill_md.read_text()`
- `if not content.startswith('—'):`
- `return False, "No YAML frontmatter found"`
- `# Extract frontmatter`
- `match = re.match(r'^—\n(?:.*?)\n—', content, re.DOTALL)`
- `if not match:`
- `return False, "Invalid frontmatter format"`
- `frontmatter_text = match.group(1)`
- `# Parse YAML frontmatter`
- `try:`
- `frontmatter = yaml.safe_load(frontmatter_text)`
- `if not isinstance(frontmatter, dict):`
- `return False, "Frontmatter must be a YAML dictionary"`
- `except yaml.YAMLError as e:`
- `return False, f"Invalid YAML in frontmatter: {e}"`
- `# Define allowed properties`
- `ALLOWED_PROPERTIES = {'name', 'description', 'license', 'allowed-tools', 'metadata'}`
- `# Check for unexpected properties (excluding nested keys under metadata)`
- `unexpected_keys = set(frontmatter.keys()) - ALLOWED_PROPERTIES`
- `if unexpected_keys:`
- `return False, (`
- `f"Unexpected key(s) in SKILL.md frontmatter: {' ', ' '.join(sorted(unexpected_keys))}." "`
- `f"Allowed properties are: {' ', ' '.join(sorted(ALLOWED_PROPERTIES))}." "`
- `)`
- `# Check required fields`
- `if 'name' not in frontmatter:`

- return False, “Missing ‘name’ in frontmatter”
- if ‘description’ not in frontmatter:
- return False, “Missing ‘description’ in frontmatter”
- # Extract name for validation
- name = frontmatter.get(‘name’, ”)
- if not isinstance(name, str):
- return False, f“Name must be a string, got {type(name).__name__}”
- name = name.strip()
- if name:
- # Check naming convention (hyphen-case: lowercase with hyphens)
- if not re.match(r‘^[a-z0-9-]+\$’, name):
- return False, f“Name ‘{name}’ should be hyphen-case (lowercase letters, digits, and hyphens only)”
- if name.startswith(‘-’) or name.endswith(‘-’) or ‘-’ in name:
- return False, f“Name ‘{name}’ cannot start/end with hyphen or contain consecutive hyphens”
- # Check name length (max 64 characters per spec)
- if len(name) > 64:
- return False, f“Name is too long ({len(name)} characters). Maximum is 64 characters.”
- # Extract and validate description
- description = frontmatter.get(‘description’, ”)
- if not isinstance(description, str):
- return False, f“Description must be a string, got {type(description).__name__}”
- description = description.strip()
- if description:
- # Check for angle brackets
- if ‘<’ in description or ‘>’ in description:
- return False, “Description cannot contain angle brackets (< or >)”
- # Check description length (max 1024 characters per spec)
- if len(description) > 1024:
- return False, f“Description is too long ({len(description)} characters). Maximum is 1024 characters.”
- return True, “Skill is valid!”
- if __name__ == “__main__”:
- if len(sys.argv) != 2:

- `print("Usage: python quick_validate.py ")`
 - `sys.exit(1)`
 - `valid, message = validate_skill(sys.argv[1])`
 - `print(message)`
 - `sys.exit(0 if valid else 1)`
-

3.43 Skill: skill-creator

File Metadata

Field	Value
name	skill-creator
description	Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Claude’s capabilities with specialized knowledge, workflows, or tool integrations.
license	Complete terms in LICENSE.txt

Skill Creator

This skill provides guidance for creating effective skills.

About Skills

Skills are modular, self-contained packages that extend Claude’s capabilities by providing specialized knowledge, workflows, and tools. Think of them as “onboarding guides” for specific domains or tasks—they transform Claude from a general-purpose agent into a specialized agent equipped with procedural knowledge that no model can fully possess.

What Skills Provide

1. Specialized workflows - Multi-step procedures for specific domains
2. Tool integrations - Instructions for working with specific file formats or APIs
3. Domain expertise - Company-specific knowledge, schemas, business logic
4. Bundled resources - Scripts, references, and assets for complex and repetitive tasks

Core Principles

Concise is Key

The context window is a public good. Skills share the context window with everything else Claude needs: system prompt, conversation history, other Skills' metadata, and the actual user request.

Default assumption: Claude is already very smart. Only add context Claude doesn't already have. Challenge each piece of information: "Does Claude really need this explanation?" and "Does this paragraph justify its token cost?"

Prefer concise examples over verbose explanations.

Set Appropriate Degrees of Freedom

Match the level of specificity to the task's fragility and variability:

High freedom (text-based instructions): Use when multiple approaches are valid, decisions depend on context, or heuristics guide the approach.

Medium freedom (pseudocode or scripts with parameters): Use when a preferred pattern exists, some variation is acceptable, or configuration affects behavior.

Low freedom (specific scripts, few parameters): Use when operations are fragile and error-prone, consistency is critical, or a specific sequence must be followed.

Think of Claude as exploring a path: a narrow bridge with cliffs needs specific guardrails (low freedom), while an open field allows many routes (high freedom).

Anatomy of a Skill

Every skill consists of a required SKILL.md file and optional bundled resources:

Structured Template

- skill-name/
- SKILL.md (required)
- YAML frontmatter metadata (required)
- name: (required)
- description: (required)
- Markdown instructions (required)
- Bundled Resources (optional)
- scripts/ - Executable code (Python/Bash/etc.)
- references/ - Documentation intended to be loaded into context as needed
- assets/ - Files used in output (templates, icons, fonts, etc.)

SKILL.md (required)

Every SKILL.md consists of:

- **Frontmatter** (YAML): Contains *name* and *description* fields. These are the only fields that Claude reads to determine when the skill gets used, thus it is very important to be clear and comprehensive in describing what the skill is, and when it should be used.
- **Body** (Markdown): Instructions and guidance for using the skill. Only loaded AFTER the skill triggers (if at all).

Bundled Resources (optional)

Scripts (*scripts/*)

Executable code (Python/Bash/etc.) for tasks that require deterministic reliability or are repeatedly rewritten.

- **When to include:** When the same code is being rewritten repeatedly or deterministic reliability is needed
- **Example:** *scripts/rotate_pdf.py* for PDF rotation tasks
- **Benefits:** Token efficient, deterministic, may be executed without loading into context
- **Note:** Scripts may still need to be read by Claude for patching or environment-specific adjustments

References (*references/*)

Documentation and reference material intended to be loaded as needed into context to inform Claude's process and thinking.

- **When to include:** For documentation that Claude should reference while working
- **Examples:** *references/finance.md* for financial schemas, *references/mnda.md* for company NDA template, *references/policies.md* for company policies, *references/api_docs.md* for API specifications
- **Use cases:** Database schemas, API documentation, domain knowledge, company policies, detailed workflow guides
- **Benefits:** Keeps SKILL.md lean, loaded only when Claude determines it's needed
- **Best practice:** If files are large (>10k words), include grep search patterns in SKILL.md
- **Avoid duplication:** Information should live in either SKILL.md or references files, not both. Prefer references files for detailed information unless it's truly core to the skill—this keeps SKILL.md lean while making information discoverable without hogging the context window. Keep only essential procedural instructions and workflow guidance in SKILL.md; move detailed reference material, schemas, and examples to references files.

Assets (*assets/*)

Files not intended to be loaded into context, but rather used within the output Claude produces.

- **When to include:** When the skill needs files that will be used in the final output
- **Examples:** *assets/logo.png* for brand assets, *assets/slides.pptx* for PowerPoint templates, *assets/frontend-template/* for HTML/React boilerplate, *assets/font.ttf* for typography
- **Use cases:** Templates, images, icons, boilerplate code, fonts, sample documents that get copied or modified

- **Benefits:** Separates output resources from documentation, enables Claude to use files without loading them into context

What to Not Include in a Skill

A skill should only contain essential files that directly support its functionality. Do NOT create extraneous documentation or auxiliary files, including:

- README.md
- INSTALLATION_GUIDE.md
- QUICK_REFERENCE.md
- CHANGELOG.md
- etc.

The skill should only contain the information needed for an AI agent to do the job at hand. It should not contain auxiliary context about the process that went into creating it, setup and testing procedures, user-facing documentation, etc. Creating additional documentation files just adds clutter and confusion.

Progressive Disclosure Design Principle

Skills use a three-level loading system to manage context efficiently:

1. **Metadata (name + description)** - Always in context (~100 words)
2. **SKILL.md body** - When skill triggers (<5k words)
3. **Bundled resources** - As needed by Claude (Unlimited because scripts can be executed without reading into context window)

Progressive Disclosure Patterns

Keep SKILL.md body to the essentials and under 500 lines to minimize context bloat. Split content into separate files when approaching this limit. When splitting out content into other files, it is very important to reference them from SKILL.md and describe clearly when to read them, to ensure the reader of the skill knows they exist and when to use them.

Key principle: When a skill supports multiple variations, frameworks, or options, keep only the core workflow and selection guidance in SKILL.md. Move variant-specific details (patterns, examples, configuration) into separate reference files.

Pattern 1: High-level guide with references

Structured Template

- # PDF Processing
- ## Quick start
- Extract text with pdfplumber:
- [code example]

- **##** Advanced features
- - **Form filling**: See [FORMS.md](#) for complete guide
- - **API reference**: See [REFERENCE.md](#) for all methods
- - **Examples**: See [EXAMPLES.md](#) for common patterns

Claude loads FORMS.md, REFERENCE.md, or EXAMPLES.md only when needed.

Pattern 2: Domain-specific organization

For Skills with multiple domains, organize content by domain to avoid loading irrelevant context:

Structured Template

- bigquery-skill/
- SKILL.md (overview and navigation)
- reference/
- finance.md (revenue, billing metrics)
- sales.md (opportunities, pipeline)
- product.md (API usage, features)
- marketing.md (campaigns, attribution)

When a user asks about sales metrics, Claude only reads sales.md.

Similarly, for skills supporting multiple frameworks or variants, organize by variant:

Structured Template

- cloud-deploy/
- SKILL.md (workflow + provider selection)
- references/
- aws.md (AWS deployment patterns)
- gcp.md (GCP deployment patterns)
- azure.md (Azure deployment patterns)

When the user chooses AWS, Claude only reads aws.md.

Pattern 3: Conditional details

Show basic content, link to advanced content:

Structured Template

- **#** DOCX Processing
- **##** Creating documents
- Use docx-js for new documents. See [DOCX-JS.md](#).
- **##** Editing documents
- For simple edits, modify the XML directly.

- **For tracked changes:** See [REDLINING.md](#)
- **For OOXML details:** See [OOXML.md](#)

Claude reads REDLINING.md or OOXML.md only when the user needs those features.

Important guidelines:

- **Avoid deeply nested references** - Keep references one level deep from SKILL.md. All reference files should link directly from SKILL.md.
- **Structure longer reference files** - For files longer than 100 lines, include a table of contents at the top so Claude can see the full scope when previewing.

Skill Creation Process

Skill creation involves these steps:

1. Understand the skill with concrete examples
2. Plan reusable skill contents (scripts, references, assets)
3. Initialize the skill (run `init_skill.py`)
4. Edit the skill (implement resources and write SKILL.md)
5. Package the skill (run `package_skill.py`)
6. Iterate based on real usage

Follow these steps in order, skipping only if there is a clear reason why they are not applicable.

Step 1: Understanding the Skill with Concrete Examples

Skip this step only when the skill's usage patterns are already clearly understood. It remains valuable even when working with an existing skill.

To create an effective skill, clearly understand concrete examples of how the skill will be used. This understanding can come from either direct user examples or generated examples that are validated with user feedback.

For example, when building an image-editor skill, relevant questions include:

- “What functionality should the image-editor skill support? Editing, rotating, anything else?”
- “Can you give some examples of how this skill would be used?”
- “I can imagine users asking for things like ‘Remove the red-eye from this image’ or ‘Rotate this image’. Are there other ways you imagine this skill being used?”
- “What would a user say that should trigger this skill?”

To avoid overwhelming users, avoid asking too many questions in a single message. Start with the most important questions and follow up as needed for better effectiveness.

Conclude this step when there is a clear sense of the functionality the skill should support.

Step 2: Planning the Reusable Skill Contents

To turn concrete examples into an effective skill, analyze each example by:

1. Considering how to execute on the example from scratch
2. Identifying what scripts, references, and assets would be helpful when executing these workflows repeatedly

Example: When building a *pdf-editor* skill to handle queries like “Help me rotate this PDF,” the analysis shows:

1. Rotating a PDF requires re-writing the same code each time
2. A *scripts/rotate_pdf.py* script would be helpful to store in the skill

Example: When designing a *frontend-webapp-builder* skill for queries like “Build me a todo app” or “Build me a dashboard to track my steps,” the analysis shows:

1. Writing a frontend webapp requires the same boilerplate HTML/React each time
2. An *assets/hello-world/* template containing the boilerplate HTML/React project files would be helpful to store in the skill

Example: When building a *big-query* skill to handle queries like “How many users have logged in today?” the analysis shows:

1. Querying BigQuery requires re-discovering the table schemas and relationships each time
2. A *references/schema.md* file documenting the table schemas would be helpful to store in the skill

To establish the skill’s contents, analyze each concrete example to create a list of the reusable resources to include: scripts, references, and assets.

Step 3: Initializing the Skill

At this point, it is time to actually create the skill.

Skip this step only if the skill being developed already exists, and iteration or packaging is needed. In this case, continue to the next step.

When creating a new skill from scratch, always run the *init_skill.py* script. The script conveniently generates a new template skill directory that automatically includes everything a skill requires, making the skill creation process much more efficient and reliable.

Usage:

Structured Template

- *scripts/init_skill.py* –path

The script:

- Creates the skill directory at the specified path
- Generates a *SKILL.md* template with proper frontmatter and TODO placeholders
- Creates example resource directories: *scripts/*, *references/*, and *assets/*
- Adds example files in each directory that can be customized or deleted

After initialization, customize or remove the generated SKILL.md and example files as needed.

Step 4: Edit the Skill

When editing the (newly-generated or existing) skill, remember that the skill is being created for another instance of Claude to use. Include information that would be beneficial and non-obvious to Claude. Consider what procedural knowledge, domain-specific details, or reusable assets would help another Claude instance execute these tasks more effectively.

Learn Proven Design Patterns

Consult these helpful guides based on your skill's needs:

- **Multi-step processes:** See references/workflows.md for sequential workflows and conditional logic
- **Specific output formats or quality standards:** See references/output-patterns.md for template and example patterns

These files contain established best practices for effective skill design.

Start with Reusable Skill Contents

To begin implementation, start with the reusable resources identified above: *scripts/*, *references/*, and *assets/* files. Note that this step may require user input. For example, when implementing a *brand-guidelines* skill, the user may need to provide brand assets or templates to store in *assets/*, or documentation to store in *references/*.

Added scripts must be tested by actually running them to ensure there are no bugs and that the output matches what is expected. If there are many similar scripts, only a representative sample needs to be tested to ensure confidence that they all work while balancing time to completion.

Any example files and directories not needed for the skill should be deleted. The initialization script creates example files in *scripts/*, *references/*, and *assets/* to demonstrate structure, but most skills won't need all of them.

Update SKILL.md

Writing Guidelines: Always use imperative/infinite form.

Frontmatter

Write the YAML frontmatter with *name* and *description*:

- *name*: The skill name
- *description*: This is the primary triggering mechanism for your skill, and helps Claude understand when to use the skill.
 - Include both what the Skill does and specific triggers/contexts for when to use it.
 - Include all “when to use” information here - Not in the body. The body is only loaded after triggering, so “When to Use This Skill” sections in the body are not helpful to Claude.
 - Example description for a *docx* skill: “Comprehensive document creation, editing, and analysis with support for tracked changes, comments, formatting preservation, and text extraction. Use when Claude needs to work

with professional documents (.docx files) for: (1) Creating new documents, (2) Modifying or editing content, (3) Working with tracked changes, (4) Adding comments, or any other document tasks”

Do not include any other fields in YAML frontmatter.

Body

Write instructions for using the skill and its bundled resources.

Step 5: Packaging a Skill

Once development of the skill is complete, it must be packaged into a distributable .skill file that gets shared with the user. The packaging process automatically validates the skill first to ensure it meets all requirements:

Structured Template

- scripts/package_skill.py

Optional output directory specification:

Structured Template

- scripts/package_skill.py ./dist

The packaging script will:

1. **Validate** the skill automatically, checking:
 - YAML frontmatter format and required fields
 - Skill naming conventions and directory structure
 - Description completeness and quality
 - File organization and resource references
2. **Package** the skill if validation passes, creating a .skill file named after the skill (e.g., *my-skill.skill*) that includes all files and maintains the proper directory structure for distribution. The .skill file is a zip file with a .skill extension.

If validation fails, the script will report the errors and exit without creating a package. Fix any validation errors and run the packaging command again.

Step 6: Iterate

After testing the skill, users may request improvements. Often this happens right after using the skill, with fresh context of how the skill performed.

Iteration workflow:

1. Use the skill on real tasks
2. Notice struggles or inefficiencies
3. Identify how SKILL.md or bundled resources should be updated
4. Implement changes and test again

3.44 Skill: `xlsx-author`

File Metadata

Field	Value
name	xlsx-author
description	Produce a .xlsx file on disk (headless) instead of driving a live Excel workbook — for managed-agent sessions with no open Office app.

`xlsx-author`

Use this skill when running **headless** (managed-agent / CMA mode) and you need to deliver an Excel workbook as a **file artifact** rather than editing a live workbook via `mcp__office__excel_*`.

Output contract

- Write to `./out/.xlsx`. Create `./out/` if it does not exist.
- Return the relative path in your final message so the orchestration layer can collect it.

How to build the workbook

Write a short Python script and run it with Bash. Use `openpyxl`:

Structured Template

- `from openpyxl import Workbook`
- `from openpyxl.styles import Font, PatternFill`
- `wb = Workbook()`
- `ws = wb.active; ws.title = "Inputs"`
- `ws["B2"] = "Revenue"; ws["C2"] = 1_250_000_000`
- `ws["C2"].font = Font(color="0000FF")` # blue = hardcoded input
- `calc = wb.create_sheet("DCF")`
- `calc["C5"] = "=Inputs!C2*(1+Inputs!C3)"` # black = formula
- `wb.save("./out/model.xlsx")`

Conventions (mirror *audit-xls*)

- **Blue / black / green.** Blue = hardcoded input, black = formula, green = link to another sheet/file.
- **No hardcodes in calc cells.** Every calculation cell is a formula; every input lives on an Inputs tab.

- **Named ranges** for any value referenced from a deck or memo.
- **Balance checks.** Include a Checks tab that ties (BS balances, CF ties to cash, etc.) and surfaces TRUE/FALSE.
- **One model per file.** Do not append to an existing workbook unless explicitly asked.

When NOT to use

If *mcp__office__excel_** tools are available (Cowork plugin mode), use those instead — they drive the user’s live workbook with review checkpoints. This skill is the file-producing fallback for headless runs.
